Heidelberg University

Department of Physics and Astronomy

Lecture Notes

Fundamentals of Numerical Programming I

Leonard Storcks

February 1, 2026

Abstract

For numerous mathematical and scientific problems, from integration to optimization and data analysis, analytical approaches carried out by hand are insufficient. Numerical methods can open new viewing angles and solution strategies to these problems. In this course we cover some *fundamentals of numerical programming* including numerical solutions to linear systems, numerical derivatives, optimization and numerical integration.

Contents

1 Introduction

From the Gaussian integral $\int_a^b \exp(-x^2)\, dx$ and the motion of three bodies to fluid dynamics and weather prediction, many problems from mathematics, science and engineering don't have *nice analytical solutions*, i.e. compact expressions comprised of elementary functions.

The ability of computers to rapidly perform simple arithmetic operations opens new angles of attack for such problems.[1] Let us approximate the Gaussian integral only using elementary arithmetic operations. We can approximate the exponential function with a truncated power series

$$\exp(x) = \sum_{k=0}^{\infty} \frac{x^k}{k!} \approx \sum_{k=0}^{N_{\text{order}}} \frac{x^k}{k!} \tag{1}$$

and the integral with

$$\int_a^b f(x)\, dx \approx \sum_{i=1}^{N_{\text{eval}}} f(x_i)\Delta x, \quad \Delta x = \frac{b-a}{N_{\text{eval}}}, \quad x_i = a + i\,\Delta x \tag{2}$$

The approximation of the integral is illustrated in Fig. 1, `Python` code on the example is given in Code-Snippet 1.

> **Question to the reader:** How can we control the accuracy of our approximation to $\int_a^b \exp(-x^2)\, dx$?

However, numerical algorithms and computation are not without caveats. For example, the precision with which we can handle real numbers is limited by storage and compute. This has very profound implications all through numerics, e.g. bounds derived under exact arithmetic might not hold under approximate arithmetic on the computer.

Our plan for the course is as follows:

1. we will introduce how numbers are represented digitally and raise awareness for the finite precision of floating point operations

2. we will discuss how linear systems can be solved numerically

3. we will introduce finite difference approximations to derivatives, discuss truncation

[1] For reference, the ENIAC computer developed in the Second World War could perform around 500 simple operations per second while a modern GPU can perform more than 10^{12} of such *floating-point operations* per second.

```python
import math
import numpy as np

def exp_series(x, n_terms=20):
    """
    Approximate exp(x) by a truncated series.
    """
    result = 1.0
    term = 1.0

    for k in range(1, n_terms):
        term = term * x / k
        result = result + term

    return result

def integrate(func, a, b, n_points=10_000):
    """
    Simple integral approximation, function
    evaluated at left interval edges.
    """

    dx = (b - a) / n_points
    x = a + dx * np.arange(n_points)

    return np.sum(func(x) * dx)

# approximate the integral of exp(-x^2)
# from a to b
func = lambda x: exp_series(-x ** 2)
a, b = 0.0, 1.0
approx = integrate(func, a, b)
```

Code-Snippet 1: Python code for approximating $\int_a^b \exp\left(-x^2\right) dx$.

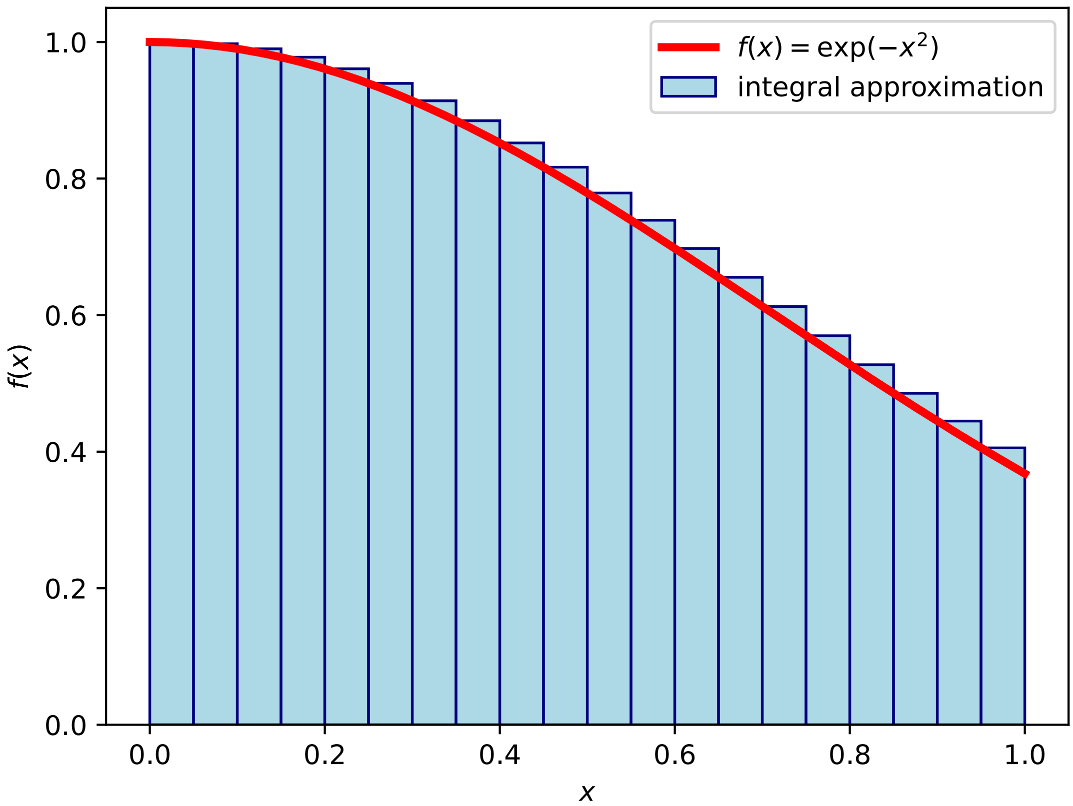


Figure 1: Approximation of the integral $\int_a^b \exp(-x^2)\,dx$.

errors, hint at powerful applications of finite differencing to differential equations and explain interpolation techniques

4. we will discuss numerical solutions to non-linear equations and optimization problems

5. we will go through methods for numerical integration

The methods you will learn in this course can empower you to solve a wide range of problems - and we're excited to see what you will use them on :)

2 Digital Representation of Numbers

We want to operate on numbers on a computer where storage is finite, and fixed-size representations in terms of discrete states are sought for efficient operations. However, the natural numbers are a countably infinite set and the real numbers are an uncountably infinite set. There is simply no finite-size encoding which could exactly represent every number, let's say every real between 0 and 1 - a fundamental error source of numerics. These restrictions result in caveats.

In C, `int x = 100 * 200 * 300 * 400;` will surprisingly yield -1894967296 (for a 32-bit integer representation) (*overflow*), `float f1 = (2.3 + 1e20) - 1e20;` yields 0.0 (*round-off-error*) but `float f2 = 2.3 + (1e20 - 1e20);` correctly gives 2.3.

We want to understand and mitigate such caveats of arithmetic on computers, where we want quick calculations while also using as little storage as possible.

Further details can be found in Higham, 2002 and Bryant and O'Hallaron, 2011.

2.1 Integer Arithmetic

In C, integers ($\in \mathbb{Z}$) are stored as fixed-size bit-sequences. While for us humans with commonly 10 fingers a base-10 number system is natural, two-state systems are physically easier to construct and reliably operate on (e.g. a transistor conducts or not) (also see Higham, 2002, Sec. 2.7). Therefore, we are dealing with binary (base-2) representations throughout this chapter.

The respective size of the bit-sequence dictates a range. C provides both unsigned and signed integer types, with negative numbers in the signed case represented using *two's-complements*.

2.1.1 Unsigned integers

For a bit-vector $\underline{x} = [x_{\omega-1}, x_{\omega-2}, \ldots, x_0]$ the unsigned conversion is

$$B2U_\omega := \sum_{i=0}^{\omega-1} x_i 2^i \tag{3}$$

(illustrated in Fig. 2) and the range is $0, \ldots, 2^\omega - 1$. In case of overflow in operations, the overflow is truncated in the most significant bits (see Fig. 3). As $B2U_\omega[x_{\omega-1}, x_{\omega-2}, \ldots, x_0] \mod 2^k = B2U_\omega[x_{k-1}, x_{k-2}, \ldots, x_0]$ we effectively store arithmetic result $\mod 2^\omega$.

$x_{\omega-1}$ is called most significant bit (MSB) and x_0 least significant bit (LSB).

Problem: When the result of an arithmetic operation exceeds the range of an integer type, unexpected results occur (overflow) (and also underflow in the signed case)

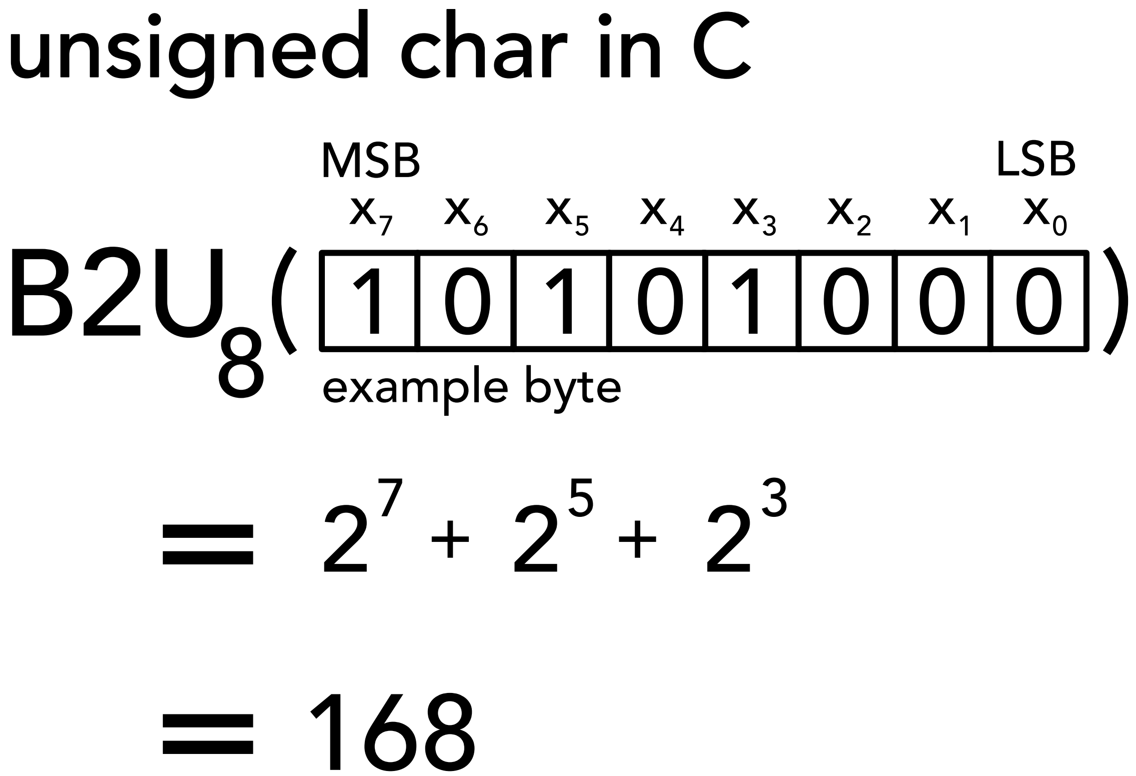


Figure 2: Example of an unsigned char in C.

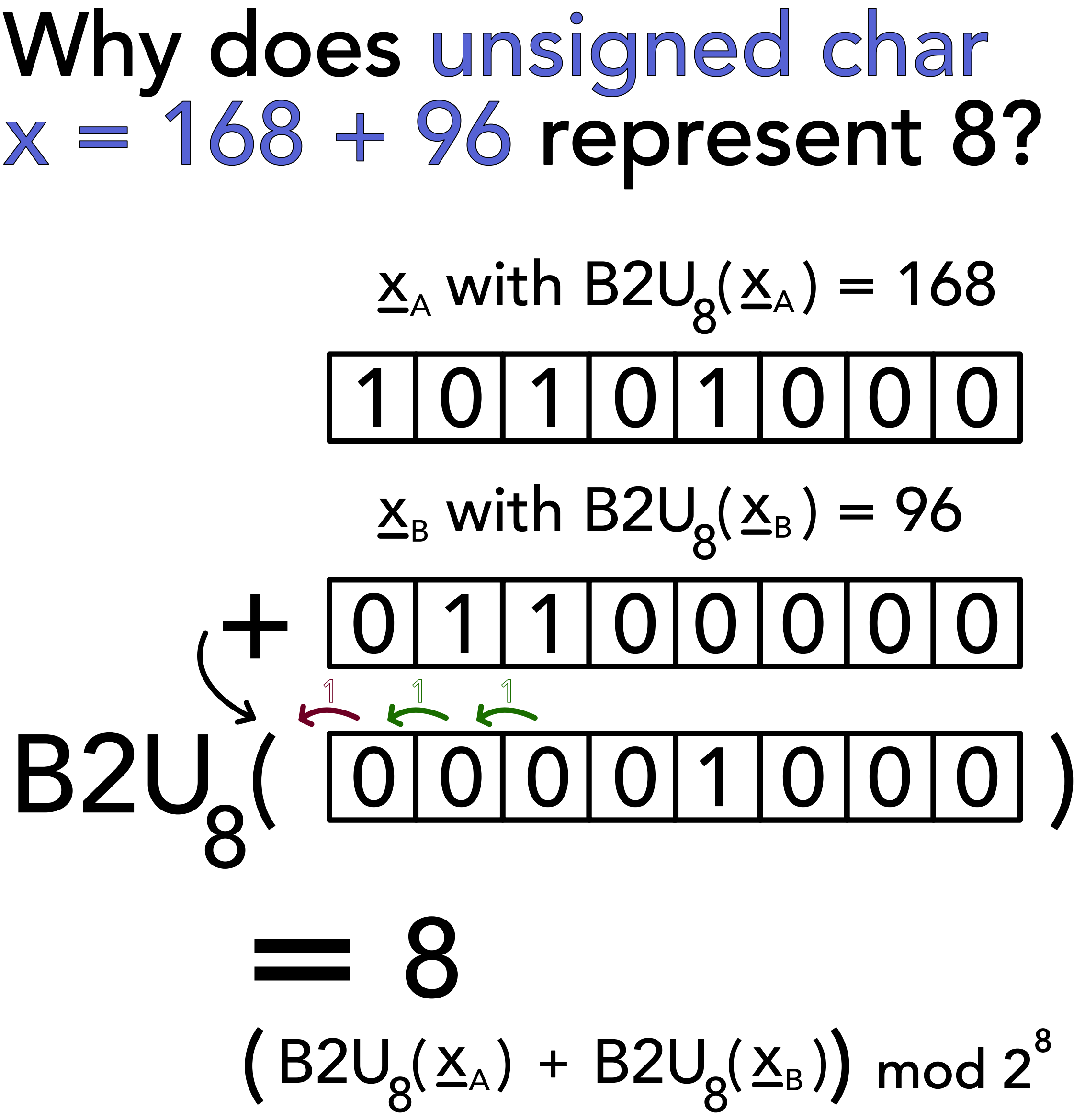


Figure 3: Example of unsigned char overflow.

2.1.2 Two's complement for negative numbers

Note that addition of integers by bitwise addition with carry-on is very fast.

Why can't we just let the MSB encode the sign?: A representation of the form

$$B2S_\omega(\underline{x}) := (-1)^{x_{\omega-1}} \cdot \sum_{i=0}^{\omega-2} x_i 2^i \tag{4}$$

(sign magnitude) has the disadvantage, that normal bitwise addition with carry-on does not work. Also, zero is encoded twice, as ± 0.

Idea of the two's-complement: The MSB flags the sign in $\underline{x} = [x_{\omega-1}, x_{\omega-2}, \ldots, x_0]$ by having a weighting factor $-2^{\omega-1}$

$$B2T_\omega(\underline{x}) := -x_{\omega-1}2^{\omega-1} + \sum_{i=0}^{\omega-2} x_i 2^i \tag{5}$$

Carry-on to the ω-th bit in addition is again ignored. As we really just have to add positive numbers in bits x_0 to $x_{\omega-2}$ with correct sign-switch by carry-on, no special handling is necessary (see Fig. 4). The range is $-2^{\omega-1} \ldots 2^{\omega-1} - 1$.

From an unsigned int to the two's complement and vice versa we can get by

1. invert all bits
2. add +1 to the result

These rules intuitively follow from the constraint, that bitwise addition with carry-on of $-u$ and u should result in an all-zero bitvector. Adding a bitvector to its inverted self results in an all-1 bitvector, adding one more then results in all zeros, as the last carry-on is discarded.

If we want $\underline{x}$ with $B2T_\omega(\underline{x}) = -u$ then the bits following the MSB must encode k with $-u = -2^{\omega-1} + k$, so the encoded unsigned number must be $B2T_\omega(\underline{x}) = \underbrace{2^{\omega-1}}_{\text{sign bit}} + k = 2^\omega - u$ - a *two's complement*.

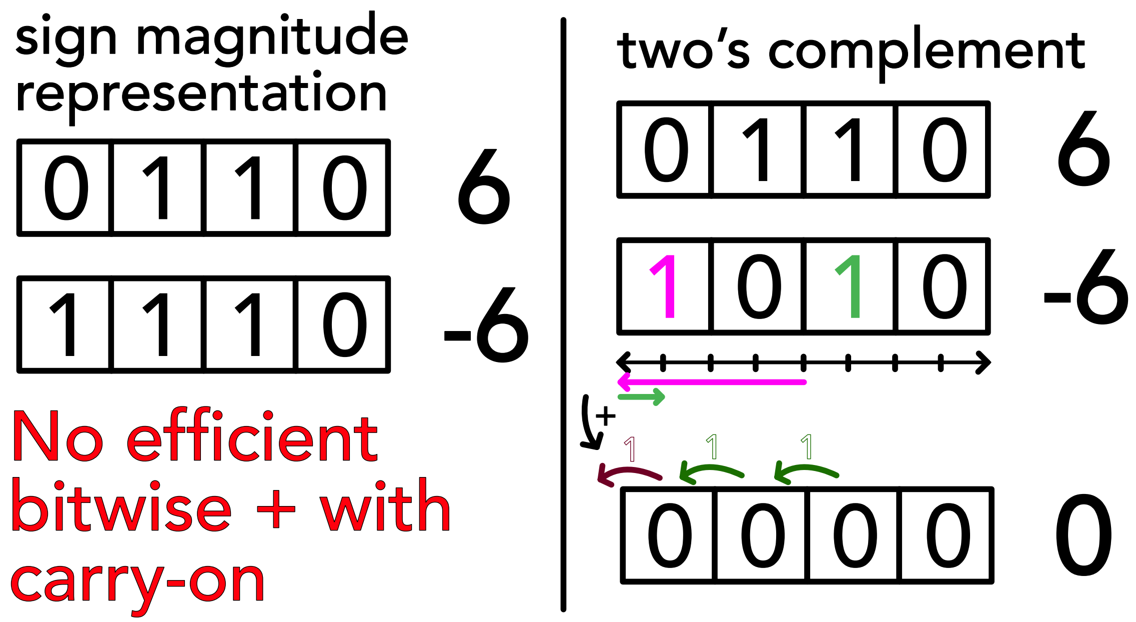


Figure 4: Illustration of the defining feature of the two's complement - addition is simple.

2.1.3 Integer types in C

Some common integer types with respective minimum sizes are given in table 1.

Type	Minimum Size ω
`char`	8 bits
`short`	16 bits
`int`	16 bits
`long`	32 bits
`long long`	64 bits

Table 1: Common integer types in C, signed ranges are $-2^{\omega-1} \dots 2^{\omega-1} - 1$, unsigned ranges are $0 \dots 2^{\omega} - 1$.

Problem: Using unsigned can come with unexpected results, when being cast from a negative number. E.g. `unsigned int x = -1;` will result in $2^{32} - 1$ in a 32-bit system (the two's complement is read as if an unsigned representation). More deviously, $-1 < 0U$ will evaluate to false, as all integers in a comparison are cast to unsigned, when one of them is unsigned.

2.1.4 Properties and caveats of integer arithmetic

Typical problems in integer arithmetic are overflow, integer division, the modulo operation and implicit type conversion.

Overflow: The respective integer ranges have finite ranges, overflow in arithmetic operations is cut-off. We must choose a type with sufficient range - mind that choosing types with too large of a footprint (e.g. always long) wastes storage and compute.

Example: In `char c = 100 * 4; // range -128 to 127` the result is -112 as 400 in bits is 000110010000 where a cut-off to one byte means 10010000 so $-2^7+2^4 = -128+16 = -112$.

Integer division: All decimal places are truncated, so

$$5/3 = 1; -5/3 = -1; 5/-3 = -1$$

2.1.5 Can there be integer-overflow in python?

Note that in python3, integers are implemented as "long" integer objects of arbitrary size, overflows are impossible (at the cost of speed; mind that e.g. numpy is based on C code).

2.2 Floating Point Arithmetic

In the following, we will encode numbers in the form $V = x \cdot 2^y$, with $x, y \in \mathbb{Z}$. Similar to the decimal notation

$$d_m d_{m-1} \dots d_0.d_{-1}d_{-2} \dots d_{-p} = \sum_{i=-p}^{m} d_i 10^i \tag{6}$$

we can write in binary notation

$$b_m b_{m-1} \dots b_0.b_{-1}b_{-2} \dots b_{-p} = \sum_{i=-p}^{m} b_i 2^i, \quad 0.01_2 = 0.25_{10} \tag{7}$$

or generally in base β in scientific notation

$$\begin{gathered}(-1)^s \cdot \underbrace{b_0.b_{-1}b_{-2} \dots b_{-p}}_{\text{mantissa M}} \cdot \beta^e = \beta^e \cdot \sum_{i=-p}^{0} b_i \beta^i, \quad \text{exponent } e, \\ \text{precision with implicit first bit } p+1, \quad \text{sign-bit } s \in \{0, 1\}\end{gathered} \tag{8}$$

Note that in this notation (in the form $V = x \cdot 2^y$) we cannot exactly represent e.g. 0.1 or 0.2.

$$0.1_{10} = 1.10011[0011]\dots_2 \cdot 2^{-4} \tag{9}$$

Historic example: In the first Gulf War (more specifically on 25th February 1991), a Patriot missile defense battery failed to intercept an incoming Iraqi Scud missile, because it used an internal clock counting up in tenths of a second represented by a 23-bit sequence. Future missile positions are predicted by extrapolating from past position with constant velocity. The Patriot system mixed both this inaccurate internal clock and a more accurate one, leading to the failed interception and 28 deaths among soldiers.

2.2.1 An intuition for limited precision in fixed-size scientific notation

Consider we want to design a fixed-size representation of real numbers, let's say we can store six decimal digits and two signs. We choose our format to be

$$\pm d_1.d_2d_3d_4 \cdot 10^{\pm d_5 d_6}, \quad \text{e.g. } +9.821 \cdot 10^{-12} \tag{10}$$

Task to the reader: Think about the range of this representation and the number closest to 0 we can represent.

The tiniest number ($\neq 0$) we can represent is

$$+0.001 \cdot 10^{-99} \tag{11}$$

However, consider we have stored 1 then the smallest increment we can resolve in the mantissa is 0.001 - **our precision is limited and relative!**

Let us now go from my naive example to a proper floating point representation.

2.2.2 IEEE 754 floating point standard

Based on the representation in scientific notation, we can store a floating point number in a bit-vector with

- a sign bit s
- an exponent e stored as an unsigned integer $E = e + b$ with bias b
- a mantissa M

where for single precision (32-bit)

- the exponent is stored in 8 bits, $b = 127$, $e_{min} = -126$, $e_{max} = 127$ with $E = 255$ and $E = 0$ reserved for special cases
- the mantissa is stored in 23 bits, with the first bit being implicitly 1 (**normalization**), which can always be assumed by appropriately choosing the exponent (floating point representations are not unique), so we have $p = 23(+1)$ bits of precision encoding an integer M but need a special representation for 0

where based on the exponent we differentiate between

- **normalized values for** $1 \leq E \leq 254$ with value of the floating point number

$$\begin{gathered} V = (-1)^s \cdot \left(1 + \frac{M}{2^p}\right) \cdot 2^{E-b}, \quad \text{sign bit } s \\ \text{biased exponent } E = e + b, \quad \text{integer representation of mantissa } M \\ \text{precision } p = \#\text{mantissa bits, not including the 1 implicit bit} \end{gathered} \tag{12}$$

- **denormalized values for** $E = 0$ with value of the floating point number

$$V = (-1)^s \cdot \frac{M}{2^p} \cdot 2^{-b+1}, \quad \text{for } M = 0, V = \pm 0 \text{ depending on } s \tag{13}$$

  The denormalized numbers start just below the normalized ones (by the factor 2^{-b+1} with +1 as we do not have the implicit leading 1 here) and now no normalization (implicit starting 1-bit) is assumed. This allows to approach zero with gradually decreasing precision (and even spacing) (smaller numbers occupy fewer digits as of the leading zeros) and reduces the risk of underflow.

- **$\pm\infty$ for $E = 255$ and $M = 0$**
- **NaN (Not a Number) for $E = 255$ and $M \neq 0$**

Why is the exponent stored in a biased way, not two's complement?: In a two's complement representation of the exponent to compare two numbers, we have to compare the exponents and mantissas separately and the exponent-comparison is a bit more complicated than comparing biased representations. In the biased exponent representation we can just compare the bitvectors of exponent followed by mantissa interpreted as integers (also mind the sign).

A specific example of a floating point representation is given in Fig. 5.

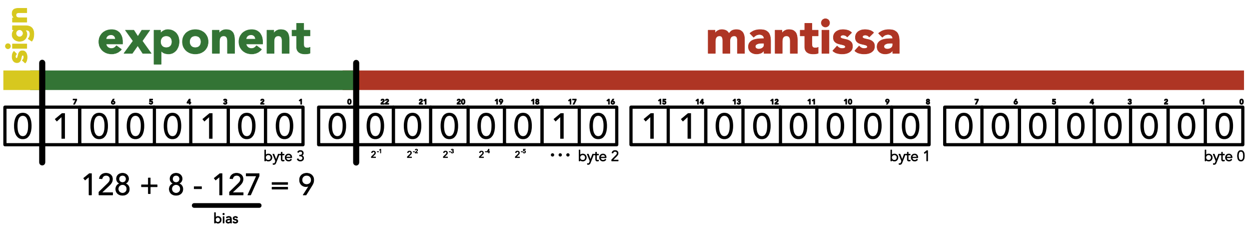


Figure 5: 523 can be written as $1.000001011_2 \cdot 2^9$ so $e = 9, E = e + 127 = 136$. As of the normalization, the leading 1 in the mantissa is implicit. The number is given as a single precision float in big endian.

2.2.3 Only a finite set of floating point numbers can be represented exactly

With 32 bits, 2^{32} states can be encoded (and mind that here e.g. NaN has multiple representations). In any case, the number of exactly representable numbers is finite, above 0 starting at

$$V_{\text{denorm,min}} = \frac{1}{2^p} \cdot 2^{-b+1} \underbrace{\approx}_{\substack{\text{single}\\\text{p.}}} 1.4 \cdot 10^{-45} \tag{14}$$

with the smallest normalized number being

$$V_{\text{norm,min}} = (1+0) \cdot 2^{-b} \underbrace{\approx}_{\substack{\text{single}\\\text{p.}}} 1.2 \cdot 10^{-38} \tag{15}$$

and the largest normalized number being

$$V_{\text{norm,max}} = (1 + \frac{2^p - 1}{2^p}) \cdot 2^{E_{\max}-b} \underbrace{\approx}_{\substack{\text{single} \\ \text{p.}}} 3.4 \cdot 10^{38} \tag{16}$$

2.2.4 Machine precision is finite

The smallest increment in the mantissa, in $1 + \frac{M}{2^p}$, is the **machine precision**

$$\epsilon_{\text{mach}} = \frac{1}{2^p} \underbrace{\approx}_{\substack{\text{single} \\ \text{p.}}} 1.2 \cdot 10^{-7} \tag{17}$$

Consider two floating point numbers $V_1, V_2 > 0$ *next to each other*

$$V_1 = \left(1 + \frac{M}{2^p}\right) \cdot 2^{E-b}, \quad V_2 = \left(1 + \frac{M+1}{2^p}\right) \cdot 2^{E-b} \tag{18}$$

so their relative difference is bound by

$$\frac{V_2 - V_1}{V_2} = \frac{\frac{1}{2^p} \cdot 2^{E-b}}{\left(1 + \frac{M+1}{2^p}\right) \cdot 2^{E-b}} \leq \epsilon_{\text{mach}} \tag{19}$$

Finite floating point precision is one of the fundamental error sources in numerics.

Given x is in the range of representable floating point numbers, then the floating point representation

$$fl(x) = x \cdot (1 + \delta), \quad |\delta| \leq \epsilon_{\text{mach}} \tag{20}$$

(Higham, 2002, Theorem 2.2). The modified formulation

$$fl(x) = \frac{x}{1 + \delta}, \quad |\delta| \leq \epsilon_{\text{mach}} \tag{21}$$

also holds (Higham, 2002, Theorem 2.2). For multiple floating point operations the following result can be helpful (Higham, 2002, Lemma 3.1): If $|\delta_i| \leq \epsilon_{\text{mach}}$ and $\rho_i \in \{+1, -1\}$ for $i = 1, ..., l$ and $l\epsilon_{\text{mach}} < 1$ then

$$\prod_{i=1}^{l}(1+\delta_i)^{\rho_i} = 1+\theta_l, \quad |\theta_l| \leq \frac{l\epsilon_{\text{mach}}}{1-l\epsilon_{\text{mach}}} =: \gamma_l \tag{22}$$

2.2.5 Rounding and pitfalls of floating point arithmetic

In the IEEE standard, results of addition, subtraction, multiplication and division must equal to one where the arithmetic operations are assumed to be exact and then there is rounding to the nearest representable number (computation is done at higher (typically double or higher) precision). Therefore (mind the section before)

$$\text{relative error } \frac{|x-\hat{x}|}{|x|} \leq \epsilon_{\text{mach}}, \quad \text{number } x, \text{ number on machine } \hat{x} \tag{23}$$

with the common pitfalls

- **Limitation of machine precision:** $a+b=a$ typically for $|b| < \epsilon_{\text{mach}}|a|$, i.e. when b cannot be resolved by the mantissa of a, e.g. $(1+0.5\epsilon)-0.5\epsilon$ in floating point arithmetic yields 0.9999999999999999 not 1.
- **Associativity is not guaranteed:** $(a+b)+c \underbrace{\neq}_{\text{i.A.}} a+(b+c)$, e.g. $(2.3+1e20)-1e20$ yields 0.0 but $2.3+(1e20-1e20)$ yields 2.3
- **Problems of representability:** As e.g. 0.1 is not exactly representable in base $\beta=2$, $x/10 \neq 0.1 \cdot x$ in general while $x/2.0 = 0.5 \cdot x$ is exact. The compiler may automatically choose the multiplication variant as multiplication is faster than division.
- **Cancellation:** For $x=a-b$ subtractive cancellation causes relative errors already present in $\hat{a}$ and $\hat{b}$ to be (relatively) amplified, when a and b are of similar size. Significant digits are lost and the relative error explodes. Consider e.g. $a=1.75682, b=1.75471$ with $\hat{a}=1.76$ and $\hat{b}=1.75$. While $\hat{a},\hat{b}$ have small relative errors (3 digits precision[2] Note that here 0.123 ans 0.127 agree in two significant digits, where one intuitively might say this should be rather 1.), $a-b=0.00211$ and $\hat{a}-\hat{b}=0.01$ has a large relative error (no precise digit).
- **NaN and Inf:** All calculations including NaN yield NaN, calculations with inf mostly inf (except of course $1/\text{inf}=0$, ...)

where we should

[2] $\hat{x}$ is said to approximate x to the r-th digit, if the absolute error is at most $\frac{1}{2}$ in the r-th digit, so

$$\text{largest integer s so that } 10^s < |x|, \quad |x-\hat{x}| < 1/2 \cdot 10^{s-r+1} \tag{24}$$

- **Rewrite calculations so that errors do not amplify:** For $x = 10^8, y = 10^5, z = -1 - 10^5$ we have $xy + xz = -1.0066 \cdot 10^8$ (problem in resolving the -1 in xz) but $x(y + z) = -1.0 \cdot 10^8$.
- **Compare floats based on a maximum relative error:** Instead of $x == y$ we should use e.g. $|x - y| \leq \epsilon_{\text{mach}} \cdot \max(|x|, |y|)$.
- **Avoid overflow to inf and nan:** E.g. in `float x = 1e20; float y = x * x; float z = y / x` z will be inf.

2.2.6 Rewriting expressions to avoid cancellation I

Consider $f(x) = \frac{1-\cos x}{x^2}$. For $x_e = 10^{-4}$, we have (when we represent 6 figures)

$$c := \cos x_e, \quad \hat{c} = 0.999999, \quad 1 - \hat{c} = 10^{-6}, \quad \text{while } 1 - c \approx 5 \cdot 10^{-9} \tag{25}$$

$1 - c$ has only one significant digit, the relative error is enormously amplified and we get

$$\frac{1 - \hat{c}}{x_e^2} = 100, \qquad \text{while in reality for } x \neq 0 : 0 \leq f(x) \leq \frac{1}{2} \tag{26}$$

To avoid cancellation we can rewrite using $\cos x = \cos^2 \frac{x}{2} - \sin^2 \frac{x}{2} = 1 - 2\sin^2 \frac{x}{2}$ to

$$f(x) = \frac{1}{2} \left(\frac{\sin \frac{x}{2}}{\frac{x}{2}} \right)^2 \tag{27}$$

A comparison of both versions in python can be found in Fig. 6, at some point (roughly below 10^{-8}), $\cos x$ is too close to 1 to be resolved by the mantissa (consider $\cos x = 1 - \frac{x^2}{2} + \ldots$, for $x \sim 10^{-8}$, the x^2 term falls below what machine precision can resolve), so the total result is 0, above that the magnified error is visible.

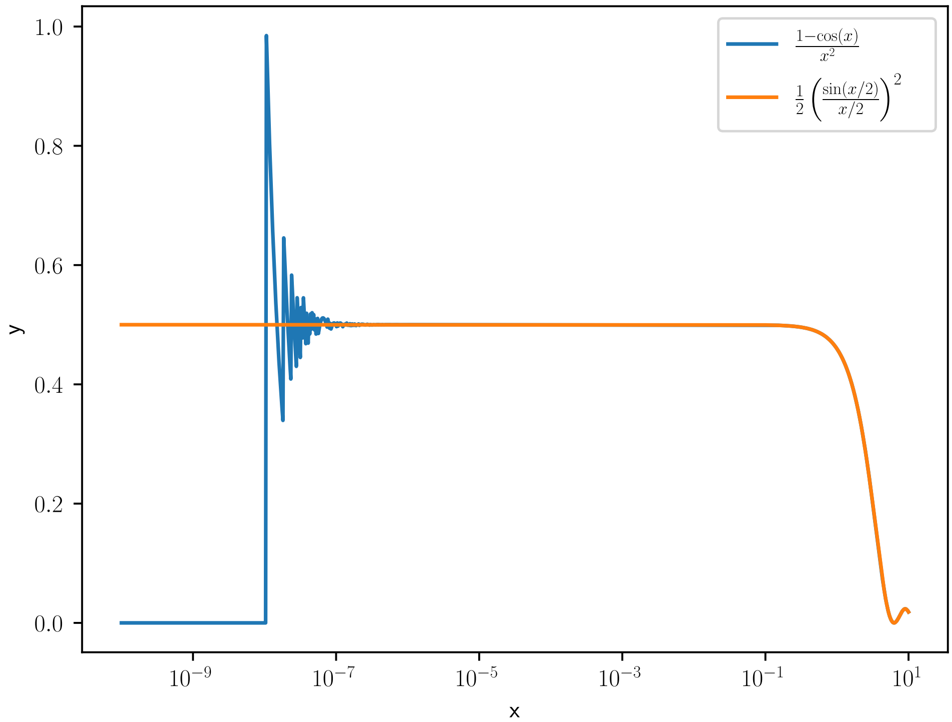


Figure 6: Comparison of the mathematically equivalent expressions $f(x) = \frac{1-\cos x}{x^2}$ and $f(x) = \frac{1}{2}\left(\frac{\sin\frac{x}{2}}{\frac{x}{2}}\right)^2$ in python.

2.2.7 Rewriting expressions to avoid cancellation II

Consider the following expressions for the sample variance of $\{x_i\}_{i=1}^N$

$$\begin{aligned}\text{two-pass formula: } \bar{x} &= \frac{1}{N}\sum_{i=1}^{N-1} x_i, \quad \sigma_N^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^2 \\ \text{one-pass formula: } \sigma_N^2 &= \frac{1}{N}\left(\sum_{i=1}^{N} x_i^2 - \frac{1}{N}\left(\sum_{i=1}^{N} x_i\right)^2\right) \\ &\text{as } \sigma^2 = E[(x-\bar{x})^2] = E[x^2]-E[x]^2\end{aligned} \tag{28}$$

While for the one-pass formula, we can calculate all necessary sums in one pass through the data, it suffers heavily from cancellation: For $\{10000, 10001, 10002\}$, the two-pass formula in single precision correctly gives 1.0 while the one-pass formula yields 0.0 (cancellation) (there are better one-pass formulas though).

2.2.8 Accumulation of round-off errors

Consider the sum

$$\sum_{k=1}^{\infty} k^{-2} = \frac{\pi^2}{6} \tag{29}$$

which we want to approximate by finitely many summands. If we sum the terms just as the formula suggests from large to small, at some point, the small changes will not be resolved anymore - so better sum up from small to large. Summing up in single precision for $N = 10^7$ terms, one gets

$$\begin{aligned} &\text{big to small: } 1.644725323, \quad \text{small to big: } 1.644933939, \\ &\qquad\qquad \text{exact (till 9th digit): } 1.644934058 \end{aligned} \tag{30}$$

As expected the big-to-small summation is too small.

2.2.9 Higher precision

The above pitfalls are less severe in higher precision. For instance in double precision (64-bit)

$$\begin{aligned} &p = 52(+1) \text{ mantissa bits}, \quad 11 \text{ exponent bits, with } e_{min} = -1022, e_{max} = 1023, \\ &\qquad \text{smallest and largest repr. numbers } f_{min} \simeq 2.2 \cdot 10^{-308}, f_{max} \simeq 1.8 \cdot 10^{308}, \\ &\qquad\qquad |e_{min}| < |e_{max}| \rightarrow \frac{1}{f_{min}} < f_{max} \end{aligned} \tag{31}$$

with machine precision $\epsilon_{\text{mach}} \approx 2.2 \cdot 10^{-16}$. There is even quad-double precision (128-bit) (not supported on hardware though and therefore relatively slow as it has to be emulated). Packages for nearly arbitrary precision also exist.

2.3 A more general view on sources of numerical error

In numerical computation, the typical error sources are

- rounding
- data uncertainty
- truncation (of terms in numerical schemata, e.g. in the approximation of a function by its Taylor series)

where we have now discussed rounding errors and their effects to some extent.

2.4 Backward error, forward error and condition number

Consider we approximate $y = f(x)$ as $\hat{y}$ in an arithmetic of limited precision, with $f : \mathbb{R} \to \mathbb{R}$.

- **Forward error:** The absolute or relative error between $\hat{y}$ and y is the forward error (living in the output space)
- **Backward error:** The backward error is the smallest Δx (in the input space) so that $f(x + \Delta x) = \hat{y}$, so the smallest perturbation where the exact function gives our approximate result.

If this Δx is sufficiently small, e.g. as small as the uncertainty in the data in the first place, we speak of **backward stability**. A weaker formulation is the mixed forward-backward error

$$\hat{y} + \Delta y = f(x + \Delta x), \quad |\Delta y| \leq \epsilon |y|, \quad |\Delta x| \leq \eta |x| \tag{32}$$

In the context of rounding errors, we call the algorithm numerically stable if $\hat{y}$ is almost the right answer for almost the right data (ϵ, η small).

2.4.1 Conditioning

Backward and forward error are connected by the conditioning of a problem, the sensitivity of the solution to perturbations in the data. Assuming $\hat{y} = f(x + \Delta x)$ and f differentiable, we have

$$\hat{y} - y = f(x + \Delta x) - f(x) = f'(x)\Delta x + \mathcal{O}((\Delta x)^2) \tag{33}$$

so the relative error is

$$\frac{\hat{y} - y}{y} = \frac{f'(x)\Delta x}{f(x)} + \mathcal{O}((\Delta x)^2) \tag{34}$$

leading to the relative condition number

$$\kappa(x) \underbrace{=}_{\text{i.A.}} \lim_{\epsilon \to 0^+} \sup_{||\Delta x|| \leq \epsilon} \frac{||y - \hat{y}||/||y||}{||\Delta x||/||x||} = \left| \frac{x f'(x)}{f(x)} \right| \tag{35}$$

for small Δx measuring the relative change in the output over a relative change in the input. As a rule of thumb

$$\text{forward error} \lesssim \text{condition number} \cdot \text{backward error} \tag{36}$$

so ill-conditioned problems can have large forward errors. Note that the condition number is inherent to the problem not the numerical implementation.

Application on Matrices

Consider the linear system $\underline{\underline{A}}\,\underline{y} = \underline{x}, \underline{y} = \underline{\underline{A}}^{-1}\underline{x}, \hat{\underline{y}} = \underline{\underline{A}}^{-1}(\underline{x} + \underline{\Delta x})$. The condition number follows as

$$\begin{aligned}
\kappa(\underline{\underline{A}}) &= \max_{\underline{x},\underline{\Delta x}\neq 0} \frac{||\underline{\underline{A}}^{-1}\underline{x} - \underline{\underline{A}}^{-1}(\underline{x} + \underline{\Delta x})||/||\underline{\underline{A}}^{-1}\underline{x}||}{||\underline{\Delta x}||/||\underline{x}||} \\
&= \max_{\underline{\Delta x}\neq 0} \frac{||\underline{\underline{A}}^{-1}\underline{\Delta x}||}{||\Delta x||} \max_{\underline{x}\neq 0} \frac{||\underline{x}||}{||\underline{\underline{A}}^{-1}\underline{x}||} \\
&= \max_{\underline{\Delta x}\neq 0} \frac{||\underline{\underline{A}}^{-1}\underline{\Delta x}||}{||\underline{\Delta x}||} \max_{\underline{y}\neq 0} \frac{||\underline{\underline{A}}\,\underline{y}||}{||\underline{y}||} \\
&= ||\underline{\underline{A}}^{-1}|| \cdot ||\underline{\underline{A}}||
\end{aligned} \tag{37}$$

where we used the definition of the matrix norm $||\underline{\underline{A}}|| = \max_{\underline{x}\neq 0} \frac{||\underline{\underline{A}}\underline{x}||}{||\underline{x}||}$. For large condition numbers, small perturbations in the input $\underline{x}$ lead to large changes in the solution $\underline{y}$.

3 Solving Linear Systems

Solving linear systems $\underline{\underline{A}}\underline{x} = \underline{b}$ (invertible $\underline{\underline{A}} \in \mathbb{R}^{n \times n}$, $\underline{x} \in \mathbb{R}^n$, $\underline{b} \in \mathbb{R}^n$) is important throughout scientific computing and at the core of many numerical algorithms.

Numerical methods for solving linear systems include

- **Direct solvers: Gaussian elimination / factorization methods**: The linear system is transformed into a triangular form which can be trivially solved. Reaching the triangular form has a cost of $\mathcal{O}(n^3)$ operations which can be prohibitive for large matrices.
- **Iterative methods**: An iterative procedure is designed which converges to the solution of the linear system (a dynamical system with $\underline{x}$ as the steady state).
  - Classical iterative schemes include Jacobi and Gauss-Seidel iteration.
  - An important class of iterative methods are Krylov subspace methods (like *GMRES*) which do not require $\underline{\underline{A}}$ to be materialized, only matrix vector products $\underline{\underline{A}}\underline{v}$ are necessary. This is for instance very helpful when $\underline{\underline{A}}$ is a prohibitively large Jacobian.

  For large, especially for sparse, matrices, iterative methods typically outperform direct solvers.

We will cover Gaussian elimination, LU factorization and classical iterative schemes here, in the appendix you can find Krylov subspace methods (Section A.4) and multigrid techniques (Section A.5).

3.1 Why don't we typically explicitly invert matrices?

One can efficiently invert matrices numerically. Indeed, matrix multiplication and inversion are equivalently hard problems in terms of asymptotic runtime (Cormen et al., 2022, Sec. 28.2), such that the scaling of Strassen's algorithm, $\mathcal{O}(N^{\log_2 7})$ (Cormen et al., 2022, Sec. 4.2), applies.

However, while $\underline{\underline{A}} \in \mathbb{R}^{N \times N}$ might often be sparse (e.g. a sparse Jacobian), $\underline{\underline{A}}^{-1}$ is dense in general and our storage might not be able to handle N^2 terms of a dense inverse. »Note that some may say that this is done for numerical stability reasons: that is incorrect. In fact, under reasonable assumptions for how the inverse is computed, it will be as numerically stable as other techniques we will mention.« (Rackauckas et al., 2022, Ch. 6).

3.2 Example application of solving linear systems - linear regression

Given N measurements of p different *features* $x_{i1}, ..., x_{ip}$ (these might for instance be concentrations of different drugs administered to a patient) and outputs y_i^{data} (for instance the concentration of pathogens one day after drug administration), we want to find parameters $\beta_0, ..., \beta_p$ in the linear relation

$$y_i^{\text{predicted}} = \beta_0 + \sum_{k=1}^{p} \beta_k x_{ik} \tag{38}$$

such that $\sum_{i=1}^{N} ||y_i^{\text{predicted}} - y_i^{\text{data}}||^2$ is minimized.

In matrix notation with

$$\underline{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_i \\ \vdots \\ y_N \end{pmatrix}, \quad \underline{\underline{X}} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & \cdots & x_{Np} \end{bmatrix} \in \mathbb{R}^{N \times p} \tag{39}$$

$$\underline{\underline{X}} \text{ with 1 column to allow for constant offset } \beta_0$$

we can write our model as

$$\underline{y}^{\text{predicted}} = \underline{\underline{X}}\,\underline{\beta} \tag{40}$$

We can therefore estimate the *optimal* parameters with

$$\underline{\hat{\beta}} = \underset{\underline{\beta}}{\text{argmin}}\, ||\underline{y}^{\text{data}} - \underline{\underline{X}}\,\underline{\beta}||_2^2 \tag{41}$$

so

$$\partial_{\underline{\beta}} ||\underline{y}^{\text{data}} - \underline{\underline{X}}\,\underline{\beta}||_2^2 = -2\underline{\underline{X}}^T(\underline{y}^{\text{data}} - \underline{\underline{X}}\,\underline{\beta}) \underset{!}{=} 0 \tag{42}$$

and written as a linear system

$$\underline{\underline{X}}^T \underline{\underline{X}} \underline{\beta} = \underline{\underline{X}}^T \underline{y}^{\text{data}} \tag{43}$$

3.3 Direct solvers: Gaussian elimination and LU factorization

3.3.1 Introduction on an example

Let us start with a simple linear system

$$\begin{aligned} x_1 + x_2 &= 1 \\ x_1 - 2x_2 &= 4 \end{aligned} \tag{44}$$

We can eliminate x_1 from the second equation by subtracting the first equation, such that we obtain

$$\begin{aligned} x_1 + x_2 &= 1 \\ -3x_2 &= 3 \end{aligned} \tag{45}$$

which we can then solve bottom-up to $x_2 = -1$ and $x_1 = 2$. We can apply the same procedure to larger systems, like

$$\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 2 \\ 1 & 4 & 9 & 16 & 3 \\ 1 & 8 & 27 & 64 & 4 \end{array}\right) \tag{46}$$

where we wrote $\underline{\underline{A}}\underline{x} = \underline{b}$ in the form $(\underline{\underline{A}}|\underline{b})$.

We start by eliminating the elements in the first column below the diagonal

$$\begin{aligned} &\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 2 \\ 1 & 4 & 9 & 16 & 3 \\ 1 & 8 & 27 & 64 & 4 \end{array}\right) \times(-1) \rightarrow \left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 1 & 4 & 9 & 16 & 3 \\ 1 & 8 & 27 & 64 & 4 \end{array}\right) \times(-1) \rightarrow \\ &\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 3 & 8 & 15 & 2 \\ 1 & 8 & 27 & 64 & 4 \end{array}\right) \times(-1) \rightarrow \left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 3 & 8 & 15 & 2 \\ 0 & 7 & 26 & 63 & 3 \end{array}\right) \end{aligned} \tag{47}$$

continue with the second column

$$
\begin{aligned}
&\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 3 & 8 & 15 & 2 \\ 0 & 7 & 26 & 63 & 3 \end{array}\right) \times(-3) \rightarrow \\
&\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 0 & 2 & 6 & -1 \\ 0 & 7 & 26 & 63 & 3 \end{array}\right) \times(-7) \rightarrow \left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 0 & 2 & 6 & -1 \\ 0 & 0 & 12 & 42 & -4 \end{array}\right)
\end{aligned}
\tag{48}
$$

and finally we arrive at an upper triangular system

$$
\left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 0 & 2 & 6 & -1 \\ 0 & 0 & 12 & 42 & -4 \end{array}\right) \times(-6) \rightarrow \left(\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 0 & 2 & 6 & -1 \\ 0 & 0 & 0 & 6 & 2 \end{array}\right)
\tag{49}
$$

which we can solve bottom-up to $x_4 = \frac{1}{3}$, $2x_3 + 6x_4 = -1 \rightarrow x_3 = -\frac{3}{2}$, $x_2 + 2x_3 + 3x_4 = 1 \rightarrow x_2 = 3$ and $x_1 + x_2 + x_3 + x_4 = 1 \rightarrow x_1 = -\frac{5}{6}$.

3.3.2 General principle

Formulated abstractly, we

1. go through the columns of $\underline{\underline{A}}$ from left to right and eliminate all entries below the diagonal until we obtain an upper system, at the k-th stage of this procedure, we have reached a system $\underline{\underline{A}}^{(k)}\underline{x} = b^{(k)}$ where

$$
\underline{\underline{A}}^{(k)} = \begin{pmatrix} \underline{\underline{A}}_{11}^{(k)} & \underline{\underline{A}}_{12}^{(k)} \\ 0 & \underline{\underline{A}}_{22}^{(k)} \end{pmatrix}
\tag{50}
$$

with $\underline{\underline{A}}_{11}^{(k)} \in \mathcal{R}^{(k-1)\times(k-1)}$ upper triangular. We then eliminate all elements in the k-th column below the diagonal by subtracting scaled versions on the k-th row to all rows i with $i > k$:

$$
\begin{aligned}
a_{ij}^{(k+1)} &= a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)}, \quad i = k+1, ..., n, j = k+1, ..., n, \\
b_i^{(k+1)} &= b_i^{(k)} - m_{ik} b_k^{(k)}, \quad i = k+1, ..., n,
\end{aligned}
\tag{51}
$$

with $m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}$. Each stage requires $\mathcal{O}(n^2)$ operations, the whole procedure $\mathcal{O}(n^3)$. j starts at $k+1$, because we already know the result for $j = k$ (by design $a_{ik}^{(k+1)} = 0, i > k$).

2. the triangular system can then be solved bottom-up

3.3.3 The need for pivoting

Problem: The procedure above breaks down for $a_{kk}^{(k)} = 0$. For large $m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}$, low-order digits of $a_{ij}^{(k)}$ and $b_i^{(k)}$ will be lost.

Consider the simple example system

$$\left(\begin{array}{cc|c} \epsilon & 1 & 1 \\ 1 & 1 & 2 \end{array}\right) \tag{52}$$

with Gaussian elimination we yield

$$\left(\begin{array}{cc|c} \epsilon & 1 & 1 \\ 0 & 1 - \frac{1}{\epsilon} & 2 - \frac{1}{\epsilon} \end{array}\right) \tag{53}$$

breaking down for $\epsilon = 0$ and leading to cancellation for small ϵ.

Idea: We exchange rows (and columns) to minimize m_{ik} at the k-th stage of Gaussian elimination. For instance

$$\left(\begin{array}{cc|c} \epsilon & 1 & 1 \\ 1 & 1 & 2 \end{array}\right) \rightarrow \left(\begin{array}{cc|c} 1 & 1 & 2 \\ \epsilon & 1 & 1 \end{array}\right) \rightarrow \left(\begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 1-\epsilon & 1-\epsilon \end{array}\right) \tag{54}$$

is unproblematic.

Partial pivoting: At the start of the k-th stage of Gaussian elimination exchange the k-th and r-th row, where

$$r = \underset{k \leq i \leq n}{\operatorname{argmax}} |a_{ik}^{(k)}| \tag{55}$$

such that $m_{ik} < 1, i = k+1, ..., n$.

One can also combine row and column swaps to minimize m_{ik} further (*complete pivoting*) but this requires $\mathcal{O}(n^3)$ comparison operations in total compared to the $\mathcal{O}(n^2)$ comparison operations of partial pivoting.

The stability of Gaussian elimination with a given pivoting strategy is characterized by the growth factor

$$\rho_n(\underline{\underline{A}}) = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|} \tag{56}$$

3.3.4 Wilkinson's backward error stability result for Gaussian elimination with partial pivoting

Consider $\underline{\hat{x}}$ is the numerical solution to $\underline{\underline{A}}\underline{x} = \underline{b}$, $\underline{\underline{A}} \in \mathbb{R}^{n\times n}$ obtained using Gaussian elimination with partial pivoting. Then $\underline{\hat{x}}$ is the exact solution to the perturbed problem

$$(\underline{\underline{A}} + \underline{\underline{\Delta A}})\underline{\hat{x}} = \underline{b}, \quad ||\underline{\underline{\Delta A}}||_\infty \leq n^2\gamma_{3n}g_W(n)||\underline{\underline{A}}||_\infty \tag{57}$$

where

$$||\underline{\underline{A}}||_\infty = \max_{1\leq i\leq n} \sum_{j=1}^{n} |a_{ij}| \quad \text{max row sum} \tag{58}$$

and γ_{3n} is defined in Eq. 22. For partial pivoting, $g_W(n) \leq 2^{n-1}$ (an exponential bound on the matrix size), in practice g_W is of the order $n^{2/3}$ to $n^{1/2}$. For the details and proof see Higham, 2002, Theorem 9.5.

3.3.5 LU factorization

Let us ignore pivoting for now. We will

1. rewrite Gauss elimination in matrix form
2. see that Gauss elimination implicitly performs a matrix factorization $\underline{\underline{A}} = \underline{\underline{L}}\underline{\underline{U}}$ with L lower-diagonal and U upper diagonal

3.3.5.1 Why are we interested in an LU factorization?

Once we have a factorization of the form $\underline{\underline{A}} = \underline{\underline{L}}\underline{\underline{U}}$, we can solve for any right-hand side $\underline{b}$ in $\underline{\underline{A}}\underline{x} = \underline{b}$ based on

$$\underline{\underline{A}}\underline{x} = \underline{\underline{L}}\underline{\underline{R}}\underline{x} \quad \leftrightarrow \quad \underline{\underline{L}}\underline{y} = \underline{b}, \quad \underline{\underline{R}}\underline{x} = \underline{y} \tag{59}$$

so by solving two triangular systems.

1. (Forward substitution) Solve $\underline{\underline{L}}\underline{y} = \underline{b}$ for $\underline{y}$. We can write the system as

$$\begin{pmatrix} l_{11} & \\ \underline{L}_{*1} & \underline{\underline{L}}_{**} \end{pmatrix} \begin{pmatrix} y_1 \\ \underline{y}_* \end{pmatrix} = \begin{pmatrix} b_1 \\ \underline{b}_* \end{pmatrix} \Rightarrow l_{11} y_1 = b_1, \quad \underline{L}_{*1} y_1 + \underline{\underline{L}}_{**} \underline{y}_* = b_* \tag{60}$$

from which we can solve for $y_1 = \frac{b_1}{l_{11}}$. We can then construct a new triangular system for $\underline{y}_*$

$$\underline{\underline{L}}_{**} \underline{y}_* = \underline{b}_* - \underline{L}_{*1} y_1 \tag{61}$$

so $\underline{y}$ can be solved recursively, yielding the explicit formula

$$y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{k=1}^{i-1} l_{ik} y_k \right) \tag{62}$$

taking $\mathcal{O}(N^2)$ operations.

2. (Backward substitution) Then $\underline{\underline{R}}\underline{x} = \underline{y}$ for $\underline{x}$. The logic is the same as for forward substitution, but bottom-up. We can write the system as

$$\begin{pmatrix} \underline{\underline{R}}_{**} & \underline{R}_{*n} \\ & r_{nn} \end{pmatrix} \begin{pmatrix} \underline{x}_* \\ x_1 \end{pmatrix} = \begin{pmatrix} \underline{y}_* \\ y_n \end{pmatrix} \tag{63}$$

yielding the explicit formula

$$a_i = \frac{1}{r_{ii}} \left(y_i - \sum_{k=i+1}^{N} r_{ik} a_k \right) \tag{64}$$

also taking $\mathcal{O}(N^2)$ operations.

3.3.5.2 Gauss elimination in matrix form I: Example

We want to write our Gaussian elimination update in matrix form.

Let us consider a specific example for Gaussian elimination

$$\underline{\underline{A}}^{(2)} = \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & a_{14}^{(2)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & a_{34}^{(2)} \\ 0 & a_{42}^{(2)} & a_{43}^{(2)} & a_{44}^{(2)} \end{pmatrix} \tag{65}$$

so

$$\underline{\underline{A}}^{(3)} = \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & a_{14}^{(2)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} \\ 0 & a_{32}^{(2)} - a_{22}^{(2)} \frac{a_{32}^{(2)}}{a_{22}^{(2)}} & a_{33}^{(2)} - a_{23}^{(2)} \frac{a_{32}^{(2)}}{a_{22}^{(2)}} & a_{34}^{(2)} - a_{24}^{(2)} \frac{a_{32}^{(2)}}{a_{22}^{(2)}} \\ 0 & a_{42}^{(2)} - a_{22}^{(2)} \frac{a_{42}^{(2)}}{a_{22}^{(2)}} & a_{43}^{(2)} - a_{23}^{(2)} \frac{a_{42}^{(2)}}{a_{22}^{(2)}} & a_{44}^{(2)} - a_{24}^{(2)} \frac{a_{42}^{(2)}}{a_{22}^{(2)}} \end{pmatrix} \tag{66}$$

We plug in $m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}$

$$\begin{aligned} \underline{\underline{A}}^{(3)} &= \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & a_{14}^{(2)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} \\ 0 & a_{32}^{(2)} - a_{22}^{(2)} m_{32} & a_{33}^{(2)} - a_{23}^{(2)} m_{32} & a_{34}^{(2)} - a_{24}^{(2)} m_{32} \\ 0 & a_{42}^{(2)} - a_{22}^{(2)} m_{42} & a_{43}^{(2)} - a_{23}^{(2)} m_{42} & a_{44}^{(2)} - a_{24}^{(2)} m_{42} \end{pmatrix} \\ &= \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & a_{14}^{(2)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & a_{34}^{(2)} \\ 0 & a_{42}^{(2)} & a_{43}^{(2)} & a_{44}^{(2)} \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & a_{22}^{(2)} m_{32} & a_{23}^{(2)} m_{32} & a_{24}^{(2)} m_{32} \\ 0 & a_{22}^{(2)} m_{42} & a_{23}^{(2)} m_{42} & a_{24}^{(2)} m_{42} \end{pmatrix} \\ &= \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & a_{14}^{(2)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & a_{34}^{(2)} \\ 0 & a_{42}^{(2)} & a_{43}^{(2)} & a_{44}^{(2)} \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & m_{32} & 0 & 0 \\ 0 & m_{42} & 0 & 0 \end{pmatrix} \begin{pmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & a_{14}^{(2)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & a_{34}^{(2)} \\ 0 & a_{42}^{(2)} & a_{43}^{(2)} & a_{44}^{(2)} \end{pmatrix} \\ &= \left(\underline{\underline{1}} - \begin{pmatrix} 0 \\ 0 \\ m_{32} \\ m_{42} \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 & 0 \end{pmatrix} \right) \underline{\underline{A}}^{(2)} \\ &= \left(\underline{\underline{1}} - \underline{m}_2 \hat{\underline{e}}_2^T \right) \underline{\underline{A}}^{(2)} \end{aligned} \tag{67}$$

This generalizes to

$$\underline{\underline{A}}^{(k+1)} = \underline{\underline{M}}_k \underline{\underline{A}}^{(k)}, \quad \underline{\underline{M}}_k = (\underline{\underline{1}} - \underline{m}_k \underline{e}_k^T), \quad (\underline{m}_k)_i = \begin{cases} 0, & i \leq k \\ \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, & i > k \end{cases} \tag{68}$$

The general relation is derived in the next section.

3.3.5.3 Gauss elimination in matrix form II: Index notation

We start by removing the bounds on i and j in our Gaussian elimination update

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)}, \quad i = k+1, ..., n, \quad j = k+1, ..., n \tag{69}$$

as follows:

- we start j from k, making $a_{i>k,k}^{(k+1)} = 0$ explicit
- as for $j < k : a_{kj}^{(k)} = 0$, we can also start j at 1
- finally, by redefining

$$m_{ik} = \begin{cases} 0, & i \leq k \\ \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, & i > k \end{cases} \tag{70}$$

  we can entirely remove the bounds on the indices.

For $i, j = 1, ..., n$

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)} = a_{ij}^{(k)} - \underbrace{m_{ik}\delta_{kl}}_{=(\underline{m}_k \underline{\hat{e}}_k^T)_{il}} a_{lj}^{(k)} \tag{71}$$

where $\underline{m}_k = (m_{1k}, ..., m_{nk})^T$ and $(\underline{\hat{e}}_k)_l = \delta_{kl}$ where δ_{kl} is the Kronecker delta.

> A matrix product $\underline{\underline{C}} = \underline{\underline{AB}}$ corresponds to $c_{ij} = \sum_{l=1}^{n} a_{il} b_{lj} = a_{il} b_{lj}$ in **index notation**, where we implicitly sum over repeated indices.

The corresponding matrix notation is

$$\underline{\underline{A}}^{(k+1)} = \underline{\underline{M}}_k \underline{\underline{A}}^{(k)}, \quad \underline{\underline{M}}_k = \underline{\underline{1}} - \underline{m}_k \underline{e}_k^T \tag{72}$$

3.3.5.4 From Gauss elimination in matrix form to LU factorization

Gauss elimination results in an upper diagonal matrix

$$\underline{\underline{U}} := \underline{\underline{A}}^{(n)} = \underline{\underline{M}}_{n-1} \cdots \underline{\underline{M}}_2 \underline{\underline{M}}_1 \underline{\underline{A}} \tag{73}$$

Note that

$$\underline{\underline{M}}_k^{-1} = \underline{\underline{1}} + \underline{m}_k \underline{e}_k^T \tag{74}$$

You can prove this by showing $\underline{\underline{M}}_k^{-1} \underline{\underline{M}}_k = \underline{\underline{1}}$. You will need to use $(\underline{m}_k \underline{e}_k^T)(\underline{m}_k \underline{e}_k^T) = \underline{\underline{0}}$ as

$(\underline{m}_k)_{i\leq k} = 0$.

This naturally leads to the matrix factorization

$$\begin{aligned}
\underline{\underline{A}} &= \underline{\underline{M}}_1^{-1}\underline{\underline{M}}_2^{-1}\dots\underline{\underline{M}}_{n-1}^{-1}\underline{\underline{U}} \\
&= \left(1 + \sum_{i=1}^{n-1} \underline{m}_i \hat{\underline{e}}_i^T\right)\underline{\underline{U}} \\
&= \begin{pmatrix} 1 & & & & \\ m_{21} & 1 & & & \\ \vdots & m_{32} & \ddots & & \\ \vdots & \vdots & & \ddots & \\ m_{n1} & m_{n2} & \dots & m_{n,n-1} & 1 \end{pmatrix}\underline{\underline{U}} \\
&=: \underline{\underline{L}}\,\underline{\underline{U}}
\end{aligned} \tag{75}$$

where L is lower diagonal.

Gauss elimination implicitly performs a factorization $\underline{\underline{A}} = \underline{\underline{L}}\,\underline{\underline{U}}$ with $\underline{\underline{L}}$ lower diagonal and $\underline{\underline{U}}$ upper diagonal.

3.3.5.5 Calculating the LU-decomposition in $\mathcal{O}(N^3)$

From

$$\begin{pmatrix} a_{11} & \underline{A}_{1*}^T \\ \underline{A}_{*1} & \underline{\underline{A}}_{**} \end{pmatrix} = \begin{pmatrix} l_{11} & \\ \underline{L}_{*1} & \underline{\underline{L}}_{**} \end{pmatrix}\begin{pmatrix} r_{11} & \underline{R}_{1*}^T \\ & \underline{\underline{R}}_{**} \end{pmatrix}, \quad \text{set diagonal of } \underline{\underline{L}} \text{ to } 1 \rightarrow \text{ unique decomp} \tag{76}$$

we can devise an $\mathcal{O}(N^3)$ algorithm to calculate $\underline{\underline{L}}$ and $\underline{\underline{R}}$, see code 2.

Note: Blocked versions can be parallelized, QR-decomposition retains the condition number of $\underline{\underline{A}}$.

Problem: LU-decomposition is $\mathcal{O}(N^3)$.

```julia
# in-place decomposition of A into L and R, the upper-right part of
↪ A is overwritten by R,
# the lower-left part of A is overwritten by L without diagonal (set
↪ to 1)
function lu_decomposition!(A::Matrix{T}) where T <: Number
    N = size(A, 1)
    for i in 1:N
        for j in i+1:N
            A[j, i] /= A[i, i]
            for k in i+1:N
                A[j, k] -= A[j, i] * A[i, k]
            end
        end
    end
end
# one can then extract L = tril(A, -1) + I
# and U = triu(A) via the LinearAlgebra package
```

Code-Snippet 2: LU-decomposition of a matrix $\underline{\underline{A}} \in \mathbb{R}^{N\times N}$ in $\mathcal{O}(N^3)$.

3.3.6 When does the LU factorization exist? *

A unique LU factorization exists of $\underline{\underline{A}} \in \mathbb{R}^{n\times n}$ if and only if $\underline{\underline{A}}_k := \underline{\underline{A}}(1:k, 1:k)$ is nonsingular for $k = 1, ..., n-1$. Otherwise an LU factorization may still exist but if so not unique. See Higham, 2002, Theorem 9.2 for a proof.

3.4 Iterative methods

3.4.1 Jacobi iteration | a splitting method

Aim: We want quicker than $\mathcal{O}(N^3)$, approximate solutions to linear systems.

Consider the linear system

$$\underline{\underline{A}}\underline{x} = \underline{b}, \quad \underline{\underline{A}} \in \mathbb{R}^{N\times N}, \underline{x}, \underline{b} \in \mathbb{R}^N \tag{77}$$

For Jacobi iteration, we start with the trivial decomposition

$$\begin{gathered}\underline{\underline{A}} = \underline{\underline{D}} - (\underline{\underline{L}} + \underline{\underline{U}}), \quad \text{diagonal part } \underline{\underline{D}}, \quad \text{negative left-below-diagonal part} \underline{\underline{L}} \\ \text{negative right-above-diagonal part} \underline{\underline{U}}\end{gathered} \tag{78}$$

so

$$\underline{\underline{A}}\underline{x} = \left[\underline{\underline{D}} - (\underline{\underline{L}} + \underline{\underline{U}})\right]\underline{x} = \underline{b} \quad \rightarrow \quad \underline{x} = \underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}})\underline{x} \tag{79}$$

so $\underline{x}$ is a fixed point of the RHS, leading to fixed-point, here Jacobi iteration

$$\boxed{\underline{x}^{(n+1)} = \underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}})\underline{x}^{(n)}, \quad (\underline{\underline{D}}^{-1})_{ii} = \frac{1}{A_{ii}}} \tag{80}$$

3.4.1.1 When does the Jacobi iteration converge?

The Jacobi iteration converges if and only if all eigenvalues λ_i of the convergence matrix

$$\underline{\underline{M}} := \underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}}) \tag{81}$$

are smaller than one.

$$\text{spectral radius } \rho_s(\underline{\underline{M}}) \equiv \max_i |\lambda_i| < 1 \tag{82}$$

The smaller the spectral radius, the faster the convergence.

Derivation of the convergence criterion Consider the error at step $n+1$

$$\begin{aligned} \underline{e}^{(n+1)} &= \underline{x}_{\text{exact}} - \underline{x}^{(n+1)} \underbrace{=}_{\text{Jacobi-Iteration}} \underline{x}_{\text{exact}} - \left(\underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{M}}\underline{x}^{(n)}\right) \\ &\underbrace{=}_{\underline{x}_{\text{exact}} = \underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{M}}\underline{x}_{\text{exact}}} \underline{\underline{M}}(\underline{x}_{\text{exact}} - \underline{x}^{(n)}) = \underline{\underline{M}}\underline{e}^{(n)} \end{aligned} \tag{83}$$

so

$$\underline{e}^{(n)} = \underline{\underline{M}}^n \underline{e}^{(0)} \tag{84}$$

which scales with $\rho_s^n(\underline{\underline{M}})$ for n sufficiently large (follows from decomposing $\underline{e}^{(0)}$ into eigenvectors of $\underline{\underline{M}}$) so the convergence criterion follows.

Problem: The convergence is usually slow, better use Gauss-Seidel iteration.

3.4.1.2 Example Jacobi step

Consider

$$\underline{\underline{A}}\underline{x} = \begin{pmatrix} 10 & -5 & 0 \\ -5 & 10 & -5 \\ 0 & -5 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \underline{b} = \begin{pmatrix} 0 \\ 0 \\ 15 \end{pmatrix} \tag{85}$$

then

$$\underline{\underline{D}} = \begin{pmatrix} 10 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 5 \end{pmatrix}, \quad \underline{\underline{L}} = \begin{pmatrix} 0 & 0 & 0 \\ 5 & 0 & 0 \\ 0 & 5 & 0 \end{pmatrix}, \quad \underline{\underline{R}} = \begin{pmatrix} 0 & 5 & 0 \\ 0 & 0 & 5 \\ 0 & 0 & 0 \end{pmatrix} \tag{86}$$

so

$$\underline{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \rightarrow \quad \underline{x}^{(1)} = \begin{pmatrix} \frac{1}{10}b_0 + \frac{1}{10} \cdot 5x_2^{(0)} \\ \frac{1}{10}b_1 + \frac{1}{10} \cdot \left(5x_1^{(0)} + 5x_3^{(0)}\right) \\ \frac{1}{5}b_1 + \frac{1}{5} \cdot 5x_2^0 \end{pmatrix} \tag{87}$$

For Gauss-Seidel iteration, refer to Section A.3. For Krylov subspace methods, refer to Section A.4.

4 Numerical Derivatives and Interpolation

The derivative of a function $f : \mathbb{R} \to \mathbb{R}$ at $x \in \mathbb{R}$ is defined as

$$f'(x) := \lim_{\delta \to 0} \frac{f(x+\delta) - f(x)}{\delta} \tag{88}$$

and I assume you to be familiar with various *symbolic* differentiation rules, e.g. $\frac{d \sin}{dx}(x) = \cos(x)$.

Consider the function f is the output of a (possibly complex) computer program - how would you calculate the derivative? There are two general options:

- **finite differencing**: one approximates the derivative at a given point x based on evaluations of f
- **automatic differentiation**: the primitive building blocks of the program come with pre-implemented derivatives, the chain rule is applied to differentiate f

Details on automatic differentiation - the backbone of modern machine learning - can be found in Sec. A.6 in the appendix. Here we focus on **finite differencing**.

Aim: Approximate $f'(x)$ based on evaluations of f.

Such approximations can be derived based on Taylor expansion.

4.1 Taylor expansion

In the vicinity of a point, Taylor expansion approximates a function with a polynomial.

Let $I \subset \mathbb{R}$ and x_0 an inner point of I. Consider a function $f : \mathbb{R} \to \mathbb{R}$ which is n times continuously differentiable on I and $n+1$ times differentiable inside I.

For all $x \in I$ we can Taylor expand

$$f(x) = T_n(x) + R_n(x) \tag{89}$$

where T_n is the degree-n polynomial

$$T_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k \tag{90}$$

and $R_n(x)$ is a remainder, in *Lagrange form* given by

$$R_n(x) = \frac{f^{(n+1)}(\eta)}{(n+1)!}(x-x_0)^{n+1} \in \mathcal{O}(|x-x_0|^{n+1}), \quad \eta \in [x_0, x] \tag{91}$$

where we used Landaus big-$\mathcal{O}$ notation.

Landau notation:

- little o: $f \in o(g) \iff \lim_{x\to a} \left|\frac{f(x)}{g(x)}\right| = 0$ (f grows slower than g)
- big $\mathcal{O}$: $f \in \mathcal{O}(g) \iff \limsup_{x\to a} \left|\frac{f(x)}{g(x)}\right| < \infty$ (f grows at most as fast as g)

Consider $x = x_0 + \Delta x$, then we compactly write

$$\boxed{f(x_0+\Delta x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}\Delta x^k + \mathcal{O}(\Delta x^{n+1})} \tag{92}$$

The interpolation error is of order $n+1$.

4.2 Finite differencing

Let us discretize a function $f : \mathbb{R} \to \mathbb{R}$ to a grid of evaluations $f_i = f(x_i)$ at points x_i with $x_{i+1} = x_i + \Delta x$. We will denote $f_i' \equiv \frac{df}{dx}(x_i), f_i'' \equiv \frac{d^2 f}{dx^2}(x_i), ...$

4.2.1 Derivation of finite differencing formulas from Taylor expansion

By Taylor expansion, we can derive finite difference formulas for the derivatives.

- By a forward Taylor expansion up to first order

$$f_{i+1} = f_i + \Delta x\, f_i' + \mathcal{O}(\Delta x^2) \quad \to \quad f_i' = \frac{f_{i+1} - f_i}{\Delta x} + \mathcal{O}(\Delta x) \tag{93}$$

  we yield the **first order forward difference** formula.

- Likewise,

$$f_{i-1} = f_i - \Delta x\, f_i' + \mathcal{O}(\Delta x^2) \quad \to \quad f_i' = \frac{f_i - f_{i-1}}{\Delta x} + \mathcal{O}(\Delta x) \tag{94}$$

  yields the **first order backward difference** formula.

By considering more evaluation points, we can reach higher orders in the *truncation error*:

$$\left.\begin{aligned} f_{i+1} &= f_i + \Delta x\, f_i' + \tfrac{1}{2} f_i'' \Delta x^2 + \mathcal{O}(\Delta x^3) \\ f_{i-1} &= f_i - \Delta x\, f_i' + \tfrac{1}{2} f_i'' \Delta x^2 + \mathcal{O}(\Delta x^3) \end{aligned}\right\} \rightarrow f_i' = \frac{f_{i+1} - f_{i-1}}{2\Delta x} + \mathcal{O}(\Delta x^2) \tag{95}$$

here the **second order central difference** formula.

We can also derive approximations for **higher-order derivatives**

$$\left.\begin{aligned} f_{i+1} &= f_i + \Delta x\, f_i' + \tfrac{1}{2} f_i'' \Delta x^2 + \mathcal{O}(\Delta x^3) \\ f_{i-1} &= f_i - \Delta x\, f_i' + \tfrac{1}{2} f_i'' \Delta x^2 + \mathcal{O}(\Delta x^3) \end{aligned}\right\} \rightarrow f_i'' = \frac{f_{i+1} - 2f_i + f_{i-1}}{\Delta x^2} + \mathcal{O}(\Delta x) \tag{96}$$

4.2.2 Truncation vs. floating point errors

We would like to choose a small Δx to reduce the truncation error. However, for very small Δx cancellation occurs (as we subtract very similar numbers, e.g. f_{i+1} and f_i, magnified by dividing by a small number) and the error in our approximation is dominated by the floating point error (see e.g. Fig. 3 in Baydin et al., 2018 and Sauer, 2025).

Let us consider the **central difference formula** and assume floating point representations

$$\hat{f}_i = f_i + \epsilon_i, \quad \text{assume } |\epsilon_i| \leq \epsilon_{\text{mach}} \tag{97}$$

such that

$$f_i' = \frac{\hat{f}_{i+1} - \epsilon_{i+1} - \hat{f}_{i-1} + \epsilon_{i-1}}{2\Delta x} + \mathcal{O}(\Delta x^2) = \frac{\hat{f}_{i+1} - \hat{f}_{i-1}}{2\Delta x} + \mathcal{O}(\Delta x^2) + \frac{\epsilon_{i-1} - \epsilon_{i+1}}{2\Delta x} \tag{98}$$

where by our assumption $\epsilon_{i-1} - \epsilon_{i+1} \leq 2\epsilon_{\text{mach}}$

$$\underbrace{f_i'}_{\text{exact derivative}} = \underbrace{\frac{\hat{f}_{i+1} - \hat{f}_{i-1}}{2\Delta x}}_{\text{finite difference}} + \underbrace{\mathcal{O}(\Delta x^2)}_{\text{truncation error}} + \underbrace{\mathcal{O}(\frac{\epsilon_{\text{mach}}}{\Delta x})}_{\text{floating point error}} \tag{99}$$

Task to the reader: Show that the error term is minimal for $\Delta x \sim \epsilon_{\text{mach}}^{1/3}$.

4.3 Discretize and solve - the power of finite differencing

Scientific principles are typically captured in the form of differential equations, for instance the diffusion equation in two dimensions

$$\partial_t u = K\,(\partial_x^2 + \partial_y^2)u \tag{100}$$

Finite differencing is the simplest option for solving such equations. The two principal discretization options across the independent variables are

- discretize onto a grid
- discretize into an iteration

Spatial variables are typically discretized onto a grid and time is discretized into an iteration. For the 2D diffusion equation above, we discretize space into a regular $2D$ grid $u_{ij} = u(x_i, y_j)$

$$K\,((\partial_x^2 + \partial_y^2)u)_{ij} \approx K \left(\frac{u_{i+1,j} - 2u_{ij} + u_{i-1,j}}{\Delta x^2} + \frac{u_{i,j+1} - 2u_{ij} + u_{i,j-1}}{\Delta y^2} \right) \tag{101}$$

and time into an explicit iteration

$$\partial_t u_{ij}^{(n)} \approx \frac{u_{ij}^{(n+1)} - u_{ij}^{(n)}}{\Delta t} \approx K\,((\partial_x^2+\partial_y^2)u)_{ij}^{(n)} \quad \rightarrow \quad u_{ij}^{(n+1)} \approx u_{ij}^{(n)} + \Delta t\, K\,((\partial_x^2+\partial_y^2)u)_{ij}^{(n)} \tag{102}$$

so together with our spatial finite difference approximation, we obtain the explicit finite difference scheme

$$u_{ij}^{(n+1)} \approx u_{ij}^{(n)} + \Delta t\, K \left(\frac{u_{i+1,j}^{(n)} - 2u_{ij}^{(n)} + u_{i-1,j}^{(n)}}{\Delta x^2} + \frac{u_{i,j+1}^{(n)} - 2u_{ij}^{(n)} + u_{i,j-1}^{(n)}}{\Delta y^2} \right) \tag{103}$$

which is started from some initial conditions $u_{ij}^{(0)} = u_{ij}(t = t_0)$.

4.4 Interpolation

Consider we have data points $f_i = f(x_i), i = 0, .., m$ and would like to find a polynomial $p \in \Pi_m$ passing through those points $p(x_i) = f(x_i)$.

Theorem of Polynomial Interpolation: Let $(x_0, f_0), ..., (x_m, f_m)$ be $m + 1$ points in the plane with distinct x_i. Then there exists one and only one polynomial p of degree m or less that satisfies $p(x_i) = f_i$ for $i = 0, ..., m$. See Sauer, 2025, Theorem 2.3.

In fluid simulations, for instance, we might deal with cell-centered fluid quantities and would like to interpolate to the cell boundaries to calculate intercell fluxes.

4.4.1 Evaluating polynomials efficiently - Horner scheme

Let us start with the efficient evaluation of polynomials.

Consider $p \in \Pi_m$, so

$$p(x) = b_0 + b_1 x + b_2 x^2 + ... + b_m x^m \tag{104}$$

We could obtain all the powers $(x^2, ..., x^m)$ by $m-1$ multiplications and could then evaluate $p(x)$ with another m multiplications of the powers with the coefficients, in total $2m-1$ multiplications (and m computationally cheaper additions).

We can rewrite polynomials to reduce the number of multiplications

- $p(x) = b_0 + b_1 x + b_2 x^2 = b_0 + (b_1 + b_2 x)x$ ($m = 2$, 2 multiplications, 2 additions)
- $p(x) = b_0 + b_1 x + b_2 x^2 + b_3 x^3 = b_0 + (b_1 + (b_2 + b_3 x)x)x$ ($m = 3$, 3 multiplications, 3 additions)
- $\vdots$
- $p(x) = (\dots(b_m x + b_{m-1})x + \dots)$ (m multiplications, m additions)

- only m multiplication are necessary in the **Horner scheme**.

The Horner scheme can be implemented in the recurrent form

$$z_j = b_j + x z_{j+1}, z_m = b_m \quad \rightarrow \quad z_0 = p(x) \tag{105}$$

4.4.2 Lagrange interpolation

The simplest interpolation scheme is obtained based on Lagrange polynomials

$$l_j(x) = \prod_{k=0,k\neq j}^{m} \frac{x - x_k}{x_j - x_k} \quad \rightarrow \quad l_j(x_i) = \delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j \end{cases} \tag{106}$$

designed such that l_j is 1 on x_j but zero on all other supporting points.

The interpolation becomes trivial

$$p(x) = f(x_0)l_0(x) + f(x_1)l_1(x) + \dots + f(x_m)l_m(x) \tag{107}$$

Problems:

- The computational cost of evaluating the Lagrange polynomial is $\mathcal{O}(m^2)$.
- Cancellation problems in the evaluation of l_j.
- One cannot simply add new points x_i without recomputing all base functions.

4.4.3 Newton interpolation

Consider the Newton base

$$n_0 := 1, \quad n_j := (x - x_0) \cdot \dots \cdot (x - x_{j-1}), j = 1, ..., m, \quad \text{short:}\, n_j(x) = \prod_{i=0}^{j-1} (x - x_i) \tag{108}$$

and construct the polynomial as

$$p(x) = \sum_{j=0}^{m} d_j n_j(x) \tag{109}$$

This construction has two advantages:

- p can be evaluated in m multiplications with a modified Horner scheme

$$z_m = d_m, z_j = d_j + (x - x_j) z_{j+1} \quad \rightarrow \quad z_0 = p(x) \tag{110}$$

- the base functions n_j evaluate to zero for all evaluation points x_i with $i = 0, ..., j-1$ (a product is zero when one of the factor is zero)

This second advantage is crucial. It means that our interpolation goal

$$p(x_i) = f(x_i) = f_i, \quad i = 0, ..., m, \quad p \in \Pi_m \tag{111}$$

leads to a lower triangular system

$$\begin{pmatrix} n_0(x_0) & & \\ \vdots & \ddots & \\ n_0(x_m) & \dots & n_m(x_m) \end{pmatrix} \begin{pmatrix} d_0 \\ \vdots \\ d_m \end{pmatrix} = \begin{pmatrix} f(x_0) \\ \vdots \\ f(x_m) \end{pmatrix} \tag{112}$$

which could be solved bottom down in $\mathcal{O}(m^2)$.

4.4.3.1 Newton divided differences

The coefficients can be determined more efficiently using divided differences

$$d_j = [x_0, ..., x_j]f \tag{113}$$

where for $j < i$ we recursively define

$$\begin{aligned}[x_j]\, f &= f_j, \\ [x_j, \dots, x_i]\, f &= \frac{[x_{j+1}, \dots, x_i]\, f - [x_j, \dots, x_{i-1}]\, f}{x_i - x_j}\end{aligned} \tag{114}$$

4.4.4 Interpolation error

Consider we interpolate an $m+1$ times continuously differentiable function f on a closed interval $[a, b]$ ($f \in C^{m+1}[a, b]$) with an m-th order polynomial $p_m \in \Pi_m$ on $m+1$ distinct points $x_0, ..., x_m$ (such that $p(x_i) = f(x_i)$).

By Newton interpolation, p_m is given by ($x \in I$)

$$p_m(x) = \sum_{j=0}^{m} \left([x_0, ..., x_j]f \prod_{i=0}^{j-1} (x - x_i) \right) \tag{115}$$

Consider a polynomial p_{m+1} which also interpolates f at $x_{m+1} := x$ ($p_{m+1}(x) = f(x)$). By Newton interpolation, p_{m+1} is given by

$$\begin{aligned} p_{m+1}(\zeta) &= \sum_{j=0}^{m+1} \left([x_0, ..., x_j]f \prod_{i=0}^{j-1} (\zeta - x_i) \right) \\ &= \underbrace{\sum_{j=0}^{m} \left([x_0, ..., x_j]f \prod_{i=0}^{j-1} (\zeta - x_i) \right)}_{= p_m(\zeta)} + \underbrace{\left([x_0, ..., x_m, x]f \prod_{i=0}^{m} (\zeta - x_i) \right)}_{=: R_m(\zeta)} \\ &= p_m(\zeta) + R_m(\zeta) \end{aligned} \tag{116}$$

By our interpolation objective

$$f(x) = p_{m+1}(x) = p_m(x) + R_n(x) \quad \rightarrow \quad f(x) - p_m(x) = R_m(x) = [x_0, ..., x_m, x]f \prod_{i=0}^{m}(x - x_i) \tag{117}$$

such that R_n **is our interpolation error**.

We can apply the mean value theorem to obtain a Lagrange form of the error term R_n. For each $x \in [a, b]$ exists a $\xi(x) \in (a, b)$ such that

$$\boxed{f(x) - p_m(x) = R_n(x) = \frac{f^{m+1}(\xi)}{(m+1)!} \prod_{i=0}^{m}(x - x_i)} \tag{118}$$

Note: While we cannot typically control $f^{m+1}(\xi)$, we might be able to optimize the $\prod_{i=0}^{m}(x - x_i)$ factor (leading to *Chebyshev interpolation*).

4.4.4.1 Proof of the Lagrange form of the interpolation error

Let us repeat two essential theorems from calculus.

- **Rolle's theorem:** Consider $f \in C^1[a, b]$ and $f(a) = f(b) = 0$. Then there exists at least one $\xi \in (a, b)$ such that $f'(\epsilon) = 0$.
- **Classical mean value theorem**: Consider $f \in C^1[a, b]$ then there exists at least on $\xi \in (a, b)$ such that
$$f'(\xi) = \frac{f(b) - f(a)}{b - a} \tag{119}$$

We can now formulate a **generalized mean value theorem**: For any pairwise distinct points $x_0, ..., x_m \in [a, b]$ there exists $\xi \in (a, b)$ such that

$$[x_0, ..., x_m]f = \frac{f^{(m)}(\xi)}{m!} \tag{120}$$

Proof: We equivalently proof

$$[x_0, ..., x_m, x]f = \frac{f^{m+1}(\xi)}{(m+1)!} \tag{121}$$

to retain the context from the Lagrange error term. Our starting point is

$$f(x) - p_m(x) = R_m(x) = [x_0, ..., x_m, x]f\, n_{m+1}(x), \quad n_{m+1}(x) = \prod_{i=0}^{m}(x - x_i) \tag{122}$$

We define the auxiliary function (for an arbitrary $x \in [a,b]$, $x \neq x_i$)

$$\phi(t) = R_m(t)n_{m+1}(x) - R_m(x)n_{m+1}(t) \tag{123}$$

The proof is based on the following observations:

- $n_{m+1}(t) = \prod_{i=0}^{m}(t - x_i) = t^{m+1} + \dots$ (the highest order term is t^{m+1}) such that $n_{m+1}^{(m+1)} = (m+1)!$ (you can show this by induction)
- $R_m^{(m+1)}(t) = f^{(m+1)}(t) - \underbrace{p_m^{(m+1)}(t)}_{=0} = f^{(m+1)}(t)$
- for some function $g \in C^1[a,b]$ with n roots in (a,b), g' has at least $n-1$ roots (in-between the roots of g) in (a,b), following from Rolle's theorem
- $\phi(t)$ has $m+2$ roots: $x_0, ..., x_m$ and x

Therefore, since $\phi(t)$ has $m+2$ roots, $\phi^{(m+1)}(t)$ has at least one root $\xi \in (a,b)$:

$$\begin{aligned} 0 &= \phi^{(m+1)}(\xi) \\ &= R_m^{(m+1)}(\xi)n_{m+1}(x) - R_m(x)n_{m+1}^{(m+1)}(\xi) \\ &= f_m^{(m+1)}(\xi)n_{m+1}(x) - [x_0, ..., x_m, x]f\, n_{m+1}(x)(m+1)! \end{aligned} \tag{124}$$

such that

$$[x_0, ..., x_m, x]f = \frac{f_m^{(m+1)}(\xi)}{(m+1)!} \tag{125}$$

4.4.5 Final remarks

Let us make two final remarks.

- To interpolate between many data points one typically does not use one very high-order polynomial but one rather pieces together lower-order polynomials (typically such that the overall interpolation is twice continuously differentiable) (this avoids *Runge's phenomenon*)
- Often data points contain noise and an exact interpolation is not of interest, but we want to find the underlying functional relation which generated the data. In such cases we resort to regression, e.g. polynomial regression or neural network regression.

5 Root finding and optimization

We have already talked about linear problems in Sec. 3. However, many problems we face in practice are non-linear. For instance, how do we even calculate an arbitrary square-root $\sqrt{a}, a > 0$? Or how do we fit a non-linear model, e.g. $f_{\underline{\theta}}(t) = \theta_0 + \theta_1 \exp(-\theta_2 t), \underline{\theta} = (\theta_0, \theta_1, \theta_2)^T$, to data?

Such non-linear problems broadly fall into two categories:

- root finding: for a function $\underline{g} : \mathbb{R}^n \to \mathbb{R}^n$ find $\underline{x}^\star \in \mathbb{R}^n$ with $\underline{g}(\underline{x}^\star) = \underline{0}$. A function might generally have no or multiple roots. Calculating a square root $\sqrt{a}, a > 0$, for instance, is equivalent to finding the positive root of $g(x) = x^2 - a$.
- optimization: for a function $L : \mathbb{R}^p \to \mathbb{R}$ we want to find $\underline{\theta}^* = \operatorname{argmin}_{\underline{\theta}} L(\underline{\theta})$. A typical example is a non-linear fit. Given the data $(t_i, y_i), i = 1, ..., N$ we want to find parameters $\underline{\theta}$ which minimize the residual sum

$$\underline{\theta}^* = \underset{\underline{\theta}}{\operatorname{argmin}}\, L(\underline{\theta}), \quad L(\underline{\theta}) = \sum_{i=1}^{N} |y_i - f_{\underline{\theta}}(t_i)|^2 \tag{126}$$

  We want to find a global minimum based on local knowledge of L.

This section requires knowledge in multivariable calculus, for instance the notion of gradients, Jacobians and Hessians. An introduction to gradients is given in Sec. A.2.

5.1 Root finding

5.1.1 Bisection method

Let us start with the univariate case, $g : \mathbb{R} \to \mathbb{R}$.

Aim: Find $x^\star$ with $g(x^\star) = 0$.

Insight: Consider we know some a, b ($a < b$) where g has opposite sign, so $g(a)g(b) < 0$. Assuming g is continuous, there must exist a root $x^\star \in (a, b)$.

Start with some $a_0, b_0 \in \mathbb{R}$ with $g(a_0)g(b_0) < 0$.

1. Divide the interval $[a_i, b_i]$ by half into $[a_i, c_i]$ and $[c_i, b_i]$ where $c_i = \frac{a_i+b_i}{2}$.
2. If $g(c_i) = 0$ we have found a root and terminate. Otherwise,
   - if $g(a_i)g(c_i) < 0$ then $x^\star \in (a_i, c_i)$, set $a_{i+1} = a_i, b_{i+1} = c_i$

- if $g(c_i)g(b_i) < 0$ then $x^\star \in (c_i, b_i)$, set $a_{i+1} = c_i, b_{i+1} = b_i$

then $i \to i+1$.

We will typically want to terminate before we find the exact root. There are multiple options for termination. We know for instance that $|x^\star - c_i| \leq |b - c_i|$ (there is no point in $[a, b]$ further from c than b (or a)) and can terminate when $|b - c| \leq \epsilon_{\text{tol}}$ with a given absolute tolerance ϵ_{tol}.

Task to the reader: Find an upper bound for $|x^\star - c_n|$ after n iteration steps.

Problem: The bisection method does not generalize the higher-dimensional case $g : \mathbb{R}^n \to \mathbb{R}^n, n > 1$.

5.1.2 Newton's iterative method for root finding

Let us think about a different approach, again starting with the univariate case $g : \mathbb{R} \to \mathbb{R}$. Our aim is still to approximate a root $x^\star$ ($g(x^\star) = 0$).

We want to find a sequence x_k which quickly and reliably converges to the root $x^\star$.

To construct this sequence, we linearly approximate g around x_k

$$g(x_{k+1}) \approx g(x_k) + (x_{k+1} - x_k)\, g'(x_k) = 0 \tag{127}$$

and solve for the root

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} \tag{128}$$

This is illustrated in Fig. 7.

If g is continuously differentiable in a neighborhood around a root $x^\star$ with nonzero derivative at $x^\star$, then there exists a neighborhood around $x^\star$ where Newton's iteration converges to $x^\star$ for all starting values.

5.1.2.1 Quadratic convergence of Newton's method

Consider x_k is an approximation to $x^\star$ with error

$$\epsilon_k = x^\star - x_k \tag{129}$$

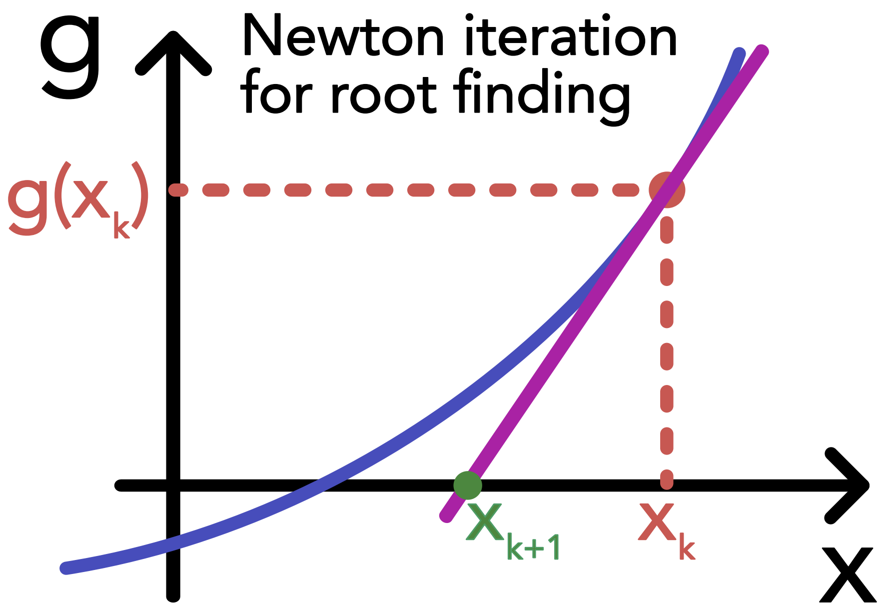


Figure 7: Illustration of the Newton-Raphson method.

We would like to relate ϵ_{k+1} to ϵ_k to characterize the error convergence.

$$\epsilon_{k+1} = x^\star - x_{k+1} = x^\star - x_k + \frac{g\,(x_k)}{g'(x_k)} = \epsilon_k + \frac{g\,(x_k)}{g'(x_k)} \tag{130}$$

Now we use the Taylor expansion of g (assume g has a continuous second derivative) from x_k to $x^\star$

$$0 = g(x^\star) = g(x_k + \epsilon_k) = g(x_k) + \epsilon_k g'(x_k) + \frac{1}{2}\epsilon_k^2 g''(\xi_k), \quad \xi_k \in (x_k, x^\star) \tag{131}$$

so

$$g(x_k) = -\epsilon_k g'(x_k) - \frac{1}{2}\epsilon_k^2 g''(\xi_k) \tag{132}$$

to obtain

$$\begin{aligned} \epsilon_{k+1} &= \epsilon_k + \frac{g\,(x_k)}{g'(x_k)} \\ &= \epsilon_k + \frac{-\epsilon_k g'(x_k) - \frac{1}{2}\epsilon_k^2 g''(\xi_k)}{g'(x_k)} \\ &= -\frac{g''(\xi_k)}{2g'(x_k)}\epsilon_k^2 \end{aligned} \tag{133}$$

so for the absolute error

$$\boxed{|\epsilon_{k+1}| = \frac{|g''(\xi_k)|}{2|g'(x_k)|}\epsilon_k^2, \quad \xi_k \in (x_k, x^\star)} \tag{134}$$

Given that for all points x closer to $x^\star$ than our original guess x_0 ($x \in I = [x_0 - |\epsilon_0|, x_0 + |\epsilon_0|]$), $\frac{|g''(\xi_k)|}{2|g'(x_k)|}$ is bound ($x \in I : f'(x) \neq 0$), we define

$$M := \frac{1}{2}\left(\sup_{x\in I} |g''(x)|\right)\left(\sup_{x\in I} \frac{1}{|g'(x)|}\right), \quad \rho := M|\epsilon_0| \tag{135}$$

to obtain an upper bound in

$$|\epsilon_{k+1}| \leq M\epsilon_k^2 \quad \rightarrow \quad M|\epsilon_1| \leq M^2\epsilon_0^2 = \rho^2, M|\epsilon_2| \leq M^2\epsilon_1^2 \leq \rho^4, ..., M|\epsilon_k| \leq \rho^{2^k} \tag{136}$$

which quadratically converges to zero given that

$$\rho = M|\epsilon_0| < 1 \tag{137}$$

5.1.2.2 Multivariate Newton iteration

Different from the bisection method, Newton's iterative method is straightforward to generalize to the multivariate case.

In the multivariate case, $g : \mathbb{R}^n \rightarrow \mathbb{R}^n$, we have

$$\underline{x}_{k+1} = \underline{x}_k - \underline{\underline{J}}_g^{-1}\left(\underline{x}_k\right)\underline{g}\left(\underline{x}_k\right), \quad \underline{\underline{J}}_g = \begin{pmatrix} - & \underline{\nabla}_{\underline{x}} g_1 & - \\ \vdots & \vdots & \vdots \\ - & \underline{\nabla}_{\underline{x}} g_n & - \end{pmatrix} \tag{138}$$

where $\underline{\underline{J}}_g \in \mathbb{R}^{n\times n}$ is the Jacobian of $\underline{g}$.

5.2 Optimization

In optimization, for a function $L : \mathbb{R}^p \rightarrow \mathbb{R}$ we want to find $\underline{\theta}^* = \operatorname{argmin}_{\underline{\theta}} L(\underline{\theta})$. Often L is an error term, called *loss* and $\underline{\theta}$ are model parameters which we tweak to minimize the loss. In constrained optimization there are further constraints on $\underline{\theta}$.

The fundamental problem in optimization is that our knowledge of $L : \mathbb{R}^p \rightarrow \mathbb{R}$ is only local. We can evaluate L and possibly its derivatives, but this comes at a cost. We would like to find a minimum of L using as little evaluations as possible.

A naive strategy for finding the minimizer in a given region would be grid-search: Consider $p = 1$ and assume the minimum to be between θ_A and θ_B. We choose N evaluation points $\theta_i = \theta_A + \frac{i}{N}(\theta_B - \theta_A)$ and estimate $\underline{\theta}^*$ with

$$\hat{\theta}^*_{\text{grid-search}} = \underset{\theta_i, i=1,\ldots,N}{\operatorname{argmin}} L(\theta_i) \tag{139}$$

Now consider $p = 2$. To maintain the same resolution per dimension as in the one dimensional case, N^2 grid points are necessary. This is illustrated in Fig. 8.

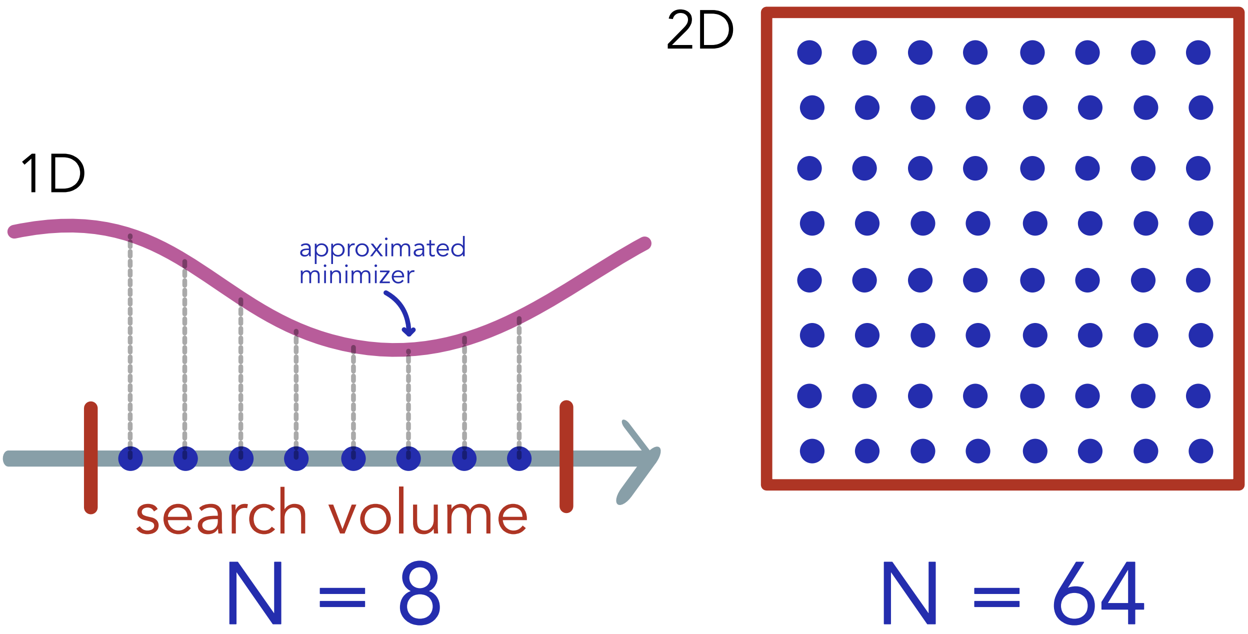


Figure 8: Curse of dimensionality in grid-search.

Curse of dimensionality in grid-search: For a given resolution per dimension, the number of evaluations necessary to approximate the minimum of $L : \mathbb{R}^p \to \mathbb{R}$ scales exponentially with N^p. This quickly becomes infeasible. Also, the larger the search volume (the more global the search), the higher the cost (for a fixed resolution).

The algorithms we use are typically designed to find *local minimizers*.

Local minimizer: A point $\underline{\theta}^*$ is a strict local minimizer of $L : \mathbb{R}^p \to \mathbb{R}$ if there is a neighborhood of $\underline{\theta}^*$ such that $L(\underline{\theta}^*) < L(\underline{\theta})$ for all $\underline{\theta}$ in the neighborhood.

Let us repeat two important results from analysis (Wright, Nocedal, et al., 1999, Theorem 2.4 and 2.5).

Sufficient conditions for local minimizers: Suppose $\underline{\theta}^*$ is a stationary point of $L : \mathbb{R}^p \to \mathbb{R}$, i.e. $\underline{\nabla} L(\underline{\theta}^*) = \underline{0}$, and the Hessian

$$\underline{\underline{H}}_L(\underline{\theta}^*) = \left[\frac{\partial^2 L}{\partial \theta_i \partial \theta_j} \right]_{i,j=1}^{p} \in \mathbb{R}^{p \times p} \tag{140}$$

is positive definite, i.e.

$$\forall \underline{v} \in \mathbb{R}^p \setminus \{0\} : \quad \underline{v}^{\mathsf{T}} \underline{\underline{H}}_L(\underline{\theta}^*) \underline{v} > 0, \tag{141}$$

and $\underline{\underline{H}}_L$ is continuous in an open neighborhood of $\underline{\theta}^*$, then $\underline{\theta}^*$ is a strict local minimizer of L.

Stationary points of convex functions: If $L : \mathbb{R}^p \to \mathbb{R}$ is differentiable and convex, i.e.

$$\forall \underline{x}, \underline{y} \in \mathbb{R}^p, \forall \alpha \in [0,1] : L(\alpha \underline{x} + (1-\alpha)\underline{y}) \leq \alpha L(\underline{x}) + (1-\alpha) L(\underline{y}) \tag{142}$$

(for a twice differentiable function its convex if and only if its Hessian is positive semidefinite) then any stationary point of L, i.e. $\underline{\nabla} L(\underline{\theta}^*) = \underline{0}$, is a **global minimizer** of L.

This is the mathematical foundation of optimization schemes searching for stationary points.

5.2.1 Iterative algorithms for optimization

We consider iterative schemes designed to find local minima.

An iteration

$$\underline{\theta}_{k+1} = \underline{\theta}_k + \alpha_k \underline{p}_k \tag{143}$$

is given by a direction $\underline{p}_k$ ($||\underline{p}_k|| = 1$) and a step length α_k.

Aim: We want to find a direction $\underline{p}_k$ and step size α_k to minimize $L(\underline{\theta}_{k+1})$.

Consider we Taylor expand L to first order

$$L(\underline{\theta}_k + \alpha_k \underline{p}_k) = L(\underline{\theta}_k) + \alpha_k \underline{p}_k^T \underline{\nabla} L(\underline{\theta}_k) + \mathcal{O}(\alpha^2) \tag{144}$$

Any direction with

$$\underline{p}_k^T \underline{\nabla} L(\underline{\theta}_k) = ||\underline{p}_k||\, ||\underline{\nabla} L(\underline{\theta}_k)|| \cos \measuredangle \left(\underline{p}_k, \underline{\nabla} L(\underline{\theta}_k) \right) < 0 \tag{145}$$

will reduce L to first order!

5.2.2 Gradient Descent

The gradient $\underline{\nabla} L$ points in the direction of maximum ascent. Therefore the most natural choice for the search direction $\underline{p}_k$ is the negative gradient, $\underline{p}_k = -\underline{\nabla} L(\underline{\theta}_k)$.

In gradient descent we walk down the gradient of L, starting at some initial guess $\underline{\theta}^{(0)}$, so

$$\begin{aligned} \underline{\theta}^{(n+1)} &= \underline{\theta}^{(n)} - \gamma \underline{\nabla} L\big|_{\underline{\theta}=\underline{\theta}^{(n)}}, \quad \text{learning rate } \gamma \\ \text{until } &\left| L\left(\underline{\theta}^{(n+1)}\right) - L\left(\underline{\theta}^{(n)}\right)\right| < \epsilon \text{ or } \# \text{ steps } > N_{\max} \end{aligned} \tag{146}$$

where ϵ is a small number and $N_{\max}$ is the maximum number of steps, set by the user. Alternatively, one can stop when $||\underline{\theta}^{(n+1)} - \underline{\theta}^{(n)}|| < \epsilon$.

Gradient descent is one of the most widely used and generally applicable optimization method. Virtually all modern machine learning models are trained based on variants of gradient descent.

Problems of standard gradient descent:

- finds local minima
- how to choose the learning rate
- gets stuck on plateaus

5.2.2.1 Improvements to standard gradient descent

- to combat local minima, we can start from multiple initial conditions and take the best final result
- we prefer an approximate global optimum over an exact local one $\rightarrow$ use stochasticity, (e.g. injected noise $\rightarrow$ jump out of local but not global minima); simulated annealing (probabilistically move to some state or stay in the current one, even locally suboptimal steps are allowed)
- adaptive learning rate γ (reduce for steep gradients) (e.g. AdaGrad, RMSProp, Adam)

5.2.3 Newton's method in optimization

We will motivate Newton's method in optimization in two ways.

We have mentioned the connection between stationary points and local minima. Stationary points where $\underline{\nabla} L(\underline{\theta}^*) = \underline{0}$ are necessary conditions for local minima (but not sufficient, we might also be dealing with a local maximum or saddle point) and for differentiable convex functions even sufficient conditions for global minima.

This motivates applying Newton's root finding method on $\underline{\nabla} L(\underline{\theta}^*)$ for finding stationary points.

$$\underline{\theta}_{k+1} = \underline{\theta}_k - \underline{\underline{H}}_L^{-1}(\underline{\theta}_k)\underline{\nabla} L(\underline{\theta}_k) \tag{147}$$

Our second viewing angle comes from the line-search principle. Let us Taylor expand L to second order

$$L(\underline{\theta}_k + \underline{p}) = \underbrace{L(\underline{\theta}_k) + \underline{p}^T \underline{\nabla} L(\underline{\theta}_k) + \frac{1}{2}\underline{p}^T \underline{\underline{H}}_L(\underline{\theta}_k)\underline{p}}_{=:m_k(\underline{p})} + \mathcal{O}(||\underline{p}_k||^3) \tag{148}$$

Assuming $\underline{\underline{H}}_L(\underline{\theta}_k)$ is positive definite, then $m_k(\underline{p})$ is convex and the stationary point $\underline{\nabla} m_k(\underline{p}_k) = \underline{0}$ is the unique minimizer of m_k, so the optimal step direction under second order Taylor expansion. Put intuitively we fit a parabola to L and move to its minimum. We can determine $\underline{p}_k$ to

$$\underline{\nabla} m_k(\underline{p}_k) = \underline{\nabla} L(\underline{\theta}_k) + \underline{\underline{H}}_L(\underline{\theta}_k)\underline{p} = 0 \quad \rightarrow \quad \underline{p}_k = -\underline{\underline{H}}_L^{-1}(\underline{\theta}_k)\underline{\nabla} L(\underline{\theta}_k) \tag{149}$$

which brings us back to Newton's method as derived above

$$\underline{\theta}_{k+1} = \underline{\theta}_k - \underline{\underline{H}}_L^{-1}(\underline{\theta}_k)\underline{\nabla} L(\underline{\theta}_k) \tag{150}$$

Note: When $\underline{\underline{H}}_L(\underline{\theta}_k)$ is not positive definite, the Hessian might not be invertible, and even if the Hessian is invertible, the search direction might not satisfy the descent condition, $\underline{p}_k^T \underline{\nabla} L(\underline{\theta}_k) < 0$.

Generally, any search direction

$$\underline{p}_k = -\underline{\underline{B}}_k^{-1} \underline{\nabla} L(\underline{\theta}_k) \tag{151}$$

where $\underline{\underline{B}}_k \in \mathbb{R}^{p \times p}$ is a symmetric positive definite matrix, **is a descent direction**, as

$$\underline{p}_k^T \underline{\nabla} L(\underline{\theta}_k) = -\underline{\nabla} L(\underline{\theta}_k)^T \underline{\underline{B}}_k^{-1} \underline{\nabla} L(\underline{\theta}_k) < 0 \tag{152}$$

as the inverse of a positive definite matrix is also positive definite.

5.2.3.1 Advantages of Newton optimization

- In the convex / concave setting the usage of curvature information by Newton iteration leads to faster convergence than gradient descent, as illustrated in Fig. 9.
- step-size is automatically adaptive
  - *small Hessian* → automatically larger steps
  - *large Hessian* → automatically smaller steps

5.2.3.2 Disadvantages of Newton optimization

- strictly only applies to convex / concave functions
- it searches critical points, not minima
- the inversion and calculation of the Hessian can be too costly → consider quasi-Newton methods like the Broyden–Fletcher–Goldfarb–Shanno algorithm

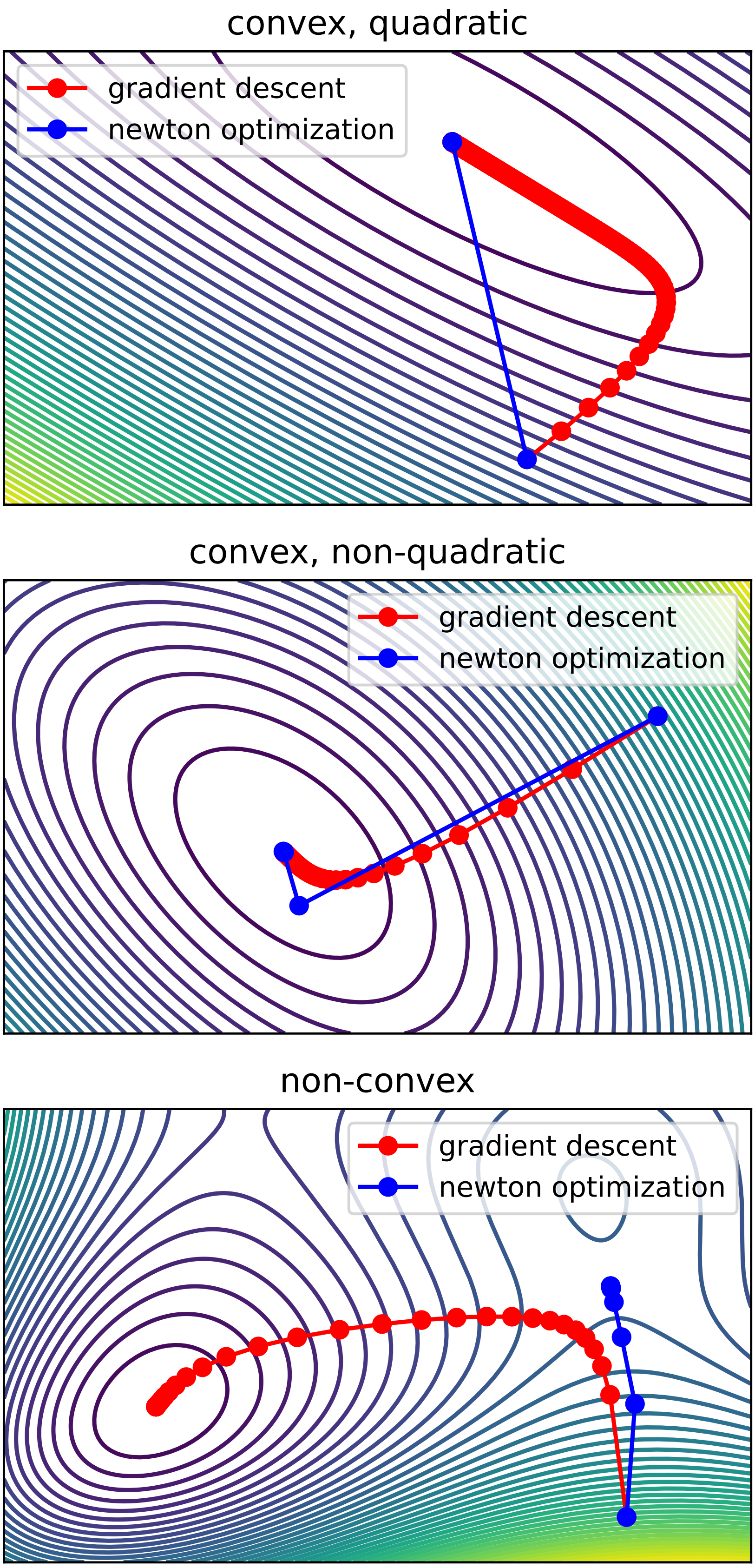


Figure 9: Comparison of Newton optimization and gradient descent. For the example of the non-convex function, Newton optimization finds a sattle point (zero gradient, indefinite Hessian).

5.2.4 Nonlinear least-squares optimization: Gauss-Newton

5.2.4.1 Problem setting of least-squares optimization

Consider $L : \mathbb{R}^p \to \mathbb{R}$ has the special form

$$L(\underline{\theta}) = \frac{1}{2}\sum_{i=1}^{N} r_i^2(\underline{\theta}) = \frac{1}{2}||\underline{r}(\underline{\theta})||_2^2, \quad \underline{r}(\underline{\theta}) = (r_1(\underline{\theta}), ..., r_N(\underline{\theta}))^T \tag{153}$$

This is typically the case when we have data $\{t_i, y_i\}_{i=1}^N$ and a model $f_{\underline{\theta}}(t)$ and we want to find parameters

$$\underline{\theta}^* = \underset{\underline{\theta}}{\operatorname{argmin}} \frac{1}{2}\sum_{i=1}^{N} (\underbrace{f_{\underline{\theta}}(t_i) - y_i}_{r_i})^2 \tag{154}$$

For a model $f_{\underline{\theta}}$ linear in $\underline{\theta}$, see Sec. 3.2. But what is $f_{\underline{\theta}}$ is nonlinear in $\underline{\theta}$, for instance

$$f_{\underline{\theta}}(t) = \theta_0 + \theta_1 \exp(-\theta_2 t) \tag{155}$$

5.2.4.2 Special structure of the gradient and Hessian in least-squares optimization

Given the Jacobian

$$\underline{\underline{J}}_{\underline{r}}(\underline{\theta}) = \left[\frac{\partial r_j}{\partial \theta_i}\right]_{\substack{j=1,\ldots,N \\ i=1,\ldots,p}} \in \mathbb{R}^{N \times p} \tag{156}$$

of

$$\underline{r} : \mathbb{R}^p \to \mathbb{R}^N, \quad \underline{r}(\underline{\theta}) = (r_1(\underline{\theta}), ..., r_N(\underline{\theta}))^T \tag{157}$$

$(N > p)$, the gradient of

$$L(\underline{\theta}) = \frac{1}{2}\sum_{i=1}^{N} r_i^2(\underline{\theta}) = \frac{1}{2}||\underline{r}(\underline{\theta})||_2^2 \tag{158}$$

is given by (chain rule)

$$\underline{\nabla} L(\underline{\theta}) = \underline{\underline{J}}_{\underline{r}}(\underline{\theta})^T \underline{r}(\underline{\theta}) \in \mathbb{R}^p \tag{159}$$

and the Hessian by

$$\begin{aligned}
\left[\underline{\underline{H}}_L(\underline{\theta})\right]_{k,l} &= \frac{1}{2}\sum_{i=1}^{N} \partial_{\theta_k}\partial_{\theta_l} r_i^2(\underline{\theta}) \\
&= \sum_{i=1}^{N} \partial_{\theta_k}\left(r_i(\underline{\theta})\partial_{\theta_l} r_i(\underline{\theta})\right) \\
&= \sum_{i=1}^{N} \left(\partial_{\theta_k} r_i(\underline{\theta})\right)\left(\partial_{\theta_l} r_i(\underline{\theta})\right) + \sum_{i=1}^{N} r_i(\underline{\theta})\partial_{\theta_k}\partial_{\theta_l} r_i(\underline{\theta})
\end{aligned} \tag{160}$$

which in matrix notation is

$$\underline{\underline{H}}_L(\underline{\theta}) = \underline{\underline{J}}_{\underline{r}}(\underline{\theta})^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta}) + \sum_{i=1}^{N} r_i(\underline{\theta})\underline{\underline{H}}_{r_i}(\underline{\theta}) \in \mathbb{R}^{p\times p} \tag{161}$$

Note that the first term in the Hessian, $\underline{\underline{J}}_{\underline{r}}(\underline{\theta})^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta})$ only requires first derivatives of the residuals, which are much less expensive than the second derivatives appearing in the second term $\sum_{i=1}^{N} r_i(\underline{\theta})\underline{\underline{H}}_{r_i}(\underline{\theta})$. **Luckily, the second term can often be neglected compared to the first term**. This can be done when

- the residuals r_i are small
- or the residuals are close to affine in $\underline{\theta}$, such that the second derivatives $\partial_{\theta_k}\partial_{\theta_l} r_i(\underline{\theta})$ are small

5.2.4.3 Gauss-Newton optimization

Plugging in our result for the gradient

$$\underline{\nabla} L(\underline{\theta}) = \underline{\underline{J}}_{\underline{r}}(\underline{\theta})^T \underline{r}(\underline{\theta}) \in \mathbb{R}^p \tag{162}$$

and the first term approximation to the Hessian

$$\underline{\underline{H}}_L(\underline{\theta}) \approx \underline{\underline{J}}_{\underline{r}}(\underline{\theta})^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta}) \tag{163}$$

into Newton optimization

$$\underline{\theta}_{k+1} = \underline{\theta}_k - \underline{\underline{H}}_L^{-1}(\underline{\theta}_k)\underline{\nabla} L(\underline{\theta}_k) \tag{164}$$

yields the Gauss-Newton method

$$\underline{\theta}_{k+1} = \underline{\theta}_k - \left(\underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)\right)^{-1} \underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)^T \underline{r}(\underline{\theta}_k) \tag{165}$$

Gauss-Newton always goes into descent directions as $\underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)$ is positive definite.

Of course $\underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)$ must be non-singular for Gauss-Newton.

5.2.4.4 Levenberg-Marquardt optimization

In the Levenberg-Marquadt method

$$\underline{\theta}_{k+1} = \underline{\theta}_k - \left(\underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k) + \alpha(\underline{\theta}_k)\underline{\underline{1}}\right)^{-1} \underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)^T \underline{r}(\underline{\theta}_k) \tag{166}$$

the $\alpha(\underline{\theta}_k)\underline{\underline{1}}$ term ensures that $\underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k)^T \underline{\underline{J}}_{\underline{r}}(\underline{\theta}_k) + \alpha(\underline{\theta}_k)\underline{\underline{1}}$ is positive definite (thus invertible) and the iterative updates are well-defined.

5.2.4.5 Trust-region view

Given our objective

$$\underline{\theta}^* = \underset{\underline{\theta}}{\operatorname{argmin}} \frac{1}{2}||\underline{r}(\underline{\theta})||_2^2 \tag{167}$$

a different way to derive Gauss-Newton is based on the local approximation

$$m_k^{GN}(\underline{p}) = \frac{1}{2}||\underline{r}(\underline{\theta}) + \underline{\underline{J}}_{\underline{r}}(\underline{\theta})\underline{p}||^2, \quad \underline{p}_k = \underset{\underline{p}}{\operatorname{argmin}}\, m_k^{GN}(\underline{p}), \quad \underline{\theta}_{k+1} = \underline{\theta}_k + \underline{p}_k \tag{168}$$

In this view Levenberg-Marquardt corresponds to the regularization (compare Lasso regression)

$$m_k^{LM}(\underline{p}) = \frac{1}{2}||\underline{r}(\underline{\theta}) + \underline{\underline{J}}_{\underline{r}}(\underline{\theta})\underline{p}||^2 + \frac{\alpha(\underline{\theta}_k)}{2}||\underline{p}||^2 \tag{169}$$

where for a given $\alpha(\underline{\theta}_k)$ in

$$\underline{p}_k = \underset{\underline{p}}{\operatorname{argmin}} \frac{1}{2}||\underline{r}(\underline{\theta}) + \underline{\underline{J}}_{\underline{r}}(\underline{\theta})\underline{p}||^2 + \frac{\alpha(\underline{\theta}_k)}{2}||\underline{p}||^2 \tag{170}$$

there exists a Δ_k such that the optimization is equivalent to

$$\underline{p}_k = \underset{\underline{p}}{\operatorname{argmin}}\, m_k^{GN}(\underline{p}), \quad \text{subject to} ||\underline{p}|| < \Delta_k \tag{171}$$

so a search in a *trust-region* with radius Δ_k.

6 Integration

Consider a function $f : [a, b] \to \mathbb{R}$. We are interested in the integral

$$\int_a^b f(x)\, dx \tag{172}$$

In general, there might not be an analytical solution to the integral which can be written in elementary form, as is the case for $f(x) = \exp(-x^2)$. So how can we calculate integrals numerically?

Monte Carlo integration, advantageous for high-dimensional integration, is covered in Sec. A.10. Additionally, notes on Gauss-Legendre quadrature are provided in Sec. A.9.

6.1 Classical one-dimensional quadrature

Idea: On small subsection, we can faithfully approximate $f(x)$ by simpler functions with analytical integrals.

We start by dividing the domain $[a, b]$ into sub-intervals $[x_i, x_{i+1}], x_0 = a, x_N = b$, such that

$$\int_a^b f(x)\, dx = \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} f(x)\, dx \tag{173}$$

Based on the approximation of $f(x)$ on these sub-intervals, different numerical integration schemes come up:

- **Midpoint rule**: For the central-constant approximation

  $$f(x) \approx f\left(\frac{x_i + x_{i+1}}{2}\right) \quad \text{for} \quad x \in [x_i, x_{i+1}] \tag{174}$$

  we obtain the midpoint rule

  $$\int_{x_i}^{x_{i+1}} f(x)\, dx = \underbrace{(x_{i+1} - x_i)\, f\left(\frac{x_i + x_{i+1}}{2}\right)}_{=:M_i} + \mathcal{O}((x_{i+1} - x_i)^3) \tag{175}$$

- **Trapezoidal rule**: For the linear approximation

  $$f(x) \approx f(x_i) + \frac{x - x_i}{x_{i+1} - x_i}\, (f(x_{i+1}) - f(x_i)) \quad \text{for} \quad x \in [x_i, x_{i+1}] \tag{176}$$

we obtain the trapezoidal rule

$$\int_{x_i}^{x_{i+1}} f(x)\,dx = \underbrace{(x_{i+1}-x_i)\left(\frac{f(x_i)+f(x_{i+1})}{2}\right)}_{=:T_i} + \mathcal{O}((x_{i+1}-x_i)^3) \tag{177}$$

- **Simpson's rule**: We interpolate $f(x)$ through

$$(x_i, f(x_i))\,, \left(\frac{x_i+x_{i+1}}{2}, f\left(\frac{x_i+x_{i+1}}{2}\right)\right), (x_{i+1}, f(x_{i+1})) \tag{178}$$

with a polynomial

$$f(x) \approx a(x-x_i)^2 + b(x-x_i) + c \quad \text{for} \quad x \in [x_i, x_{i+1}] \tag{179}$$

yielding (we leave the derivation as a task to the reader)

$$\int_{x_i}^{x_{i+1}} f(x)\,dx = \underbrace{\frac{x_{i+1}-x_i}{6}\left(f(x_i) + 4f\left(\frac{x_i+x_{i+1}}{2}\right) + f(x_{i+1})\right)}_{=:S_i} + \mathcal{O}((x_{i+1}-x_i)^5) \tag{180}$$

These quadrature rules are compared in Fig. 10.

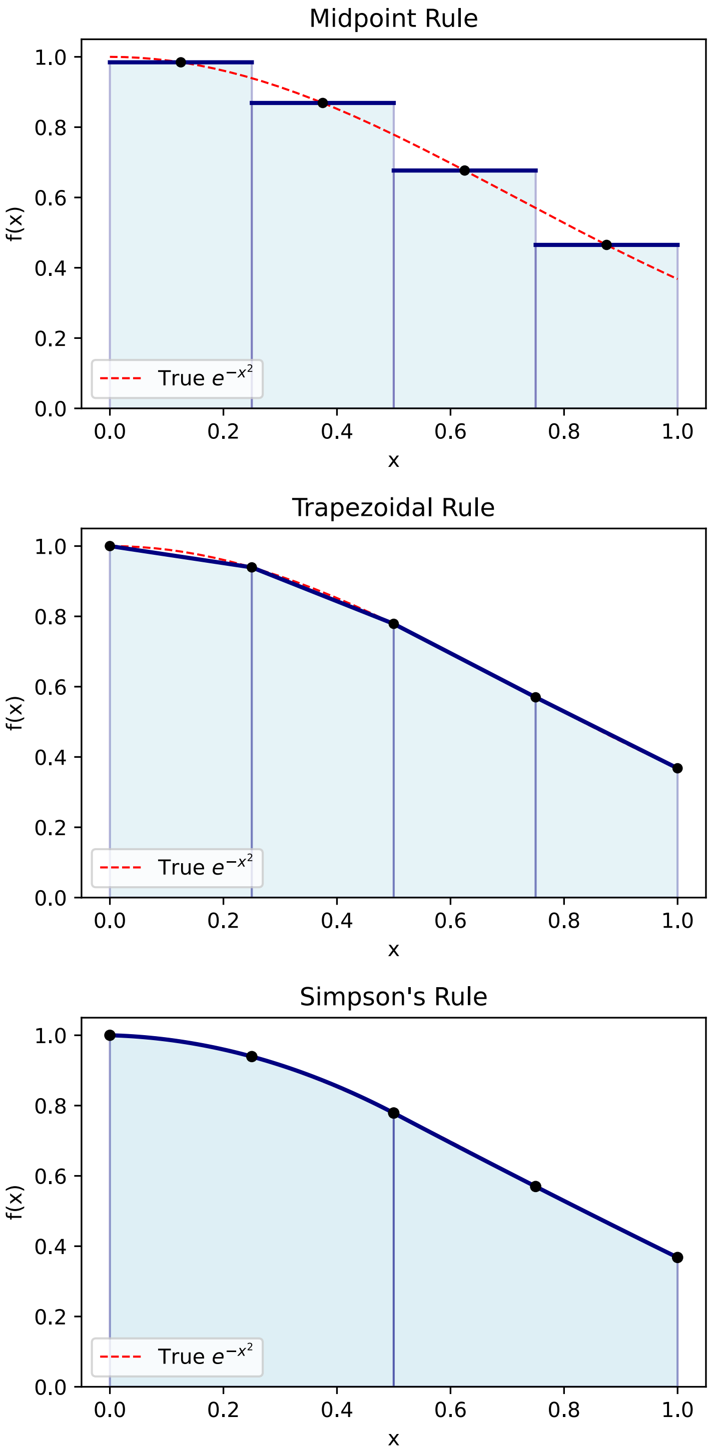


Figure 10: Comparison of the midpoint rule, trapezoidal rule and Simpson's rule for the same number of evaluation points of f.

6.1.1 Local error derivations

We can use Taylor expansion to derive error terms for quadrature rules.

Let us define

$$c_i := \frac{x_i + x_{i+1}}{2}, \quad \Delta x := x_{i+1} - x_i \tag{181}$$

and consider the **midpoint approximation**

$$f(x) = f(c_i) + (x - c_i) f'(c_i) + \frac{1}{2}(x - c_i)^2 f''(c_i) + \mathcal{O}((x - c_i)^3) \quad \text{for} \quad x \in [x_i, x_{i+1}] \tag{182}$$

yielding

$$\begin{aligned}
\int_{x_i}^{x_{i+1}} f(x)\, dx &= \Delta x f(c_i) + \underbrace{\int_{x_i}^{x_{i+1}} (x - c_i) f'(c_i)\, dx}_{=0} + \int_{x_i}^{x_{i+1}} \frac{1}{2}(x - c_i)^2 f''(c_i)\, dx + \mathcal{O}(\Delta x^4) \\
&= \Delta x f(c_i) + 2 f''(c_i) \int_{c_i}^{x_{i+1}} \frac{1}{2}(x - c_i)^2\, dx + \mathcal{O}(\Delta x^4) \\
&= \Delta x f(c_i) + \frac{1}{24} f''(c_i) \Delta x^3 + \mathcal{O}(\Delta x^4)
\end{aligned} \tag{183}$$

such that

$$M_i = M_{[x_i, x_{i+1}]}(f) := \Delta x f(c_i) = \int_{x_i}^{x_{i+1}} f(x)\, dx - \frac{1}{24} f''(x_i) \Delta x^3 + \mathcal{O}(\Delta x^4) \tag{184}$$

where we have used $f''(c_i) = f''(x_i) + \mathcal{O}(\Delta x)$.

Likewise, for the trapezoidal rule, we can find

$$T_i = T_{[x_i, x_{i+1}]}(f) := \Delta x \frac{f(x_i) + f(x_{i+1})}{2} = \int_{x_i}^{x_{i+1}} f(x)\, dx + \frac{1}{12} f''(x_i) \Delta x^3 + \mathcal{O}(\Delta x^4) \tag{185}$$

Idea: We can combine M_i and T_i to cancel the third order error terms.

$$\begin{aligned} S_i &= S_{[x_i, x_{i+1}]}(f) \\ &:= \frac{2M+T}{3} \\ &= \frac{x_{i+1}-x_i}{6}\left(f(x_i) + 4f\left(\frac{x_i+x_{i+1}}{2}\right) + f(x_{i+1})\right) \\ &= \int_{x_i}^{x_{i+1}} f(x)\,dx + \mathcal{O}(\Delta x^4) \end{aligned} \tag{186}$$

We have just re-derived Simpson's rule! But we previously stated the error term $\mathcal{O}(\Delta x^5)$. We will see that the even error terms vanish by symmetry.

Let us split f into an even and odd part around c_i:

$$f(x) = f_e(x) + f_o(x), \quad f_e(c_i - x) = f_e(c_i + x), \quad f_o(c_i - x) = -f_o(c_i + x) \tag{187}$$

By symmetry

$$S_i(f_o) = \int_{x_i}^{x_{i+1}} f_o(x)\,dx = 0, \quad c_i = \frac{x_i + x_{i+1}}{2} \tag{188}$$

i.e. the uneven part is integrated exactly.

The Taylor expansion of f_e has only even terms, such that an approximation to $\int_{x_i}^{x_{i+1}} f_e(x)\,dx$ based on truncated Taylor expansion can only have uneven error terms. Therefore

$$S_i = \frac{x_{i+1}-x_i}{6}\left(f(x_i) + 4f\left(\frac{x_i+x_{i+1}}{2}\right) + f(x_{i+1})\right) = \int_{x_i}^{x_{i+1}} f(x)\,dx + \mathcal{O}(\Delta x^5) \tag{189}$$

For a proof on the specific error term

$$E_{\text{local}} = \int_{x_i}^{x_{i+1}} (f(x) - q_i(x))\,dx = -\frac{f^{(4)}(\xi)}{32 \cdot 90}(\underbrace{x_{i+1} - x_i}_{=:\Delta x})^5 \sim \mathcal{O}(\Delta x^5) \tag{190}$$

(with q_i the polynomial that agrees with f at $x_i, \frac{x_i + x_{i+1}}{2}, x_{i+1}$) see Fejzic, 2017.

6.1.2 Global errors

Consider a constant sub-interval size $x_{i+1} - x_i = \Delta x$. The whole domain of integration $[a, b]$ is divided into $N = \frac{b-a}{\Delta x}$ sub-intervals. Therefore, when the local error scales with $\mathcal{O}(\Delta x^n)$, the global errors scales with $N\mathcal{O}(\Delta x^n) = \mathcal{O}(\Delta x^{n-1})$.

References

Baydin, Atilim Gunes et al. (2018). *Automatic differentiation in machine learning: a survey.* URL: https://arxiv.org/abs/1502.05767.

Blondel, Mathieu et al. (2022). *Efficient and Modular Implicit Differentiation.* URL: https://arxiv.org/abs/2105.15183.

Bryant, Randal E and David Richard O'Hallaron (2011). *Computer systems: a programmer's perspective.* Prentice Hall.

Cormen, Thomas H et al. (2022). *Introduction to algorithms.* MIT press.

Fejzic, Hajrudin (2017). *A short proof of the error term in Simpson's rule.* URL: https://arxiv.org/abs/1708.07727.

Higham, Nicholas J. (2002). *Accuracy and Stability of Numerical Algorithms.* Second. Society for Industrial and Applied Mathematics. DOI: 10.1137/1.9780898718027. URL: https://epubs.siam.org/doi/abs/10.1137/1.9780898718027.

Kidger, Patrick (2021). »On Neural Differential Equations«. PhD thesis. University of Oxford.

Rackauckas, Christopher et al. (Nov. 2022). *SciML/SciMLBook: v1.1.* Version v1.1. DOI: 10.5281/zenodo.7347643. URL: https://doi.org/10.5281/zenodo.7347643.

Saad, Youcef and Martin H. Schultz (1986). »GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems«. In: *SIAM Journal on Scientific and Statistical Computing* 7.3, pages 856–869. DOI: 10.1137/0907058. URL: https://doi.org/10.1137/0907058.

Sauer, Timothy (2025). *Numerical analysis.* BoD–Books on Demand.

Strassen, Volker (1969). »Gaussian elimination is not optimal«. In: *Numerische mathematik* 13.4, pages 354–356.

Stumm, Philipp and Andrea Walther (2010). »New algorithms for optimal online checkpointing«. In: *SIAM Journal on Scientific Computing* 32.2, pages 836–854.

Wright, Stephen, Jorge Nocedal, et al. (1999). »Numerical optimization«. In: *Springer Science* 35.67-68, page 7.

A Appendices

A.1 Vector and matrix norms

A **vector norm** is a function $||\cdot|| : \mathbb{C}^n \to \mathbb{R}$ satisfying

- $\|\underline{x}\| \geq 0$ with equality if and only if $\underline{x} = 0$ (nonnegativity),
- $\|\alpha\underline{x}\| = |\alpha|\|\underline{x}\|$ for all $\alpha \in \mathbb{C}, \underline{x} \in \mathbb{C}^n$ (homogeneity),
- $\|\underline{x} + \underline{y}\| \leq \|\underline{x}\| + \|\underline{y}\|$ for all $\underline{x}, \underline{y} \in \mathbb{C}^n$ (the triangle inequality)

Hölder p-norms form an important class of norms given by

$$||\underline{x}||_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}, \quad p \geq 1 \tag{191}$$

e.g. the Manhattan norm $||\underline{x}||_1$, the Euclidian norm $||\underline{x}||_2$ or the ∞-norm $||\underline{x}||_\infty = \lim_{p\to\infty} ||\underline{x}||_p = \max_{1\leq i\leq n} |x_i|$.

Similarly, **matrix norms** are functions $||\cdot|| : \mathbb{C}^{m\times n} \to \mathbb{R}$ fulfilling nonnegativity, homogeneity and the triangle inequality. A vector norm on $\mathbb{C}^n$ induces a corresponding subordinate matrix norm on $\mathbb{C}^{m\times n}$ with

$$||\underline{\underline{A}}|| = \max_{\underline{x}\neq 0} \frac{||\underline{\underline{A}}\underline{x}||}{||\underline{x}||}, \tag{192}$$

for instance

$$\begin{aligned} ||\underline{\underline{A}}||_1 &= \max_{1\leq j\leq n} \sum_{i=1}^{m} |a_{ij}| \\ ||\underline{\underline{A}}||_\infty &= \max_{1\leq i\leq m} \sum_{j=1}^{n} |a_{ij}| \\ ||\underline{\underline{A}}||_2 &= \text{maximum singular value of } \underline{\underline{A}} \end{aligned} \tag{193}$$

A.2 Repetition on gradients

The **directional derivative** of a function $f : \mathbb{R}^p \to \mathbb{R}$ at $\underline{w}$ in direction $\underline{v}$ is given by

$$\partial f(\underline{w})[\underline{v}] := \lim_{\delta\to 0} \frac{f(\underline{w} + \delta\underline{v}) - f(\underline{w})}{\delta} \tag{194}$$

f is **(Frechet) differentiable** at $\underline{w}$, if $\partial f(\underline{w})[\underline{v}]$ is

- defined along any direction
- linear in any direction $\underline{v}$
- and if

$$\lim_{||\underline{v}||_2 \to 0} \frac{|f(\underline{w}+\underline{v}) - f(\underline{w}) - \partial f(\underline{w})[\underline{v}]|}{||\underline{v}||_2} = 0 \tag{195}$$

The **gradient** is defined as

$$\underline{\nabla} f(\underline{w}) := \begin{pmatrix} \partial_1 f(\underline{w}) \\ \vdots \\ \partial_p f(\underline{w}) \end{pmatrix} = \begin{pmatrix} \partial f(\underline{w}) \left[\underline{\hat{e}}_1\right] \\ \vdots \\ \partial f(\underline{w}) \left[\underline{\hat{e}}_p\right] \end{pmatrix} \in \mathbb{R}^p \tag{196}$$

and by the linearity of the directional derivative $\partial f(\underline{w})[\underline{v}]$ with respect to $\underline{v}$, we can **express the directional derivative in terms of the gradient**

$$\partial f(\underline{w})[\underline{v}] = \underline{v} \cdot \underline{\nabla} f(\underline{w}) \tag{197}$$

A.3 Gauss-Seidel iteration | better splitting method*

Idea: In each step $\underline{x}^{(n)} \to \underline{x}^{(n+1)}$ ($\underline{x} \in \mathbb{R}^N$) consists of N scalar updates. If we do these N scalar updates iteratively (not in parallel) we can use the results from previous updates.

A.3.1 Motivational Example

For instance in the example above

$$\underline{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \to \quad \underline{x}^{(1)} = \begin{pmatrix} \frac{1}{10} b_0 + \frac{1}{10} \cdot 5 x_2^{(0)} \\ \frac{1}{10} b_1 + \frac{1}{10} \cdot \left(5 x_1^{(1)} + 5 x_3^{(0)}\right) \\ \frac{1}{5} b_1 + \frac{1}{5} \cdot 5 x_2^{1} \end{pmatrix} \tag{198}$$

A.3.2 Gauss-Seidel update

Let us devise a different fix-point equation

$$(\underline{\underline{D}} - \underline{\underline{L}})\underline{x} = \underline{\underline{U}}\underline{x} + \underline{b} \to \underline{x} = (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{\underline{U}}\underline{x} + (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{b} \tag{199}$$

from which we follow the fix-point iteration

$$\underline{x}^{(n+1)} = (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{\underline{U}}\,\underline{x}^{(n)} + (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{b} \tag{200}$$

Problem: We cannot easily compute $(\underline{\underline{D}} - \underline{\underline{L}})^{-1}$.

Therefore, multiply with $(\underline{\underline{D}} - \underline{\underline{L}})$.

$$(\underline{\underline{D}} - \underline{\underline{L}})\underline{x}^{(n+1)} = \underline{\underline{U}}\,\underline{x}^{(n)} + \underline{b} \quad \rightarrow \quad \underline{x}^{(n+1)} = \underline{\underline{D}}^{-1}\underline{\underline{L}}\,\underline{x}^{(n+1)} + \underline{\underline{D}}^{-1}\underline{\underline{U}}\,\underline{x}^{(n)} + \underline{\underline{D}}^{-1}\underline{b} \tag{201}$$

But isn't this implicit now with $\underline{x}^{(n+1)}$ on both sides?: $\underline{\underline{L}}$ is a lower diagonal matrix (diagonal and everything above zero), so in $\underline{\underline{L}}\,\underline{x}^{(n+1)}$ we always only need values we have already calculated if we solve consecutively ($\rightarrow$ problem for parallelization). This is illustrated in fig. 11.

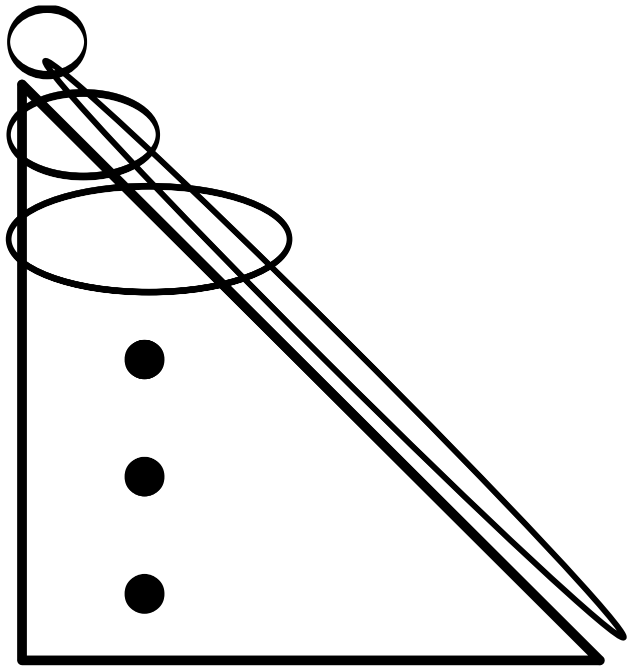


Figure 11: As of $\underline{\underline{L}}\,\underline{x}^{(n+1)}$ to calculate $x_i^{(n+1)}$ we only need $x_0^{(n+1)}, \ldots, x_{i-1}^{(n+1)}$ which we have already calculated.

Advantage of Gauss-Seidel: Convergence is sped up (often by a factor of 2) compared to Jacobi iteration. Convergence is guaranteed if $\underline{\underline{A}}$ is strictly diagonally dominant $|A_{ii}| > \sum_{j\neq i} |A_{ij}| \quad \forall i$ or symmetric and positive definite.

A.4 Krylow subspace methods

Our aim still is to solve a linear system $\underline{\underline{A}}\,\underline{x} = \underline{b}$.

Problem: LU-decomposition has cubic runtime ($\sim \frac{2}{3}n^3$) and the previously discussed iterative methods might also not always be optimal or straight infeasible if $\underline{\underline{A}}$ cannot be stored explicitly.

A.4.1 Motivation for the need of Krylow subspace methods in the context of non-linear root-finding of big systems*

Consider a non-linear equation

$$\underline{0} = \underline{g}(\underline{\xi}) \tag{202}$$

which we want to solve for $\underline{\xi}$.[3] A common root-finding approach is Newton-Iteration[4]

$$\underline{\xi}_{k+1} = \underline{\xi}_k - \underline{\underline{J}}_g^{-1}(\underline{\xi}_k)\underline{g}(\underline{\xi}_k), \quad \text{Jacobian } \underline{\underline{J}}_g \tag{204}$$

where at the lowest level we have to solve the linear equation

$$\underline{b} := \underline{g}(\underline{\xi}_k) = \underline{\underline{J}}_g(\underline{\xi}_k - \underline{\xi}_{k+1}) = \underline{\underline{J}}_g\underline{a} \tag{205}$$

for $\underline{a}$.

Problem: But what if the resulting Jacobian is too large to be stored?

---

[3]For instance an implicit Euler step can be formulated as such a root finding problem.

$$\begin{aligned} &\text{implicit step } \underline{y}^{(n+1)} = \underline{y}^{(n)} + \Delta t \underline{f}\left(\underline{y}^{(n+1)}\right) \\ \rightarrow \quad &\text{root-finding-problem } \underline{0} = \underline{y}^{(n+1)} - \underline{y}^{(n)} - \Delta t \underline{f}\left(\underline{y}^{(n+1)}\right) =: \underline{g}(\underline{\xi}) \end{aligned} \tag{203}$$

[4]Which has quadratic convergence - as the method converges on the root, the difference between the root and the approximation is squared in each step.

Idea: A directional derivative (a Jacobian vector product[a]) can be calculated in one (forward) automatic differentiation pass - cheaply and storage efficient. So what if we could solve $\underline{\underline{J}}\underline{a} = \underline{b}$ only based on Jacobian vector products?

[a]
$$\underline{\underline{J}}\underline{v} = (\underline{\nabla} \cdot \underline{g})\underline{v} = \lim_{h\to 0} \frac{g(\underline{x} + \underline{v}h) - g(\underline{x})}{h}$$
$$\left(\text{as } (\underline{\underline{J}}\underline{v})_i = \sum_{j=1}^{n} \left(\underline{\underline{J}}_g\right)_{ij} v_j = \sum_{j=1}^{n} \frac{\partial g_i}{\partial x_j} v_j = ((\underline{\nabla} \cdot \underline{g})\underline{v})_i \right) \tag{206}$$

A.4.2 Motivation | solving a system of linear equations using a gradient method

Consider $\underline{\underline{A}} \in \mathbb{R}^{N\times N}$ symmetric ($\underline{\underline{A}}^T = \underline{\underline{A}}$) and positive definite, so $\underline{x}^T\underline{\underline{A}}\underline{x} > 0 \forall \underline{x} \in \mathbb{K}^N \setminus \{\underline{0}\}$ (only positive eigenvalues). Then the following holds

$$\underline{\underline{A}}\underline{x} = \underline{b} \leftrightarrow \text{quadratic form } f(x) := \frac{1}{2}\underline{x}^T\underline{\underline{A}}\underline{x} - \underline{x}^T\underline{b} \text{ is minimal} \tag{207}$$

(intuitively as the quadratic form for a positive definite matrix is parabolic) where we can now apply standard gradient descent (**steepest descent method**)

$$\underline{x}^{(n+1)} = \underline{x}^{(n)} + \alpha^{(n)}\underline{p}^{(n)}, \quad \underline{p}^{(n)} = -\underline{\nabla} f(x^{(n)}) = -(\underline{b} - \underline{\underline{A}}\underline{x}^{(n)}) \tag{208}$$

so we take steps along the steepest descent with some *learning rate* $\alpha^{(n)}$

Note: Quadratic forms for different kinds of matrices are illustrated in Fig. 12. For an indifinite matrix the steepest descent method or the later-introduces conjugate gradient method will fail, because the solution is a saddle point. For the more general case of even non-symmetric matrices, GMRES can be used.

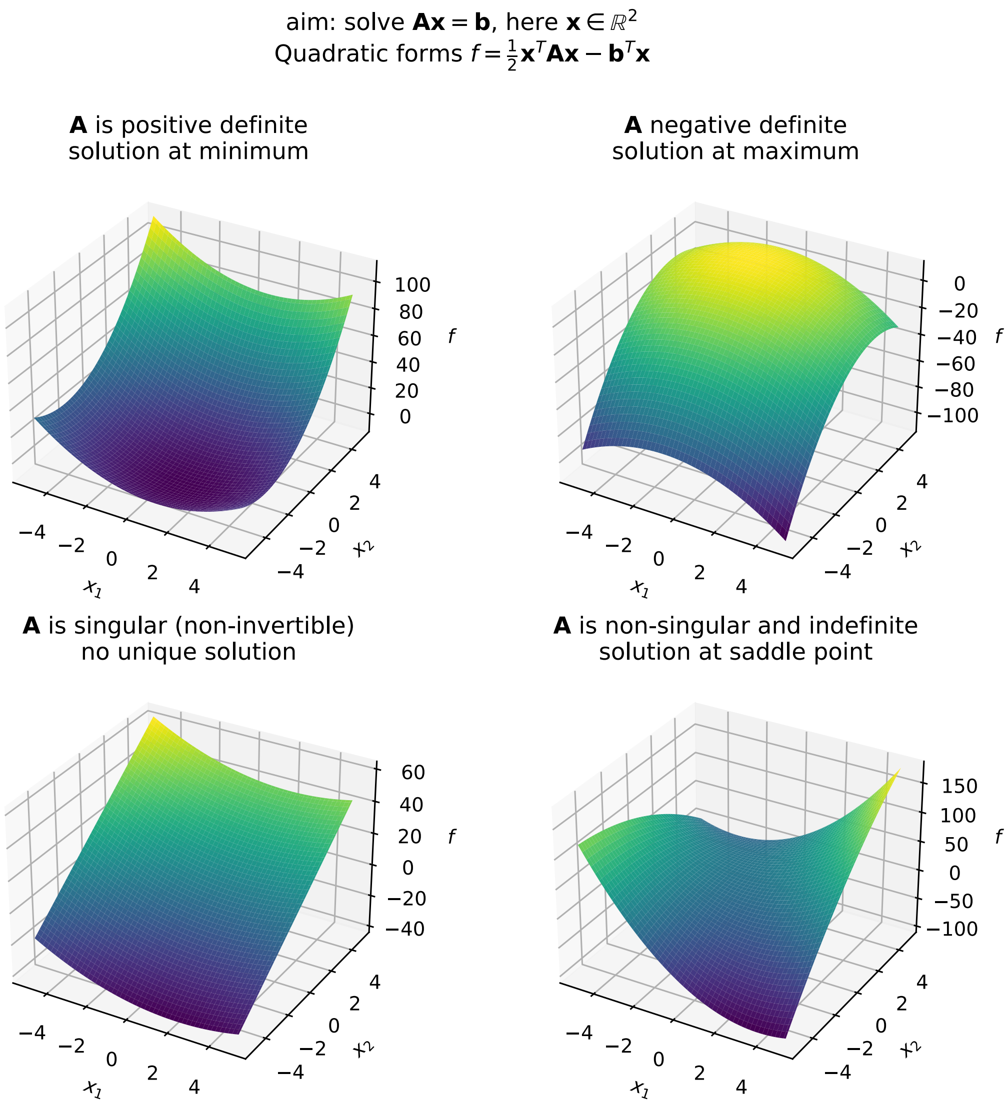


Figure 12: Quadratic forms for different kinds of matrices.

Problem: Steepest descent often takes steps in directions already taken. Steepest descent is good if our parabolic quadratic form f is nearly circular but the more elliptic f is, the more problematic steepest descent becomes, see Fig. 13.

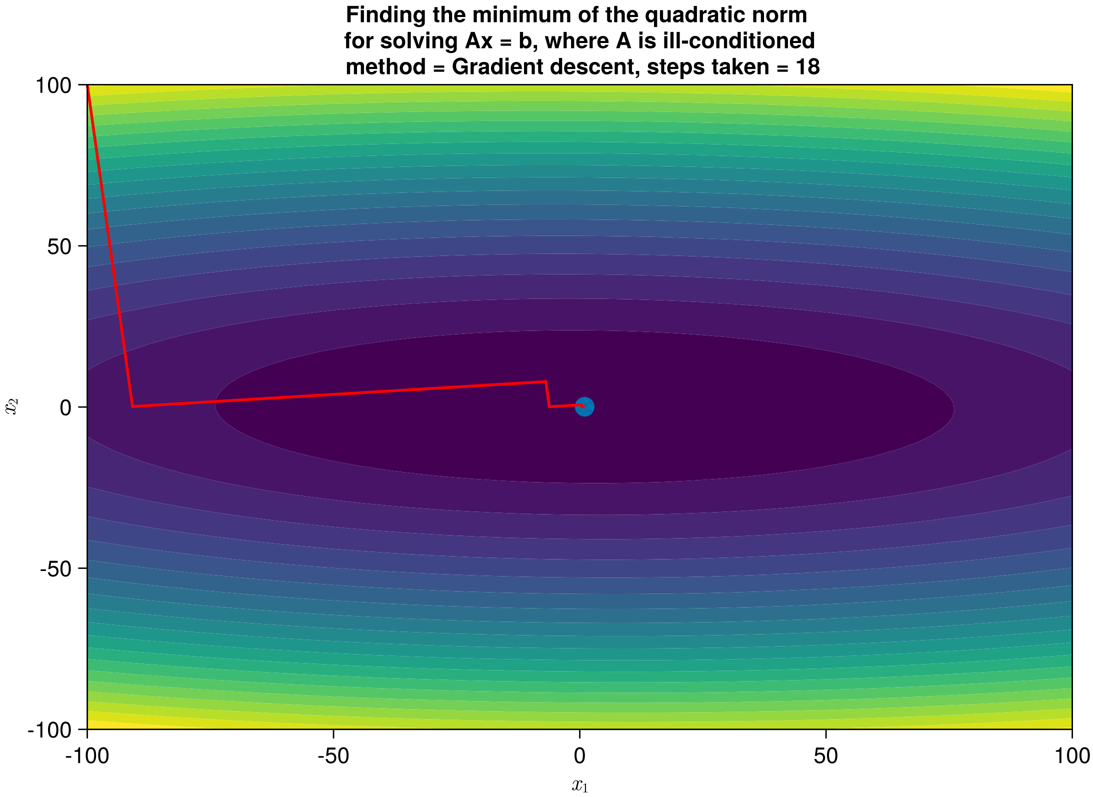


Figure 13: Steepest descent for solving a linear system $\underline{\underline{A}}\,\underline{x} = \underline{b}$ with positive definite matrix $\underline{\underline{A}}$.

Idea: Is there maybe a smarter way to choose directions $\underline{p}^{(n)}$ so that we do not go into the same direction multiple times and still find the solution?

We will introduce the conjugate gradient method, which takes at most N (size of the linear system) steps to converge to the solution, see Fig. 14.

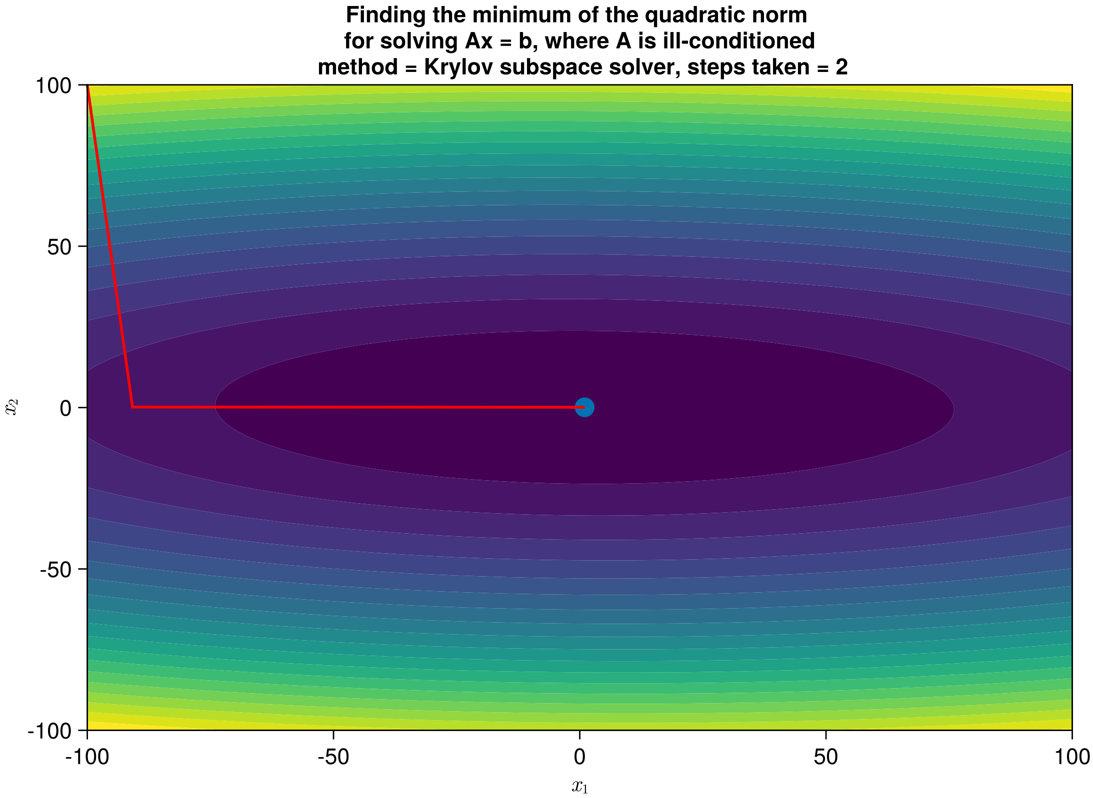


Figure 14: Conjugate gradient method for solving a linear system $\underline{\underline{A}}\,\underline{x} = \underline{b}$ with positive definite matrix $\underline{\underline{A}}$.

A.4.3 Krylov subspace and construction of iterative methods for solving $\underline{\underline{A}}\,\underline{x} = \underline{b}$

Consider the simple fixed-point iteration

$$(\underline{\underline{A}} + \underline{\underline{1}} - \underline{\underline{1}})\underline{x} = b \rightarrow \underline{x}^{(n+1)} = (\underline{\underline{1}} - \underline{\underline{A}})x^{(n)} + \underline{b} \tag{209}$$

then for $x^{(0)} = \underline{b}$

$$\underline{x}^{(1)} = 2\underline{b} - \underline{\underline{A}}b, \quad \underline{x}^{(2)} = 3\underline{b} - 3\underline{\underline{A}}\,\underline{b} + \underline{\underline{A}}^2\underline{b}, \quad \ldots \tag{210}$$

- it is only based on simple matrix vector products.

The Krylov subspace of order r is based on

$$\underline{b}, \underline{\underline{A}}\,\underline{b}, \underline{\underline{A}}^2\underline{b}, \ldots, \underline{\underline{A}}^{r-1}\underline{b} \tag{211}$$

forming a linear vector space

$$\mathcal{K}_r := \mathcal{K}_r(\underline{\underline{A}}, \underline{b}) = \operatorname{span}(\underline{b}, \underline{\underline{A}}\,\underline{b}, \underline{\underline{A}}^2\underline{b}, \ldots, \underline{\underline{A}}^{r-1}\underline{b}) \tag{212}$$

note that

- $\underline{\underline{A}}^i \underline{b}$ can be calculated by i matrix-vector products (very efficient)
- $\underline{\underline{A}}^i \underline{b}$ are linearly independent for each $i < N$, the dimension of $\mathcal{K}_r$ increases with r up to N, the dimension of $\underline{\underline{A}}$

How can we smartly (smarter than in the simple scheme above) choose $\underline{x}^{(n)} \in \mathcal{K}_n$ in an iterative scheme for solving $\underline{\underline{A}}\underline{x} = \underline{b}$?

- so that the residual $\underline{r}^{(n)} = \underline{b} - \underline{\underline{A}}\underline{x}^{(n)}$ is orthogonal to $\mathcal{K}_n$ (so $\in \mathcal{K}_{n+1}$) (**conjugate gradients**)
- so that the residual has minimum norm for $\underline{x}^{(n)} \in \mathcal{K}_n$ (GMRES)
- ...

Note: The best basis for the Krylov subspace $\mathcal{K}_r$ is orthonormal and can be computed using Arnoldi's method (essentially Gram-Schmidt). Since $\underline{r}^{(n)} \perp \mathcal{K}_n$ it will be a scaled version of the last orthonormal base vector, one needs to get from $\mathcal{K}_n$ to $\mathcal{K}_{n+1}$

A.4.4 Conjugate gradients method

The sequence

$$\underline{x}^{(n+1)} = \underline{x}^{(n)} + \alpha^{(n)} \underline{p}^{(n)}, \quad \text{residual } \underline{r}^{(n)} = \underline{b} - \underline{\underline{A}}\underline{x}^{(n)} \tag{213}$$

with (without proof)

$$\underline{p}^{(n)} = \underline{r}^{(n)} + \frac{\underline{r}^{(n)} \cdot \underline{r}^{(n)}}{\underline{r}^{(n-1)} \underline{r}^{(n-1)}} \underline{p}^{(n-1)} \tag{214}$$

and *learning rate*[5]

$$\alpha^{(n)} = -\frac{\left(\underline{p}^{(n)}\right)^T \underline{r}^{(n)}}{\left(\underline{p}^{(n)}\right)^T \underline{\underline{A}}\underline{p}^{(n)}} \tag{215}$$

solves $\underline{\underline{A}}\underline{x} = \underline{b}$, $\underline{\underline{A}} \in \mathbb{R}^{N \times N}$, $\underline{\underline{A}}$ symmetric and positive definite, in at most N steps, so $\underline{r}^{(N)} = \underline{0}$ (here stated without proof).

The directions taken are constructed so that

- the residuals are orthogonal, $\underline{r}^{(i)} \cdot \underline{r}^{(j)} = 0$ for $i \neq j$
- the step-directions are conjugate, $\left(\underline{p}^{(j)}\right)^T \underline{\underline{A}}\underline{p}^{(j)} = 0$ for $i \neq j$
- the residuals and step-directions are mutually orthogonal, $\underline{p}^{(i)} \cdot \underline{r}^{(j)} = 0$ for $i \neq j$

[5] This follows simply from $\partial_x f|_{\underline{x}^{(n)} + \alpha^{(n)} \underline{p}^{(n)}} \underset{!}{=} 0$, where f is the quadratic form of $\underline{\underline{A}}\underline{x} = \underline{b}$.

A.5 Multigrid techniques

We have now gained an understanding of iteratively solving a linear equation as a relaxation problem.

Note: In each update step information travels as given by the stencil, for the Poisson equation only between nearest neighbors. This also limits the largest possible *timestep* to the CFL criterion.

Problem: As only cells in the stencil (often only neighboring cells) communicate per step in Jacobi and Gauss-Seidel iteration, we have to make a compromise between speed of convergence and resolution:

- On a coarse grid, information travels quicker through space (in less steps) but the resolution is bad
- On a fine grid, resolution is good, but long range interactions take lots of computational steps and long-wavelength errors (in the error vector $\underline{e}$) die out only very slowly

Idea: Start on a coarse grid, where information travels quickly, long range correlations are taken care of and we have fast convergence, then get the information onto a fine grid and resolve the details.

1. But how can we map from a coarser to a finer grid and vice versa?
2. How can we solve $\underline{\underline{A}}\,\underline{x} = \underline{b}$ on the coarser grid?

Note: Coarse-to-fine is called prolongation, fine-to-coarse restriction (see Fig. 15).

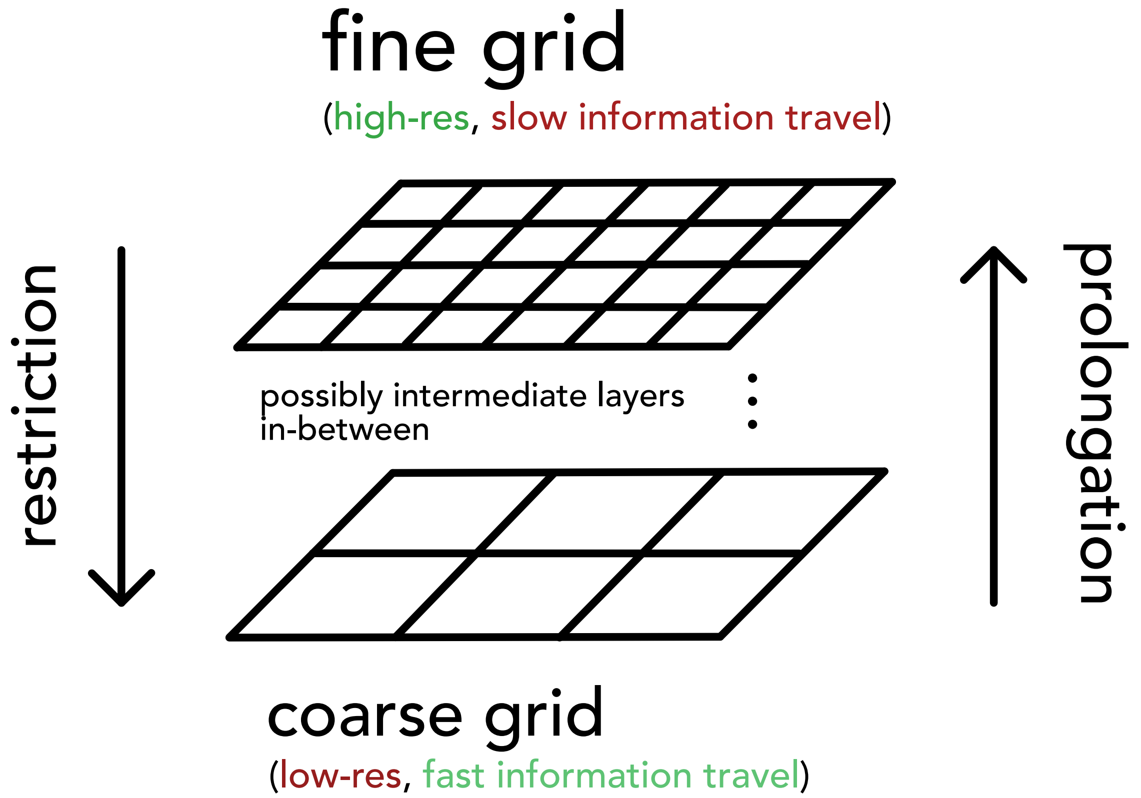


Figure 15: Grids of different resolution.

A.5.1 Getting finer and coarser | prolongation and restriction

Note: In the following we will only consider the case of a 1D grid (not 2D as illustrated before).

Consider meshes

1. Let $\Omega^{(h)}$ denote a 1D mesh with N cells $i = 1, \ldots, N$ with spacing h.
2. Let $\Omega^{(2h)}$ denote a 1D mesh with $N/2$ cells $i = 1, \ldots, N/2$ with spacing $2h$.

and let

1. $\underline{x}_i^{(h)} \in \mathbb{R}^N$ denote the solution on $\Omega^{(h)}$, so $x_i^{(h)}$ is the solution on cell i of $\Omega^{(h)}$ (e.g. a density $\rho_i^{(h)}$).
2. $\underline{x}_i^{(2h)} \in \mathbb{R}^{N/2}$ denote the solution on $\Omega^{(2h)}$, so $x_i^{(2h)}$ is the solution on cell i of $\Omega^{(2h)}$.

Restriction and prolongation in 1D are illustrated in Fig. 16.

Prolongation and restriction are linear operators written as matrices $\underline{\underline{I}}_{\text{from this spacing}}^{\text{to this spacing}}$.

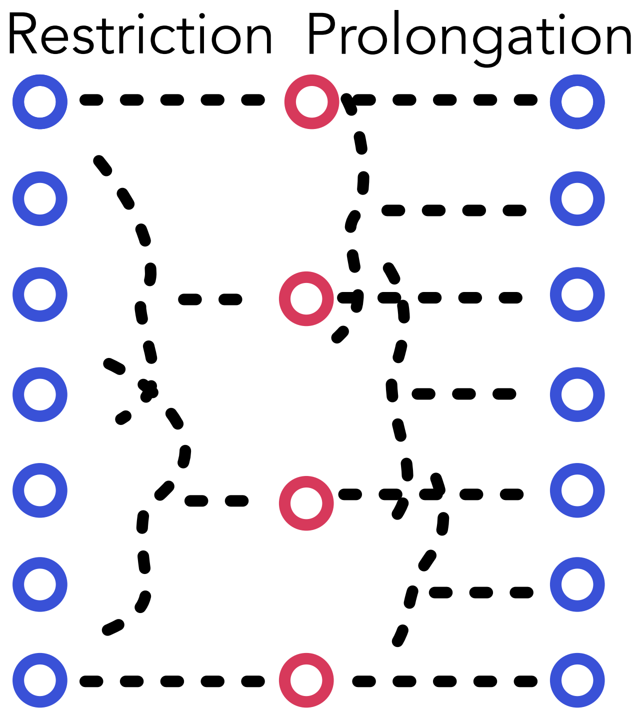


Figure 16: Illustration of restriction and prolongation.

A.5.1.1 Coarse-to-fine | interpolation called prolongation

We map from coarse to fine using

$$\underline{\underline{I}}_{2h}^{h} \underline{x}^{(2h)} = \underline{x}^{(h)} \tag{216}$$

for instance by

$$\underline{\underline{I}}_{2h}^{h} \in \mathbb{R}^{N \times \frac{N}{2}} : \text{for } 0 \leq i < \frac{N}{2} : x_{2i}^{(h)} = x_{i}^{(2h)}, \quad x_{2i+1}^{(h)} = \frac{1}{2}\left(x_{i}^{(2h)} + x_{i+1}^{(2h)}\right) \tag{217}$$

as shown in Fig. 16 and furthermore in Fig. 17.

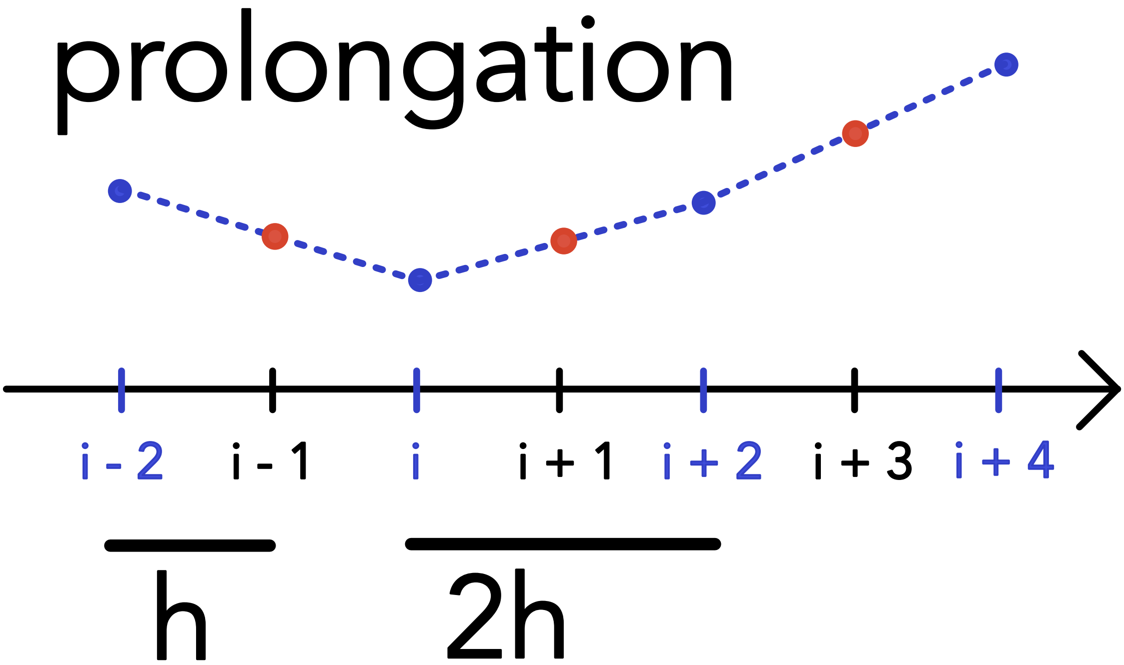


Figure 17: 1D-Prolongation from coarse to fine grid.

A.5.1.2 Fine-to-coarse | restriction

We now convert

$$\underset{=h}{I}^{2h} \underline{x}^{(h)} = \underline{x}^{(2h)} \tag{218}$$

with a simple example being

$$\underset{=h}{I}^{2h} \in \mathbb{R}^{\frac{N}{2} \times N} : \text{for } 0 \leq i < \frac{N}{2} : x_i^{(2h)} = \frac{x_{2i-1}^{(h)} + 2x_{2i}^{(h)} + x_{2i+1}^{(h)}}{4} \tag{219}$$

A.5.1.3 Relation of restriction and prolongation

Prolongation and restriction matrices are oftentimes constructed to be scaled transposes of each other, so

$$\underset{=h}{I}^{2h} = c[\underset{=2h}{I}^{h}]^T \tag{220}$$

Consider for instance

$$\text{prolongation: } \underset{=2h}{I^h} = \frac{1}{2}\begin{pmatrix} 1 & & \\ 2 & & \\ 1 & 1 & \\ & 2 & \\ & 1 & 1 \\ & & 2 \\ & & 1 \end{pmatrix} \tag{221}$$

$$\text{restriction: } \underset{=h}{I^{2h}} = \frac{1}{2}\left(\underset{=2h}{I^h}\right)^T = \frac{1}{4}\begin{pmatrix} 1 & 2 & 1 & & & & \\ & & 1 & 2 & 1 & & \\ & & & & 1 & 2 & 1 \end{pmatrix}$$

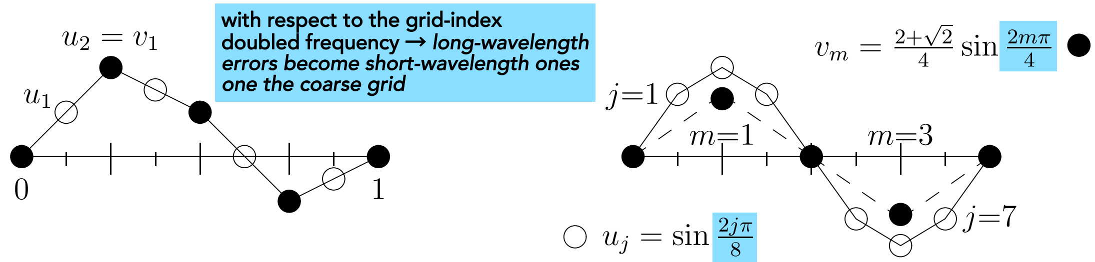
so the application of **prolongation** means

$$\frac{1}{2}\begin{pmatrix} 1 & & \\ 2 & & \\ 1 & 1 & \\ & 2 & \\ & 1 & 1 \\ & & 2 \\ & & 1 \end{pmatrix}\begin{pmatrix} x_1^{(2h)} \\ x_2^{(2h)} \\ x_3^{(2h)} \end{pmatrix} = \begin{pmatrix} x_1^{(2h)}/2 \\ x_1^{(2h)} \\ x_1^{(2h)}/2 + x_2^{(2h)}/2 \\ x_2^{(2h)} \\ x_2^{(2h)}/2 + x_3^{(2h)}/2 \\ x_3^{(2h)} \\ x_3^{(2h)}/2 \end{pmatrix} = \begin{pmatrix} x_1^{(h)} \\ x_2^{(h)} \\ x_3^{(h)} \\ x_4^{(h)} \\ x_5^{(h)} \\ x_6^{(h)} \\ x_7^{(h)} \end{pmatrix} \tag{222}$$

and **restriction**

$$\frac{1}{4}\begin{pmatrix} 1 & 2 & 1 & & & & \\ & & 1 & 2 & 1 & & \\ & & & & 1 & 2 & 1 \end{pmatrix}\begin{pmatrix} x_1^{(h)} \\ x_2^{(h)} \\ x_3^{(h)} \\ x_4^{(h)} \\ x_5^{(h)} \\ x_6^{(h)} \\ x_7^{(h)} \end{pmatrix} = \begin{pmatrix} x_1^{(2h)} \\ x_2^{(2h)} \\ x_3^{(2h)} \end{pmatrix} \tag{223}$$

Both are illustrated in Fig. 18, also illustrating that with respect to the grid index, low-frequency errors on the fine grid have double (so higher) frequency on the coarse grid.



(a) Linear interpolation by $u = I_{2h}^{h} v$ (b) Restriction by $R_h^{2h} u = \frac{1}{2}(I_{2h}^{h})^{\mathrm{T}} u$

Figure 18: Prolongation and restriction of a sine wave (figure by Gilbert Strang).

Note: Taking prolongation and restriction as transposes of each others breaks at the boundaries, if we do not as in Fig. 18 have zeroes on the boundaries.

A.5.1.4 Short-hand stencil notation

Short-hand notations for prolongation and restriction are given in table 2.

Prolongation	**Restriction**
We can write prolongation as $$\text{1D prolongation: } \underline{\underline{I}}_{2h}^{h} : \Big] \; \tfrac{1}{2} \quad 1 \quad \tfrac{1}{2} \; \Big[\tag{224}$$ meaning that every coarse point is added to three fine points with these respective weights (see previous example).	We can write restriction as $$\text{1D restriction: } \underline{\underline{I}}_{h}^{2h} : \Big[\; \tfrac{1}{4} \quad \tfrac{1}{2} \quad \tfrac{1}{4} \; \Big] \tag{225}$$ - the coarse point is the weighted sum of the three fine points.

Table 2: Short-hand notations for prolongation and restriction.

Similar short-hand notations can be devised for 2D and 3D.

A.5.2 Multigrid V-cycle

Our aim is still to solve $\underline{\underline{A}}\,\underline{x} = \underline{b}$ smartly, by - in the view of a relaxation problem - doing the rough large-scale work quickly on a coarse and the fine details on a fine grid afterwards. We still have to anwer

- How can such a procedure look like?
- How do we convert $\underline{\underline{A}}$ to coarser grids?

A.5.2.1 Initial definitions

The error is defined as

$$\underline{e} \equiv \underline{x}_{\text{exact}} - \underline{\tilde{x}}, \quad \text{current approximate solution } \underline{\tilde{x}} \tag{226}$$

Note: The residual is the error in the solution, not $\underline{x}$. The error and residual are related linearly by $\underline{\underline{A}}$.

$$\underline{\underline{A}}\underline{x}_{\text{exact}} = \underline{b} \quad \rightarrow \quad \underline{\underline{A}}(\underline{e} + \underline{\tilde{x}}) = \underline{b} \quad \rightarrow \quad \underline{\underline{A}}\underline{e} = \underline{r} \tag{227}$$

Idea: Consider we have an initial guess $\underline{\tilde{x}}$ on the fine grid. We can easily calculate the residual $\underline{r}$. We can then restrict $\underline{r}$ to a coarse grid, solve for the error there ($\underline{\underline{A}}\underline{e} = \underline{r}$) (given we can transform $\underline{\underline{A}}$ to the coarse grid) and then prolong the error to the fine grid and make the correction $\underline{x}_{\text{exact}} = \underline{\tilde{x}} + \underline{e}$.

Let us formalize this idea.

A.5.2.2 Coarse-grid correction scheme

We use the error calculated on the coarse grid to correct on the fine grid.

Let us start with a (fine-grid) guess $\underline{\tilde{x}}^{(h)}$ for the problem

$$\underline{\underline{A}}^{(h)}\underline{x}^{(h)} = \underline{b}^{(h)} \tag{228}$$

and we want to obtain an improvement $\underline{\tilde{x}}'^{(h)} = CG(\underline{\tilde{x}}^{(h)}, \underline{b}^{(h)})$.

CG consists of the following steps

1. Perform one iterative *relaxation* step on the current grid h, e.g. Jacobi or Gauss-Seidel
2. Compute the residual
$$\underline{r}^{(h)} = \underline{b} - \underline{\underline{A}}\underline{\tilde{x}}^{(h)} \tag{229}$$
3. Restrict the residual to the coarser mesh
$$\underline{r}^{(2h)} = \underline{\underline{I}}_h^{2h}\underline{r}^{(h)} \tag{230}$$
4. Solve for the error on the coarser mesh
$$\underline{\underline{A}}^{(2h)}\underline{e}^{(2h)} = \underline{r}^{(2h)}, \quad \text{starting guess } \underline{\tilde{e}}^{(2h)} = \underline{0} \tag{231}$$

5. Correct $\underline{\tilde{x}}^{(h)}$ on the finer mesh using the prolonged error $\underline{e}^{(h)} = \underline{\underline{I}}_{2h}^{h} \underline{e}^{2h}$

$$\underline{\tilde{x}}'^{(h)} = \underline{\tilde{x}}^{(h)} + \underline{e}^{(h)} \tag{232}$$

6. Further iterative *relaxation* step on the fine mesh

How to solve for the error on the coarse mesh?: Step 4 can be performed by recursively calling $CG(\underline{\tilde{x}}^{(2^i h)}, \underline{b}^{(2^i h)}), i \geq 1$ until a coarseness is reached where one can easily exactly solve or solve by multiple relaxation steps.

How to find $\underline{\underline{A}}^{(2h)}$ on the coarse mesh?: Generally in the **Galerkin coarse grid approximation** one defines

$$\underline{\underline{A}}^{(2h)} = \underline{\underline{I}}_{h}^{2h} \underline{\underline{A}}^{(h)} \underline{\underline{I}}_{2h}^{h} \tag{233}$$

which is an additional matrix operation that might enlarge the stencil (and so the computational cost), depending on the interpolation operator.

When $\underline{\underline{A}}$ was brought forth by formulating a discretization in matrix form, we can use the same discrete equations as one the fine grid (**direct method**). In other words, the same stencil (*Schablone*) is used to go over the points and collect for the linear relations, e.g. for the 2D-Poisson - no matter the coarseness - we would construct $\underline{\underline{A}}$ based on eq. **??**.

A.5.2.3 V-cycle

The 6-step iteration process with a recursion call in step 4 leads to a cycle as illustrated in Fig. 19.

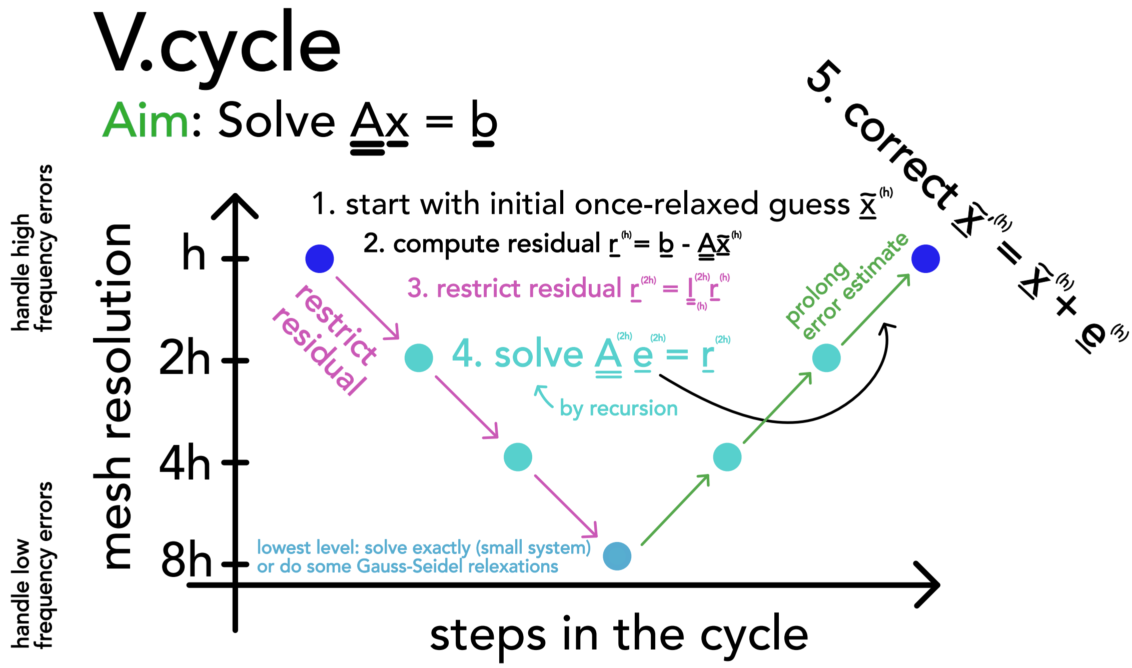


Figure 19: V-cycle.

The V-cycle for the Poisson equation has the same cost as FFT-based methods but requires less thread communication when parallelized.

Computational costs

$$\begin{gathered}\text{computational cost per V-cycle:}\mathcal{O}(N_{\text{grid}})\\ \text{computational cost until convergence to machine error (mult. cycles) } \mathcal{O}(N_{\text{grid}}\log N_{\text{grid}})\\ \text{with } N_{\text{grid}} \text{ being the number of grid-cells on the fine grid}\end{gathered} \tag{234}$$

A.5.2.4 Full multigrid method

Problem: How to make a good initial guess $\underset{\sim}{\tilde{x}}^{(h)}$ (if not available e.g. by taking the result from the last time-step in a simulation)?

Idea: First solve on the coarsest grid, then interpolate up to get a good initial guess. Based on this idea, the full multigrid cycle is constructed.

The multigrid cycle is illustrated in Fig. 20.

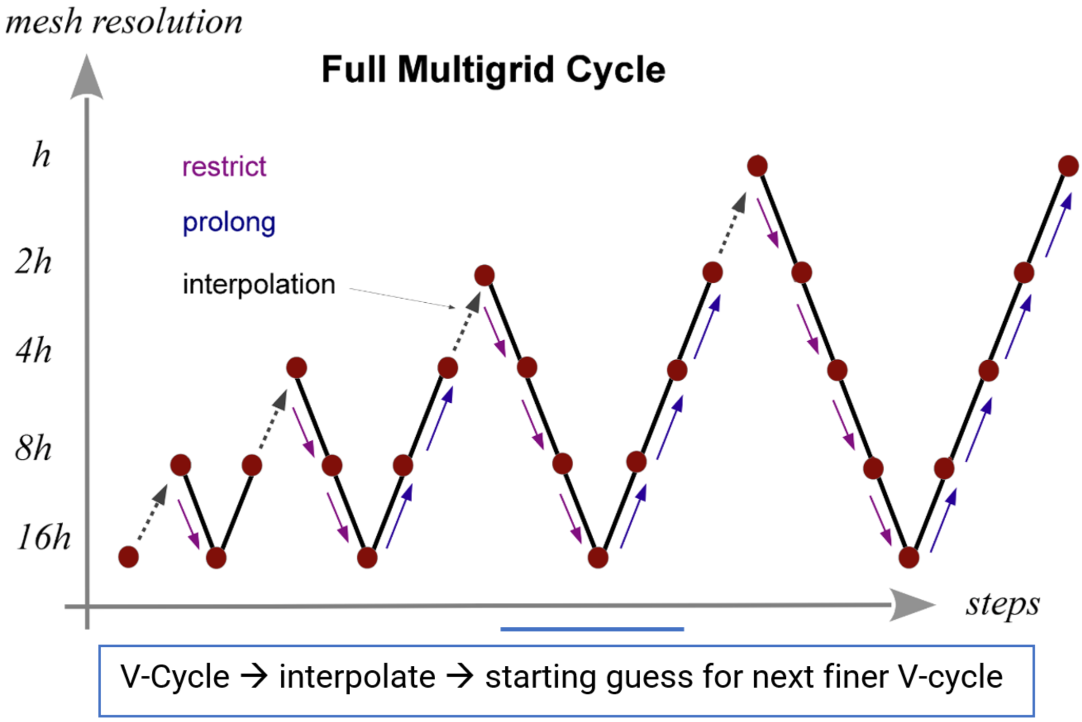


Figure 20: Full multigrid cycle.

The steps are

1. Initialize the righthand side on all grid levels $\underline{b}^{(h)}, \underline{b}^{(2h)}, \underline{b}^{(H)}$.
2. Solve $A^{(H)}\underline{x}^{(H)} = \underline{b}^{(H)}$ exactly on the coarsest level H.
3. Obtain an initial guess for a finer grid by interpolating the solution from the next coarser level below, i.e. $\underline{\tilde{x}}^{(h)} = \underline{\underline{I}}^{h}_{2h}\underline{\tilde{x}}^{(2h)}$
4. Solve the problem in the finer level using this starting guess $\underline{\tilde{x}}^{(h)}$ and one v-cycle.
5. repeat step 3 until finest level is reached

Computational cost of one multigrid cycle: As the V-cycle it has $\mathcal{O}(N_{\text{grid}})$ as N_{grid} is large compared to the cost of the V-cycle hierarchy.

A.6 Introducing Automatic Differentiation

Consider we are interested in the derivatives of a function $f : \mathbb{R}^n \to \mathbb{R}^m$, e.g. the simulation output as a function of the parameters.

Based on the chain rule and known derivatives of a set of primitive operations, we can efficiently and accurately (to machine precision) calculate these derivatives.

Intuitively

- forward-mode automatic differentiation forward propagates a perturbation $\underline{v} \in \mathbb{R}^n$ at $\underline{x}$ (with $\underline{v}$ sufficiently small so we can linearize $\underline{f}$ around $\underline{x}$) in the input space through the computational graph of the function $\underline{f}$ to a resulting perturbation $\underline{\underline{J}}_{\underline{f}}\Big|_{\underline{x}} \underline{v}$ in the output space
- backward-mode automatic differentiation relates output perturbations back to input perturbations, projecting perturbations in the output space $\underline{p} \in \mathbb{R}^m$ onto the output perturbations caused by the base vectors of the input space

$$\underline{\underline{J}}^T_{\underline{f}}\Big|_{\underline{x}} \underline{p} = \begin{pmatrix} - & (\partial_{x_1}\underline{f})^T & - \\ - & (\partial_{x_2}\underline{f})^T & - \\ & \vdots & \\ - & (\partial_{x_n}\underline{f})^T & - \end{pmatrix} \underline{p} = \begin{pmatrix} (\partial_{x_1}\underline{f})^T \underline{p} \\ (\partial_{x_2}\underline{f})^T \underline{p} \\ \vdots \\ (\partial_{x_n}\underline{f})^T \underline{p} \end{pmatrix} \tag{235}$$

for a scalar output and $p = 1$ this amounts to the derivatives of this output with respect to all inputs.

A.6.1 Forward-mode automatic differentiation

In a simulation we typically start at initial conditions $\underline{x} \in \mathbb{R}^N$ on which we apply a chain of transformations, e.g.

$$\underline{f} = \underline{g}_m \circ \cdots \circ \underline{g}_2 \circ \underline{g}_1, \quad \underline{f}(\underline{x}) = \underline{g}_m(\dots \underline{g}_2(\underline{g}_1(\underline{x}))) \tag{236}$$

with for simplicity $\underline{f}, \underline{g}_m, \cdots, \underline{g}_1 : \mathbb{R}^N \to \mathbb{R}^N$. We might be interested in the sensitivity of the simulation output to a wiggle $\underline{v}$ in the initial conditions as captured by the Jacobian vector product (JVP) $\underline{\underline{J}}_{\underline{f}}(\underline{x})\underline{v}$ with $\underline{\underline{J}}_{\underline{f}}(\underline{x}) \in \mathbb{R}^{N\times N}$ if we linearize $\underline{f}$ around $\underline{x}$.

By the chain rule

$$\underline{\underline{J}}_{\underline{f}}(\underline{x}) = \underline{\underline{J}}_{\underline{g}_m}(\underline{g}_{m-1}, \cdots, g_1(\underline{x})) \cdots \underline{\underline{J}}_{\underline{g}_2}(\underline{g}_1(\underline{x}))\underline{\underline{J}}_{\underline{g}_1}(\underline{x}) \tag{237}$$

which we can calculate in $m-1$ matrix-matrix multiplications, each scaling with $\mathcal{O}(N^3)$ (naively) (or e.g. $\mathcal{O}(N^{\log_2 7})$ with Strassen's algorithm (Strassen, 1969)), leading to an overall scaling of calculating the Jacobian of $\mathcal{O}(mN^3)$. But for a JVP we don't actually need to compute the Jacobian, we can just evaluate

$$\underline{\underline{J}}_{\underline{f}}(\underline{x})\underline{v} = \underline{\underline{J}}_{\underline{g}_m}(\underline{g}_{m-1}(\cdots(g_1(\underline{x})))) \cdots \underline{\underline{J}}_{\underline{g}_2}(\underline{g}_1(\underline{x}))\underline{\underline{J}}_{\underline{g}_1}(\underline{x})\underline{v} \tag{238}$$

from right to left with $m-1$ matrix-vector products, each costing only $\mathcal{O}(N^2)$!

If we capture the dependency structure of a composition of functions as a computational graph, with functions as nodes and dependencies as arrows, calculating the JVP amounts to

- starting from the inputs evaluate the nodes to calculate the inputs to the next nodes (*primals*) in a forward pass
- in the same forward pass, starting at a *tangent* $\underline{v}$ in the input space calculate the JVPs of the nodes (evaluated at the *primals*), giving the *tangents* to the following nodes

Forward differentiating through computational graphs is commonly called forward-mode automatic differentiation[6], forwardpropagation, or, drawing from differential geometry terminology, pushforward.[7]

Partial derivatives $\partial_{x_i}\underline{f}$ are JVPs with the basis vectors, $\partial_{x_i}\underline{f} = \underline{\underline{J}}_{\underline{f}}\underline{\hat{e}}_i$.

A.6.1.1 Example of forward differentiation through a computational graph

Consider the following composition of functions

$$\underline{f}(\underline{x}) = \begin{pmatrix} g(h(x_1, x_2)) \\ u(h(x_1, x_2), x_2) \end{pmatrix} \tag{239}$$

with some functions $g : \mathbb{R} \to \mathbb{R}, h : \mathbb{R}^2 \to \mathbb{R}, u : \mathbb{R}^2 \to \mathbb{R}$. As $\underline{f}$ is a composition of functions, a chain of dependencies, it makes sense to illustrate it in a computational graph with functions as nodes and dependencies as arrows, as done in Fig. 21.

Consider a Jacobian-vector-product (JVP), here for our example function $\underline{f}$

$$\underline{\underline{J}}_{\underline{f}}\underline{v} = \begin{pmatrix} (\partial_1 h)\,(\partial_1 g) & (\partial_2 h)\,(\partial_1 g) \\ (\partial_1 h)\,(\partial_1 u) & (\partial_2 u) + (\partial_2 h)\,(\partial_1 u) \end{pmatrix} \underline{v} = \begin{pmatrix} (v_1\,\partial_1 h + v_2\,\partial_2 h)\,\partial_1 g \\ (v_1\,\partial_1 h + v_2\,\partial_2 h)\,\partial_1 u + v_2\,\partial_2 u \end{pmatrix} \tag{240}$$

where ∂_i denotes the partial derivative of a function with respect to its i-th argument.

[6]We will also use *autodiff* to refer to automatic differentiation.

[7]More details regarding the relation to differential geometry can be found under https://juliadiff.org/ChainRulesCore.jl/stable/maths/propagators.html.

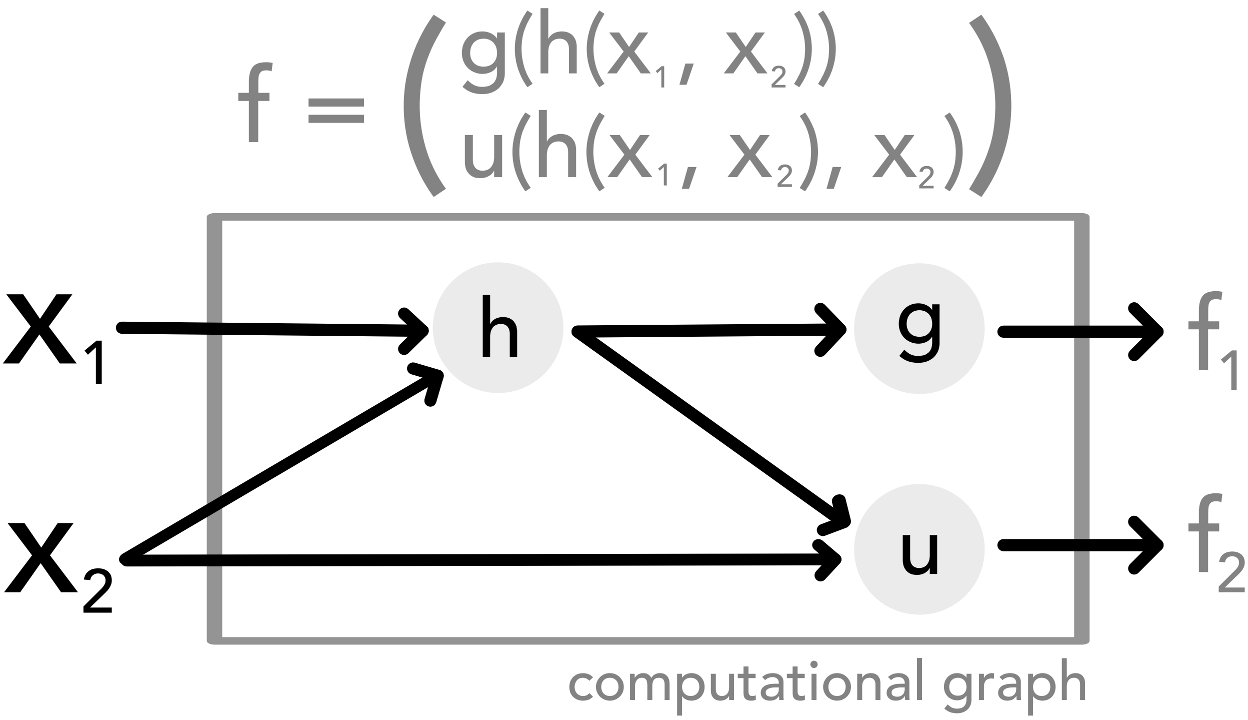


Figure 21: Illustrated is the computational graph of a function $\underline{f} : \mathbb{R}^2 \to \mathbb{R}^2$ (given in Eq. 239) which is composed of functions $g : \mathbb{R} \to \mathbb{R}, h : \mathbb{R}^2 \to \mathbb{R}, u : \mathbb{R}^2 \to \mathbb{R}$ depicted as nodes in the computational graph. Input relations are depicted as arrows.

Calculating the JVP through the computational graph is illustrated in Fig. 22.

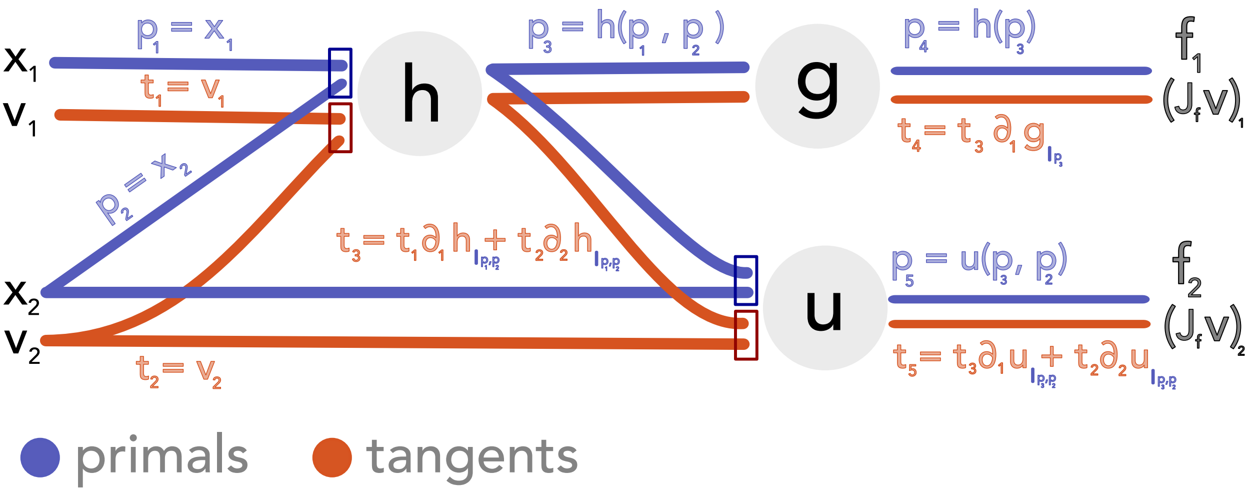


Figure 22: Illustrated is the forward differentiation through the function $\underline{f}$ (given in Eq. 239). The computational graph is evaluated from left to right, the tangent expressions correspond to the terms of the Jacobian-vector product as given in Eq. 240.

A.6.2 Reverse-mode automatic differentiation

Let us reconsider our simulation $\underline{f} = \underline{g}_m \circ \cdots \circ \underline{g}_2 \circ \underline{g}_1$ from before but we apply a scalar function $L : \mathbb{R}^N \to \mathbb{R}$ to the simulation output $\underline{f}(\underline{x})$, e.g. a loss $L(\underline{f}(\underline{x})) = ||\underline{f}(\underline{x}) - \underline{y}_{exp}||^2$

to an experimental measurement $\underline{y}_{exp} \in \mathbb{R}^N$.

To optimize L, we are interested in the gradient $\underline{\nabla}_{\underline{x}} L$, which we could calculate in N forward differentiations (for each base vector $\hat{\underline{e}}_i, i = 1, ..., N$).

Let us write out this gradient

$$\underline{\nabla}_{\underline{x}} L(\underline{f}(\underline{x})) = \underline{\underline{J}}_{\underline{f}}^T \underline{\nabla}_{\underline{f}} L = \underline{\underline{J}}_{\underline{g}_1}^T(\underline{x}) \underline{\underline{J}}_{\underline{g}_2}^T(\underline{g}_1(\underline{x})) \cdots \underline{\underline{J}}_{\underline{g}_m}^T(\underline{g}_{m-1}(\cdots(g_1(\underline{x})))) \underline{\nabla}_{\underline{f}} L \tag{241}$$

We see that calculating the gradient amounts to a Jacobian transpose vector product (J'VP). Similar to the forward-mode automatic differentiation rather than calculating the Jacobian matrix explicitly by matrix multiplication, we perform a chain of matrix vector products, here J'VPs. Note that the transpose has permuted the order of the Jacobians - we now move backwards through our computational graph.

To evaluate the Jacobians, we need the evaluations $\underline{g}_1, ...\underline{g}_{m-1}$ obtained in a *forward pass* through the computational graph. Therefore, **reverse-mode autodiff has memory complexity scaling linearly with the number of algorithm iterations**, which can become a bottleneck.

We can calculate the gradient of a scalar function in a single backward pass through the computational graph (which must be preceded by a forward pass).

If we capture the dependency structure of a composition of functions as a computational graph, with functions as nodes and dependencies as arrows, calculating the J'VP amounts to

- in a forward pass, evaluate all nodes, possibly precalculate the Jacobians of the nodes with respect to their inputs
- starting from the output space calculate the J'VP backwards through the computational graph

Backward differentiation through a computational graph like this is commonly called reverse-mode automatic differentiation, backpropagation, or, again drawing from differential geometry terminology, pullback (of a differential form).

A.6.2.1 Example of backward differentiation through a computational graph

We consider the same example as before (Eq. 239). The J'VP reads

$$\underline{\underline{J}}_{\underline{f}}^T \underline{s} = \begin{pmatrix} (\partial_1 h)\,(\partial_1 g) & (\partial_2 h)\,(\partial_1 g) \\ (\partial_1 h)\,(\partial_1 u) & (\partial_2 u) + (\partial_2 h)\,(\partial_1 u) \end{pmatrix}^T \underline{s} = \begin{pmatrix} (\partial_1 h)\,((\partial_1 g)s_1 + (\partial_1 u)s_2) \\ (\partial_2 h)\,((\partial_1 g)s_1 + (\partial_1 u)s_2) + (\partial_2 u)s_2 \end{pmatrix} \tag{242}$$

which is illustrated on the computational graph in Fig. 23. Often, one considers the transpose of the J'VP $(\underline{\underline{J}}_{\underline{f}}^T \underline{s})^T = \underline{s}^T \underline{\underline{J}}_{\underline{f}}$ and calls it Vector-Jacobian Product (VJP).

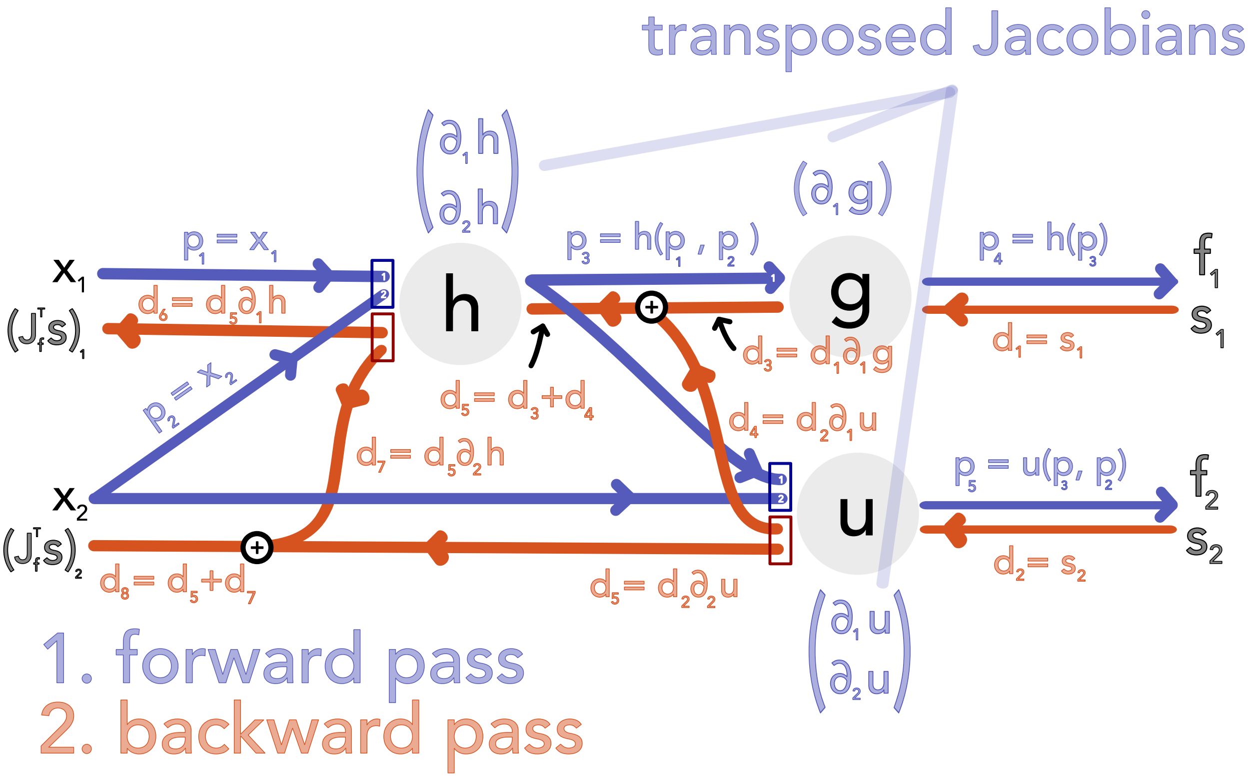


Figure 23: Backward differentiation illustrated for Eq. 239, the result from the backward pass corresponds to the Jacobian transpose vector product in Eq. 242.

A.6.2.2 Checkpointing for memory efficiency

Consider we have created a computational graph of our simulation with primitive operations as nodes in the graph. To calculate the J'VPs in the backward pass, the *linearization points*, e.g. all inputs to (elementwise nonlinear) primitive operations, must be known - this saves space compared to the naive approach of storing the Jacobians of the primitive operations.

It can, however, be too costly memorywise to store or too costly computationally (with regards to the computational cost of memory access) to retrieve all these linearization points.

Alternatively, we might only store certain *checkpoints* and recompute intermediate linearization points, e.g. for a larger function we might only store its inputs and calculate the linearization points of its primitive operations by reevaluating the function in the backward pass as necessary.

A note on optimal checkpointing: Let us consider a simulation of $l = 6$ timesteps with states $x_0 \to x_1 \to \cdots \to x_6$ and assume we are able to store $c = 2$ checkpoints at a time. We might reasonably start with x_0 and x_3 checkpointed, but in the backward pass might want to change which state is checkpointed for the optimal checkpointing strategy. This is illustrated in Fig. 24, an adaptation of Fig. 2.1 in Stumm and Walther, 2010. But how do we choose initial checkpoints when the number of simulation steps is unknown a priori, i.e. in adaptive time-stepping with a while-loop? For this there exist optimal *online* checkpointing strategies, e.g. Algorithm 1 from Stumm and Walther, 2010.[a] The algorithmic details are beyond the scope of this thesis, but in Fig. 25 adapted from Fig. 2.2 in Stumm and Walther, 2010 we provide intuition on how one adapts which states are checkpointed while moving forward in the loop.

[a] An implementation in `JAX` by Patrick Kidger is available under `https://github.com/patrick-kidger/equinox/blob/main/equinox/internal/_loop/checkpointed.py`

More information on checkpointing in the context of `JAX` can be found under `https://docs.jax.dev/en/latest/_autosummary/jax.checkpoint.html`.

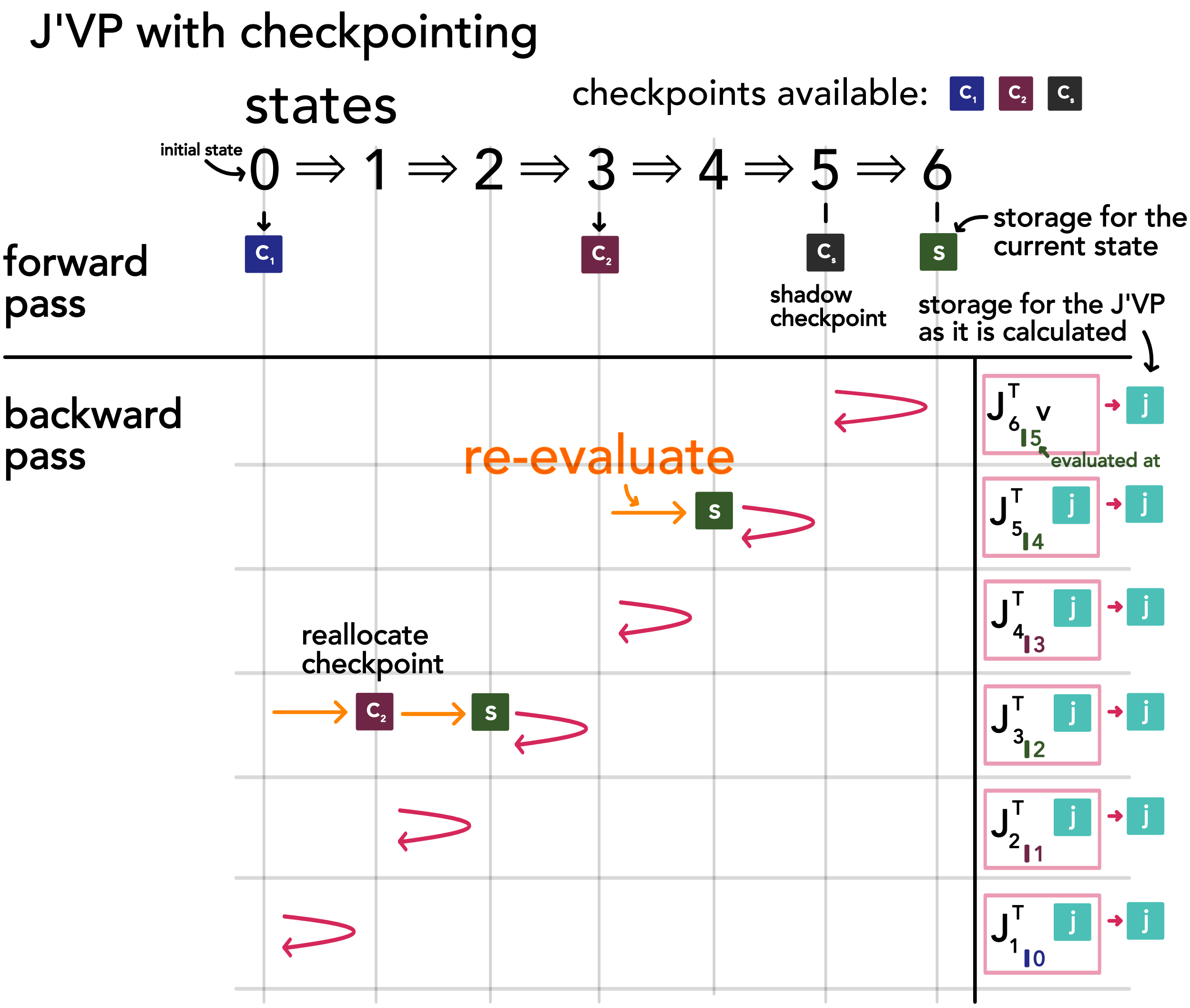


Figure 24: Illustrated is the calculation of a Jacobian transpose vector product (J'VP) using checkpointing through a loop with six iterations and with two checkpoints available. The illustration is based on Fig. 2.1 in Stumm and Walther, 2010. To calculate the derivative of a state in the sequence with respect to its inputs (i.e. the previous state), these inputs must be recalculated from checkpoints. We start out with state 0 and 3 checkpointed and state 5 stored as a *shadow checkpoint* (we assume that in the iteration we store the current and last state). With state 5 known we can differentiate state 6. In the next step we have to reconstruct state 4 from the checkpoint at state 3 to differentiate state 5 and so forth. Checkpoints can be reallocated to reduce the number of necessary recomputations.

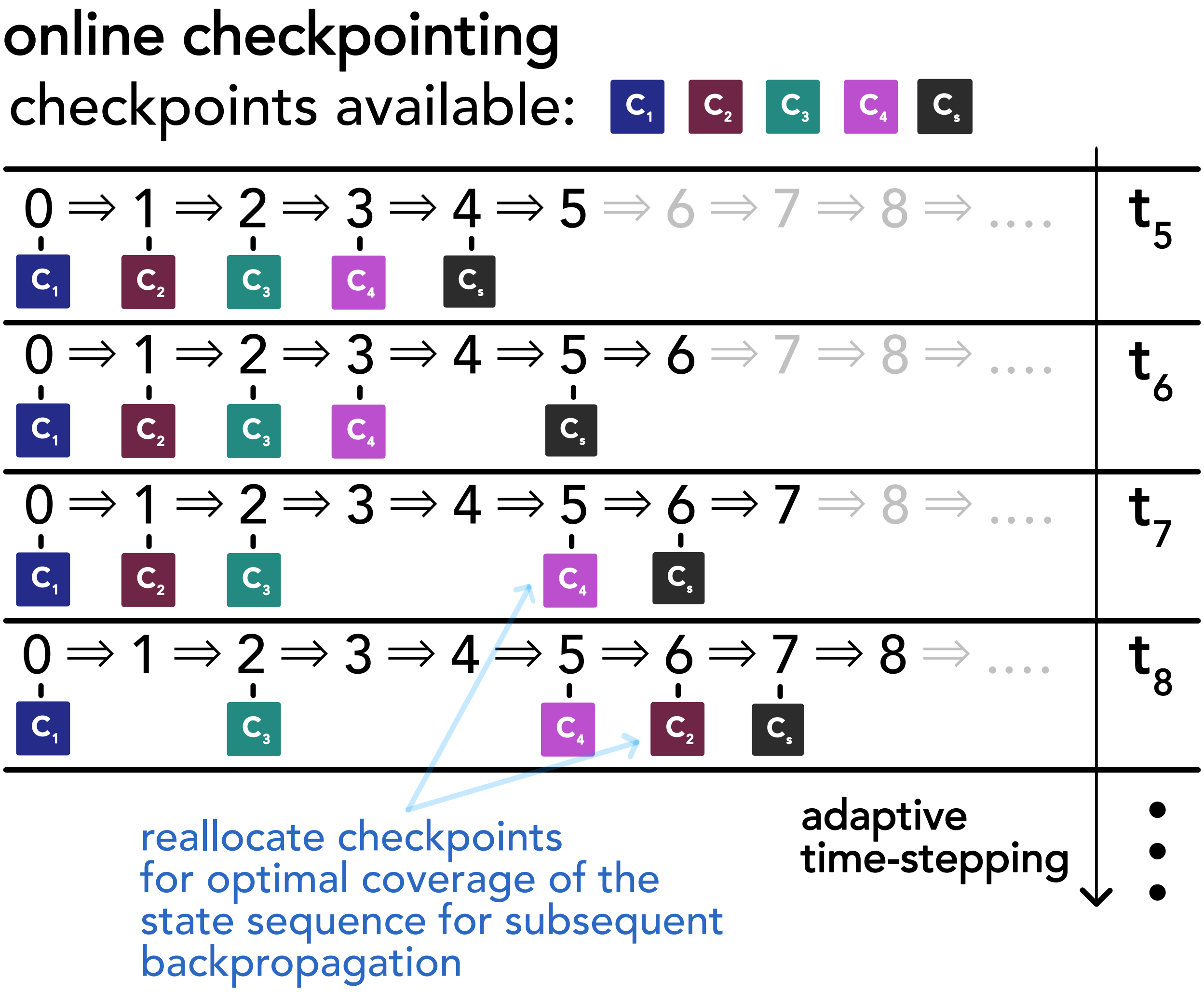


Figure 25: Illustrated is *online checkpointing* with four checkpoints available, i.e. in a loop with unknown number of iterations how do we reallocate the checkpoints as we go for optimal coverage of the state sequence for subsequent backpropagation. This illustration is based on Fig. 2.2 in Stumm and Walther, 2010. A shadow checkpoint c_s always trails behind the current state. The figure is only meant to give a rough idea, for the details see Algorithm 1 in Stumm and Walther, 2010.

A.6.3 Example of custom differentiation rules: Implicit differentiation through a fixed-point iteration

There might be parts of our simulator for which we know differentiation rules so that we can avoid the memory and computational cost of differentiating through the primitive operations making up these parts.

To equip a function with custom rules for forward and backward differentiation, we need to define a custom JVP and J'VP, here its transpose the VJP, respectively.

As an example, consider our simulation contains a fixed point iteration, approximating $\underline{x}^\star(\underline{\theta})$ implicitly defined by

$$\underline{x}^\star(\underline{\theta}) = \underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta}) \tag{243}$$

by iterating $\underline{x}_{t+1} = \underline{f}(\underline{x}_t, \underline{\theta})$ (assuming there exists one attracting fixed-point, which we usually show using Banach's fixed-point theorem).

By the chain rule (assuming $\underline{x}^\star(\underline{\theta}) = \underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta})$ holds in a neighborhood around $\underline{\theta}_{eval}$ around which we want to differentiate)[8]

$$\underline{\underline{J}} = \partial_{\underline{\theta}}\underline{x}^\star(\underline{\theta}) = \underbrace{\partial_{\underline{\theta}}\underline{x}^\star(\underline{\theta})}_{=\underline{\underline{J}}}\underbrace{\partial_1\underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta})}_{=:\underline{\underline{A}}} + \underbrace{\partial_2\underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta})}_{=:\underline{\underline{B}}} \tag{244}$$

(where we have implicitly applied the implicit function theorem, see Blondel et al., 2022 for more details on this).

We can rewrite Eq. 244 to give an explicit expression

$$\underline{\underline{J}} = (\underline{\underline{1}} - \underline{\underline{A}})^{-1}\underline{\underline{B}} \tag{245}$$

The JVP can therefore be calculated by solving the linear system

$$(\underline{\underline{1}} - \underline{\underline{A}})(\underline{\underline{J}}\underline{v}) = \underline{\underline{B}}\underline{v} \tag{246}$$

and the VJP by

[8] The derivation follows `https://docs.jax.dev/en/latest/advanced-autodiff.html#example-implicit-function-differentiation-of-iterative-implementations`.

$$(\underline{\underline{1}} - \underline{\underline{A}})^T \underline{u} = \underline{v} \quad \rightarrow \quad \underline{v}^T \underline{\underline{J}} = \underbrace{\underline{v}^T (\underline{\underline{1}} - \underline{\underline{A}})^{-1}}_{=\underline{u}^T} \underline{\underline{B}} \tag{247}$$

where solving the linear systems might be done using GMRES (Saad and Schultz, 1986). While there are sophisticated techniques for inverting a matrix, the inverse can potentially be very dense even if the matrix itself is not, which would interfere with smart memory optimizations (Rackauckas et al., 2022).

For the VJP, we might also solve

$$(\underline{\underline{1}} - \underline{\underline{A}})^T \underline{u} = \underline{v} \quad \Longleftrightarrow \quad \underline{u}^T = \underline{v}^T + \underline{u}^T \underline{\underline{A}} \tag{248}$$

using a fixed-point iteration.

As a simple example, based on Heron's root finding algorithm, we construct

$$\underline{f}(\underline{x}, \underline{\theta}) = \alpha \underline{x} + (1 - \alpha) \begin{pmatrix} \frac{1}{x_1} \exp\left(\theta_1^2 + 2\theta_2\right) \\ \frac{1}{x_2} \exp\left(2\theta_1 + \theta_2^2\right) \end{pmatrix} \tag{249}$$

with analytical solution

$$\underline{x}^\star(\underline{\theta}) = \begin{pmatrix} \exp\left(\frac{1}{2}\theta_1^2 + \theta_2\right) \\ \exp\left(\theta_1 + \frac{1}{2}\theta_2^2\right) \end{pmatrix} \tag{250}$$

where we use $\alpha = 0.8$ for slower convergence.[9]

[9] The fixed point method is based on

$$x = \alpha x + (1 - \alpha)\frac{S}{x} \iff x = \sqrt{S} \tag{251}$$

with $S > 0$, known as Heron's method for $\alpha = 0.5$. It converges with

$$e_{n+1} = x_{n+1} - \sqrt{S} \underbrace{=}_{x_n = \sqrt{S} + e_n} \alpha(\sqrt{S} + e_n) + (1 - \alpha)\frac{\sqrt{S}}{1 + \frac{e_n}{\sqrt{S}}} - \sqrt{S} \tag{252}$$

where approximating $\frac{1}{1+\frac{e_n}{\sqrt{S}}}$ to 2nd order around $e_n = 0$ to $1 - \frac{e_n}{\sqrt{S}} + \frac{e_n^2}{S}$ yields

$$e_{n+1} \approx (2\alpha - 1)e_n + (1 - \alpha)\frac{e_n^2}{\sqrt{S}} \tag{253}$$

so only for $\alpha = 0.5$ do we get quadratic convergence, for $\alpha \neq 0.5$ the linear term dominates and the error decreases by a factor $2\alpha - 1$ per iteration.

For $v = \begin{pmatrix}1\\1\end{pmatrix}$ and $\underline{\theta}_{eval} = \begin{pmatrix}1\\1\end{pmatrix}$ we compare the accuracy of the JVP and VJP compared to the ground thruth between a direct differentiation through the fixed-point iteration and the implicit differentiation for different numbers of iterations in Fig. 26 with the source code for the custom JVP given in Code-Snippet 3, and the custom VJP in Code-Snippet 4. The error of the JVP and VJP from direct automatic differentiation through the fixed-point iteration decreases with the same scaling as the fixed-point error itself, in the example around two orders of magnitude per 10 iterations. This is expected, the automatic differentiation calculates the accurate derivatives of our approximation of the fixed point to the parameters. Interestingly, the implicit differentiation, derived for the *true* fixed-points, has better error scaling behavior (roughly 4 orders of magnitude per 10 iterations). By design the reverse-mode automatic differentiation has a memory overhead compared to the implicit differentiation (scaling with the number of iterations) while the linear solve (or fixed-point iteration if one follows Eq. 248) in the implicit differentiation is a computational overhead compared to the backward mode automatic differentiation (not scaling with the iterations but the number of parameters). We leave more in-depth profiling to future work.

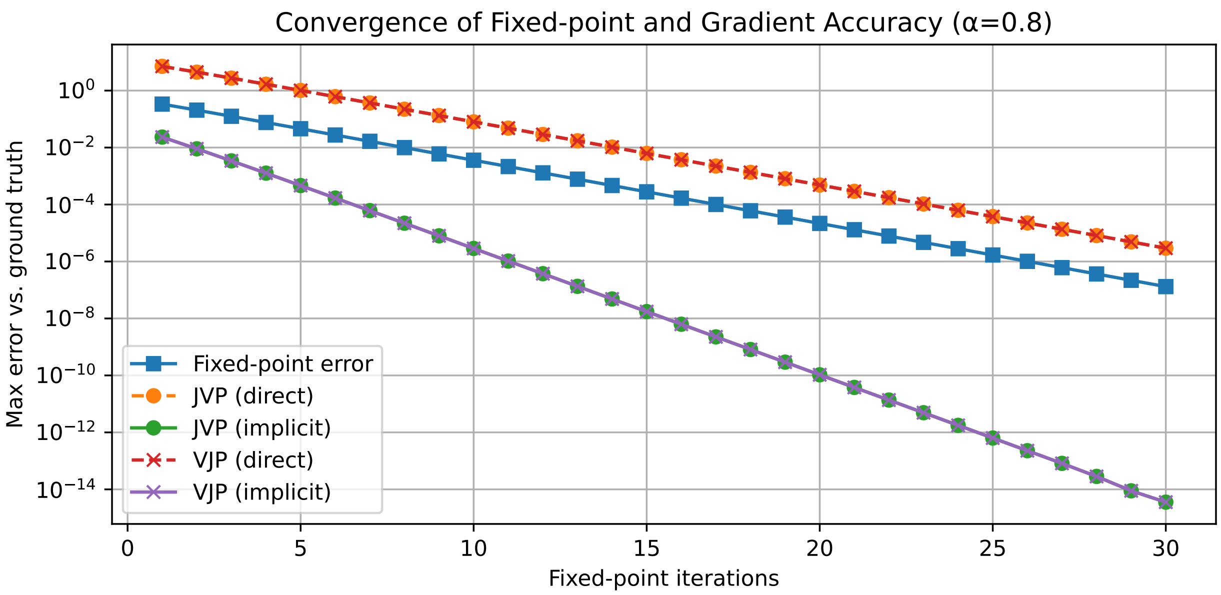


Figure 26: Our aim is to differentiate the fixed-point of Eq. 249 with respect to the parameters θ_1, θ_2 or rather calculate JVPs and VJPs. We compare direct automatic differentiation through the fixed-point iteration and implicit differentiation (evaluated at the current estimate of the fixed-point) to the ground truth (based on the analytical expression for the fixed-point, Eq. 250).

```python
@partial(custom_jvp, nondiff_argnums=(0, 3))
def fixed_point_jvp(f, x_guess, theta, num_iters=100):
    def body(_, x): return f(x, theta)
    return fori_loop(0, num_iters, body, x_guess)

@fixed_point_jvp.defjvp
def fixed_point_jvp_rule(f, num_iters, primals, tangents):
    x_guess, theta = primals
    dx_guess, dtheta = tangents
    x_star = fixed_point_jvp(f, x_guess, theta, num_iters)
    A = jacobian(f, argnums=0)(x_star, theta)
    B = jacobian(f, argnums=1)(x_star, theta)
    dx = jnp.linalg.solve(jnp.eye(x_star.shape[0]) - A, B @ dtheta)
    # primal output, tangent output
    return x_star, dx
```

Code-Snippet 3: Implementation of a custom Jacobian vector product for the results of fixed-point iterations in `JAX` following Eq. 246

```python
@partial(custom_vjp, nondiff_argnums=(0, 3))
def fixed_point_vjp(f, x_guess, theta, num_iters=100):
    def body(_, x): return f(x, theta)
    return fori_loop(0, num_iters, body, x_guess)

def fixed_point_vjp_fwd(f, x_guess, theta, num_iters):
    x_star = fixed_point_vjp(f, x_guess, theta, num_iters)
    # primal output, stored variables for backward pass
    return x_star, (x_star, theta)

def fixed_point_vjp_bwd(f, num_iters, res, v):
    x_star, theta = res
    A = jacobian(f, argnums=0)(x_star, theta)
    B = jacobian(f, argnums=1)(x_star, theta)
    u = jnp.linalg.solve((jnp.eye(x_star.shape[0]) - A).T, v)
    return jnp.zeros_like(x_star), u @ B

fixed_point_vjp.defvjp(fixed_point_vjp_fwd, fixed_point_vjp_bwd)
```

Code-Snippet 4: Implementation of a custom vector Jacobian product for the results of fixed-point iterations in `JAX` following Eq. 247

A.7 Introducing the Continuous Adjoint Method for ODEs

We will now introduce the continuous adjoint method for differentiating the solution of ODEs, or rather a scalar function L of the solution like a loss, and compare it to direct differentiation through the numerical solver in Sec. A.8. Our treatment here follows Kidger, 2021.

Consider the initial value problem (IVP)

$$\underline{y}(0) = \underline{y}_0, \quad \frac{d\underline{y}}{dt} = \underline{f}_{\underline{\theta}}(t, \underline{y}(t)) \tag{254}$$

with

$$\underline{y}(0) \in \mathbb{R}^d, \quad \underline{\theta} \in \mathbb{R}^m, \quad \underline{f}_{\underline{\theta}} : [0, T] \times \mathbb{R}^d \to \mathbb{R}^d \tag{255}$$

where $\underline{y} : [0, T] \to \mathbb{R}^d$ is the solution to the IVP and $\underline{f}_{\underline{\theta}}$ is continuous in t and uniformly Lipschitz and continuously differentiable in $\underline{y}$.

Our aim is to calculate $\frac{dL(\underline{y}(T))}{d\underline{\theta}}$, where $L : \mathbb{R}^d \to \mathbb{R}$ and T is the time point of interest.

This can be achieved by solving the adjoint system (here of the *continuous* original problem, not its discretization)

$$\begin{aligned} \underline{a}_{\underline{y}}(T) = \frac{dL}{d\underline{y}(T)}, \quad & \frac{d\underline{a}_{\underline{y}}}{dt}(t) = -\underline{a}_{\underline{y}}(t)^T \underbrace{\frac{\partial \underline{f}_{\underline{\theta}}}{\partial \underline{y}}(t, \underline{y}(t))}_{\in \mathbb{R}^{d \times d}} \\ \underline{a}_{\underline{\theta}}(T) = 0, \quad & \frac{d\underline{a}_{\underline{\theta}}}{dt}(t) = -\underline{a}_{\underline{y}}(t)^T \underbrace{\frac{\partial \underline{f}_{\underline{\theta}}}{\partial \underline{\theta}}(t, \underline{y}(t))}_{\in \mathbb{R}^{d \times m}} \end{aligned} \tag{256}$$

with $\underline{a}_{\underline{y}} : [0, T] \to \mathbb{R}^d, \underline{a}_{\underline{\theta}} : [0, T] \to \mathbb{R}^m$ from $t = T$ to $t = 0$ backwards in time where

$$\boxed{\frac{dL(\underline{y}(T))}{d\underline{\theta}} = \underline{a}_{\underline{\theta}}(0)} \tag{257}$$

Note that to evaluate the Jacobians in Eq. 256 we need to know $\underline{y}(t)$ but to have a memory advantage over direct autodiff through the numerical solver, we cannot save the whole forward trajectory in the *forward pass*. Therefore, as we integrate $\underline{a}_{\underline{y}}$ and $\underline{a}_{\underline{\theta}}$ backwards in time starting from $t = T$, we obtain $\underline{y}(t)$ by integrating back from $y(\underline{T})$ (obtained in the forward pass). **Note, however, that the numerical integration backwards in time will not match the forward trajectory in general, except when the numerical solver itself is reversed and floating point errors are insignificant.** Also, the backward problem might generally be harder than the forward problem.

For a proof of the continuous adjoint method, we refer to Kidger, 2021, Sec. 5.1.2.1.

A.8 Application Example: AD through the solver vs. continuous adjoint method on exponential decay

Consider the ODE for exponential decay

$$\frac{dy}{dt}(t) = \lambda y(t), \quad \lambda < 0, \quad y(0) = y_0, \tag{258}$$

with analytical solution $y(t) = y_0 \exp(\lambda t)$ and analytical derivative $\partial_\lambda y = t y_0 \exp(\lambda t)$.

The adjoint system reads

$$\begin{aligned} a_y(T) &= 1, & \frac{da_y}{dt}(t) &= -a_y(t)\lambda \\ a_\lambda(T) &= 0, & \frac{da_\lambda}{dt}(t) &= -a_y(t)y(t) \end{aligned} \tag{259}$$

We will compare the following approaches for calculating $\partial_\lambda y$:

- direct automatic differentiation through the implicit Euler method with $y_{n+1} = y_n + \Delta t \lambda y_{n+1} \rightarrow y_{n+1} = \frac{y_n}{1-\Delta t \lambda}$
- direct automatic differentiation through dopri5[10]
- the continuous adjoint method with implicit Euler for the forward- and backsolve
- a discrete adjoint system of the implicit Euler method which we derive based on the

[10] All tests involving dopri5 were carried out using `diffrax`, `https://docs.kidger.site/diffrax/`.

backpropagation idea as illustrated in Fig. 27:

$$
\begin{aligned}
g(y, \lambda) = \frac{y}{1 - \Delta t \lambda} &\rightarrow y_N = \underbrace{(g \circ g \circ \cdots \circ g)}_{\times N}(y_0) \\
\rightarrow \partial_\lambda y_N = \sum_{n=0}^{N-1} \partial_\lambda g\big|_{y_n} a_{y_{n+1}} &\text{ with } a_{y_n} = \partial_y g\big|_{y_n} a_{y_{n+1}}, a_{y_N} = 1 \\
\partial_\lambda g = \Delta t \frac{y}{(1 - \Delta t \lambda)^2}, &\quad \partial_y g = \frac{1}{1 - \Delta t \lambda}
\end{aligned}
\tag{260}
$$

Let us introduce

$$
a_{\lambda_n} = a_{\lambda_{n+1}} + \partial_\lambda g\big|_{y_n} a_{y_{n+1}}, \quad a_{\lambda_N} = 0 \tag{261}
$$

such that $\partial_\lambda y_N = a_{\lambda_0}$. By manually reverting the implicit Euler step for this specific problem, we find the discrete adjoint system

$$
\begin{aligned}
y_n = y_{n+1} \cdot (1 - \Delta t \lambda), &\quad y_N \text{ from forward pass} \\
a_{y_n} = \frac{1}{1 - \Delta t \lambda} a_{y_{n+1}}&, a_{y_N} = 1 \\
a_{\lambda_n} = a_{\lambda_{n+1}} + \Delta t \frac{y_n}{(1 - \Delta t \lambda)^2} a_{y_{n+1}} &= a_{\lambda_{n+1}} + \Delta t a_{y_n} y_{n+1}, a_{\lambda_N} = 0
\end{aligned}
\tag{262}
$$

which we can recognize to be a specific discretization of the continuous adjoint equations, connecting the continuous adjoint method, backpropagation and reversible solvers.

- the continuous adjoint method with dopri5 for the forward- and backsolve

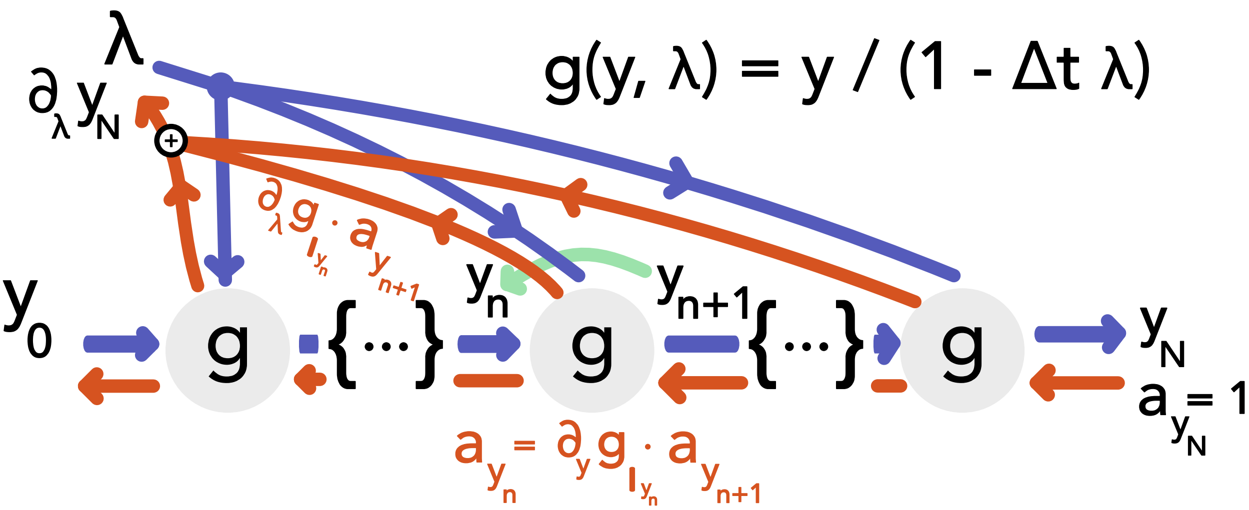


Figure 27: Illustration of backpropagation through the computational graph of an implicit Euler iteration to solve $\frac{dy}{dt}(t) = \lambda y(t), \lambda < 0, y(0) = y_0$, corresponding to Eq. 262.

An exemplary comparison is shown in Fig. 28 for $\Delta t = 0.3$ and $\lambda = -0.5$. The core observations are:

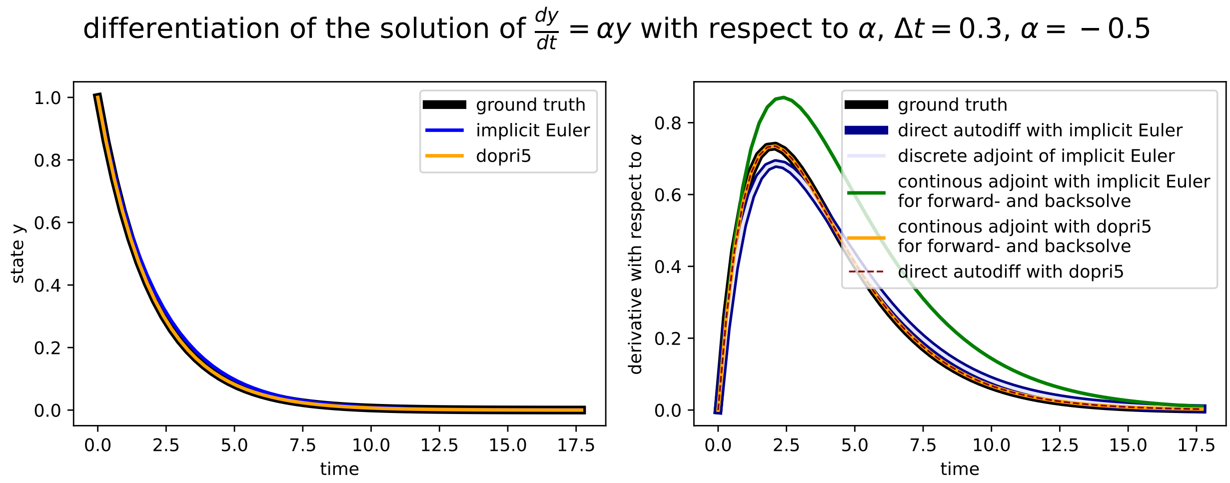


Figure 28: Comparison of different techniques for taking the derivative of the solution to an ODE with respect to its parameters for a simple exponential decay problem.

- the discrete adjoint and autodiff results match as expected
- the continuous adjoint method on implicit Euler, where the backward discretization is not an appropriate adjoint system to the forward discretization, yields the worst results
- dopri5 is sufficiently accurate such that both the direct autodiff and continuous adjoint nicely match the ground truth

A.9 Gauss-Legendre Quadrature

It is intuitive that an integral can be approximated by the mean of evaluation points. However, it turns out, that one can exactly integrate polynomials of degree $2n-1$ with n smart evaluation points and weights. One such method is called Gauss-Legendre quadrature, where the evaluation points are the roots of *Legendre polynomials.*

A.9.1 Legendre Polynomials

The Legendre Polynomials are an orthogonal system of polynomials, $P_n \in \Pi_n$, on the interval $[-1, 1]$ with

$$\int_{-1}^{1} P_m(x) P_n(x)\, dx = \frac{2}{2n+1} \delta_{nm} \tag{263}$$

and the additional condition $P_n(1) = 1$.

The first Legendre polynomials are

$$P_0(\xi) = 1, \quad P_1(\xi) = \xi, \quad P_2(\xi) = \frac{1}{2}\left(3\xi^2 - 1\right) \tag{264}$$

Note: The span of $P_0(x), P_1(x), \ldots, P_n(x)$ is the same as of $1, x, x^2, \ldots, x^n$. So

$$\forall \text{polynomial } P \text{ of degree } < n : \int_{-1}^{1} P(x) P_n(x)\, dx = 0 \tag{265}$$

A.9.2 Gauss-Legendre Quadrature

Gauss-Legendre Quadrature given by

$$\begin{gathered} \int_{-1}^{1} f(\xi) d\xi \approx \sum_{q=0}^{n} f\left(\xi_q^{1D}\right) w_q^{1D} \\ \text{roots } \xi_q^{1D} \text{ of } P_{n+1}(\xi), \text{ weights } w_q^{1D} = \frac{2}{\left(1 - \left(\xi_q^{1D}\right)^2\right)\left(P'_{n+1}\left(\xi_q^{1D}\right)\right)^2} \end{gathered} \tag{266}$$

for $f : [-1, 1] \to \mathbb{R}$ is exact for polynomials of degree $2n + 1$.

A.9.3 Proof: Gauss-Legendre quadrature with $n+1$ evaluation points is exact for polynomials of degree $2n+1$

Let $f = P(x)$ have degree $\leq 2n+1$. Then we can write $P(x) = Q(x)P_{n+1}(x) + R(x)$ with $Q(x), R(x)$ polynomials of degree $\leq n$ ($P_{n+1}(x)$ is the $(n+1)$-th Legendre polynomial). Then

$$\begin{aligned}
\int_{-1}^{1} P(x)dx &= \underbrace{\int_{-1}^{1} Q(x)P_{n+1}(x)dx}_{=0 \text{ as } Q \text{ can be written in base } \{P_0,\ldots,P_n\}\perp P_{n+1}} + \int_{-1}^{1} R(x)dx \\
&= \int_{-1}^{1} Rdx \\
&= \underbrace{\sum_{q=0}^{n} R\left(\xi_q^{1D}\right) w_q^{1D}}_{\text{exact as } R \text{ is a polynomial of degree } \leq n} \\
&= \sum_{q=0}^{n} \underbrace{\left(\underbrace{Q(\xi_q^{1D}) \cdot P_{n+1}(\xi_q^{1D})}_{=0,\text{as } \xi_q^{1D} \text{ roots of } P_{n+1}} + R\left(\xi_q^{1D}\right) \right)}_{=P(\xi_q^{1D})} w_q^{1D} \\
&= \sum_{q=0}^{n} P(\xi_q^{1D}) w_q^{1D}
\end{aligned} \tag{267}$$

A.10 Monte Carlo Integration

Classical numerical methods for integration (e.g. Simpson's rule) become infeasible for high-dimensional integrals, essentially, as the errors scale with the number of function-evaluation-points per dimension (so in high dimensions, we need to perform lots of function evaluations).

A.10.1 Intuition for Monte Carlo Integration

Consider we want to calculate the integral

$$I := \int_V g(\underline{x}) d^d \underline{x} \tag{268}$$

where V is a d-dimensional volume and g is a function $g : V \to \mathbb{R}$. Then it makes intuitive sense to approximate this integral by the mean of the function g on the domain of interest, as found as the mean of g evaluated at N random points $\underline{x}_i$ in V, multiplied by the volume of V (also denoted by V)

$$\hat{I}_N := \frac{V}{N}\sum_{i=1}^{N} g(\underline{x}_i) \xrightarrow[N\to\infty]{a.s.} I \tag{269}$$

Monte Carlo Integration: Let V for simplicity be the hypercube $[0,1]^d$ (by change of variables, the domain of interest can be mapped to this hypercube). Then Monte Carlo integration to approximate $I = \int_{[0,1]^d} g(\underline{x})d^d\underline{x}$, $\underline{x} \in \mathbb{R}^d$ is given by

- Generate N random vectors $\underline{x}$ where each component is drawn uniformly from $[0,1]$ (so we need $N \times d$ random numbers)
- Evaluate
$$\hat{I}_N = V\frac{1}{N}\sum_{i=1}^{N} g(\underline{x}_i), \quad \text{here } V = 1 \tag{270}$$

Scaling of the error: The error of the Monte Carlo integrator scales with $1/\sqrt{N}$, independent of the number of dimensions of the integration d.

A.10.2 Connection to the Monte Carlo Estimator

We can also obtain the result from above from the Monte Carlo Estimator (eq. **??**) by using the uniform distribution

$$f(\underline{x}) = \begin{cases} \frac{1}{V} & \text{if } \underline{x} \in V \\ 0 & \text{else} \end{cases} \tag{271}$$

and rewriting the integral as

$$I = \int_V g(\underline{x})d^d\underline{x} = V\int_V g(\underline{x})\cdot\frac{1}{V}d^d\underline{x} = V\int_V g(\underline{x})\cdot f(\underline{x})d^d\underline{x} \tag{272}$$

A.10.3 Intuition from calculating the area of a shape

Consider an irregular shape with area A_{shape} within a rectangle with area A_{rect}. The probability that a point drawn uniformly from the rectangle lies within the shape is

$$p_{\text{shape}} = \frac{A_{\text{shape}}}{A_{\text{rect}}} \tag{273}$$

so we can approximate

$$A_{\text{shape}} \approxeq A_{\text{rect}}\frac{\#\text{ hits in shape}}{\#\text{ hits in rectangle}} \tag{274}$$

which is the same as the Monte Carlo Integration introduced, where $g(\underline{x}) = 1_{\text{shape}}(\underline{x})$ and $V = A_{\text{rect}}$.

A.10.4 Error in Monte Carlo Integration - scaling with $\frac{1}{\sqrt{N}}$

Based on the connection to the Monte Carlo Estimator, the variance of our estimator for the integral is given by

$$\text{Var}[\hat{I}_N] = \frac{V^2}{N}\,\text{Var}[g(x)], \quad \text{estimate}\,\text{Var}[g(x)] \text{ by } S^2_{g;N} := \frac{1}{N-1}\sum_{i=1}^{N}\left(g(x_i) - \frac{\hat{I}_N}{V}\right)^2 \tag{275}$$

So the standard error of our Monte Carlo estimator

$$\hat{I}_N = \frac{V}{N}\sum_{i=1}^{N} g(\underline{x}_i) \tag{276}$$

that is if we calculate $\hat{I}_N$ multiple times with different random samples, the standard deviation of these different estimates is

$$\boxed{\sigma_{\hat{I}_N} = V\sqrt{\frac{\sigma_g^2}{N}} \propto \frac{1}{\sqrt{N}}} \tag{277}$$

which is illustrated in Fig. 29. This follows directly from the basic properties of the variance

$$\text{for } X, Y \text{ independent} \quad \text{Var}[X+Y] = \text{Var}[X] + \text{Var}[Y], \quad \text{Var}[aX] = a^2\,\text{Var}[X] \tag{278}$$

with

$$\text{Var}[\hat{I}_N] = \text{Var}\left[\frac{V}{N}\sum_{i=1}^{N} g(\underline{x}_i)\right] = \frac{V^2}{N^2}\sum_{i=1}^{N}\text{Var}[g(\underline{x}_i)] \underset{\underline{x}_i \text{ i.i.d.}}{=} \frac{V^2}{N}\,\text{Var}[g(\underline{x})] \tag{279}$$

A.10.5 Distribution of $\hat{I}_N$ and derivation of the central limit theorem*

The Monte Carlo Estimator is

$$\hat{I}_N = \frac{V}{N}\sum_{i=1}^{N} g(\underline{x}_i) = \frac{1}{N}\sum_{i=1}^{N} y_i = \sum_{i=1}^{N} s_i, \quad y_i = V g(\underline{x}_i), \quad s_i = \frac{y_i}{N} \tag{280}$$

by basic properties of variance and mean

$$\begin{gathered}\langle \hat{I}_N \rangle = N\langle s \rangle = \langle y \rangle \\ \text{Var}[\hat{I}_N] = N\,\text{Var}[s] = \frac{1}{N}\,\text{Var}[y] = \frac{V^2}{N}\,\text{Var}[g(\underline{x})]\end{gathered} \tag{281}$$

Note: $p(y)$ is the distribution of y over V along the y-axis with

$$\langle y\rangle = \int yp(y)\,dy = \int_V g(\underline{x})\,d^d\underline{x}, \quad \int p(y)\,dy = 1 \tag{282}$$

$p(y)$ **might be any distribution.**

Applying the Central Limit Theorem (eq. **??**) to the Monte Carlo Estimator (assuming the moments of $p(y)$ exist and are finite), we obtain

$$\begin{aligned} P_N(\hat{I}_N) &= \frac{1}{\sqrt{2\pi\,\mathrm{Var}[\hat{I}_N]}} \exp\left(-\frac{(\hat{I}_N - \langle\hat{I}_N\rangle)^2}{2\,\mathrm{Var}[\hat{I}_N]}\right) \\ &= \frac{\sqrt{N}}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{N(\hat{I}_N - \langle y\rangle)^2}{2\sigma_y^2}\right) \end{aligned} \tag{283}$$

independent of the shape of $p(y)$ for N sufficiently large.

Derivation of the Central Limit Theorem by Fourier Transform The central limit theorem can be derived by considering P_N in Fourier space, where the central ingredient will simply be

$$\exp x = \lim_{N\to\infty}\left(1 + \frac{x}{N}\right)^N \tag{284}$$

which is also approximately true for large N.

Distribution of the sum of independent random variables: Consider **independent** variables X, Y with PDFs f_X and f_Y. The PDF of the sum $Z = X + Y$ is naturally the convolution

$$f_Z(z) = \int_{-\infty}^{\infty} \underbrace{f_X(x) f_Y(z-x)}_{x+(z-x)=z}\,dx \tag{285}$$

Another way to write this is

$$f_Z(z) = \int f_X(x) f_Y(y) \delta(z - (x+y))\,dx\,dy \tag{286}$$

which one can easily see is equivalent to the convolution and also makes intuitive sense - the δ-function ensures that the sum of x and y is z.

Analogously to the two-variable case, we write

$$P_N\left(\hat{I}_N\right) = \int \delta\left(\hat{I}_N - \sum_i \frac{y_i}{N}\right) p\left(y_1\right) p\left(y_2\right) \cdots p\left(y_N\right) \mathrm{d}y_1\, \mathrm{d}y_1 \cdots \mathrm{d}y_N \tag{287}$$

Based on the Fourier transform and expansion of $p(y)$

$$\begin{aligned}
\hat{p}(k) &= \int p(y) \exp\left(ik(y - \langle y \rangle)\right) \mathrm{d}y \\
&\underset{\text{Series Expansion}}{=} \int p(y) \left[1 + ik(y - \langle y \rangle) - \frac{k^2}{2}(y - \langle y \rangle)^2 + \cdots\right] \mathrm{d}y \\
&= 1 - \frac{k^2}{2}\sigma_y^2 + \cdots
\end{aligned} \tag{288}$$

we find the Fourier transform of P_N to be

$$\begin{aligned}
\hat{P}_N(k) &= \int P_N\left(\hat{I}_N\right) \exp\left(ik\left(I_N - \langle I_N \rangle\right)\right) \mathrm{d}I_N \\
&\underset{\langle \hat{I}_N \rangle = \langle y \rangle}{=} \int p\left(y_1\right) \cdots p\left(y_N\right) \exp\left(i\frac{k}{N}\left(y_1 - \langle y_1 \rangle + y_2 - \langle y_2 \rangle + \ldots + y_N - \langle y_N \rangle\right)\right) \mathrm{d}y_1 \cdots \mathrm{d}y_N \\
&= \left[\hat{p}\left(\frac{k}{N}\right)\right]^N \\
&\underset{\text{series expansion of } \hat{p}}{=} \left(1 - \frac{k^2\sigma_y^2}{2N^2}\right)^N \\
&\underset{N \text{ large}}{\approxeq} \exp\left(-\frac{k^2\sigma_y^2}{2N}\right)
\end{aligned} \tag{289}$$

where the inverse Fourier transform of $\hat{P}_N(k)$

$$P_N\left(\hat{I}_N\right) = \frac{1}{2\pi} \int \exp\left(-ik\left(I_N - \langle I_N \rangle\right)\right) \hat{P}_N(k) \mathrm{d}k \tag{290}$$

yields the previously stated result for $P_N\left(\hat{I}_N\right)$[11].

A.10.6 Illustrative Example of Monte Carlo Integration

Consider the integral

$$I = \int_0^1 g(x) dx, \quad g(x) = \exp\left(-\frac{x^2}{2}\right) \tag{291}$$

Our Monte Carlo estimator for this integral is given by

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^{N} g(x_i), \quad x_i \sim \mathcal{U}(0, 1) \tag{292}$$

which is illustrated in Fig. 29. The higher the number of sampled points N the lower the variance of our estimator; the lower the variance of g along the y axis, the lower the variance

[11] Insert the result for $\hat{P}_N(k)$, complete the square and use the normalization of the normal distribution or the Gaussian integral $\int \exp\left(-\alpha x^2\right) \mathrm{d}x = \sqrt{\frac{\pi}{\alpha}}$.

of our estimator.

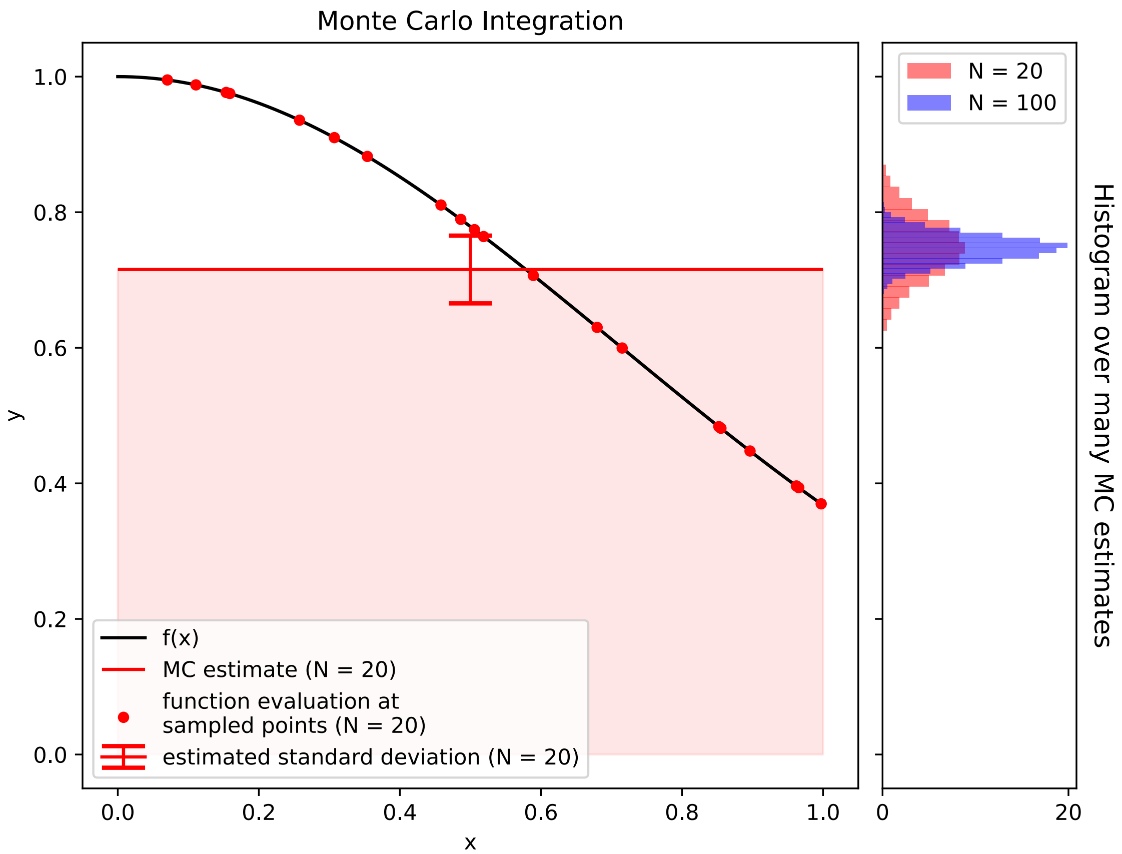


Figure 29: Illustration of Monte Carlo Integration for $I = \int_0^1 \exp\left(-\frac{x^2}{2}\right) dx$

A.10.7 Comparison to other techniques - when to use Monte Carlo integration?

Error of standard methods In a standard integration scheme like the midpoint rule or Simpsons rule[12] each dimension is divided into n regularly space points, so that we have $N = n^d$ points in total. The error scales with

$$\text{midpoint or trapezoidal rule} \propto \frac{1}{n^2}, \quad \text{Simpson's rule} \propto \frac{1}{n^4} = \frac{1}{N^{\frac{4}{d}}} \tag{293}$$

In these *regular* methods, adding more points to the integration (increasing N) helps less and less the higher we go in dimension - e.g. for Simpson's rule err $\propto \frac{1}{N^{\frac{4}{d}}}$. For a standard method if we want 10 points per dimension with $d = 10$ we already need 10^{10} function evaluations - infeasible (e.g. high-dimensional integrations in Machine Learning).

[12] $\int_a^b f(x)dx \approx \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$ which can be derived by approximating the function f by a quadratic polynomial and integrating this polynomial.

On the other hand in Monte Carlo integration the error scales with $\frac{1}{N^{\frac{1}{2}}}$ independent of the dimensionality d.

Starting at what dimension of the integral d will MC integration have a more favorable error scaling compared to Simpson's rule? Setting $\frac{1}{N^{\frac{4}{d}}} = \frac{1}{N^{\frac{1}{2}}}$ yields that starting at $d = 8$ Monte Carlo Integration's error will reduce more quickly if we increase N the number of points (/ function evaluations) used for the integration.

Intuition for the better scaling One intuition for this advantage of Monte Carlo is that the higher the dimension, the more evenly pairs of random points are spread with respect to their pair-wise distance - normally known as the curse of dimensionality, illustrated in Fig. 30.

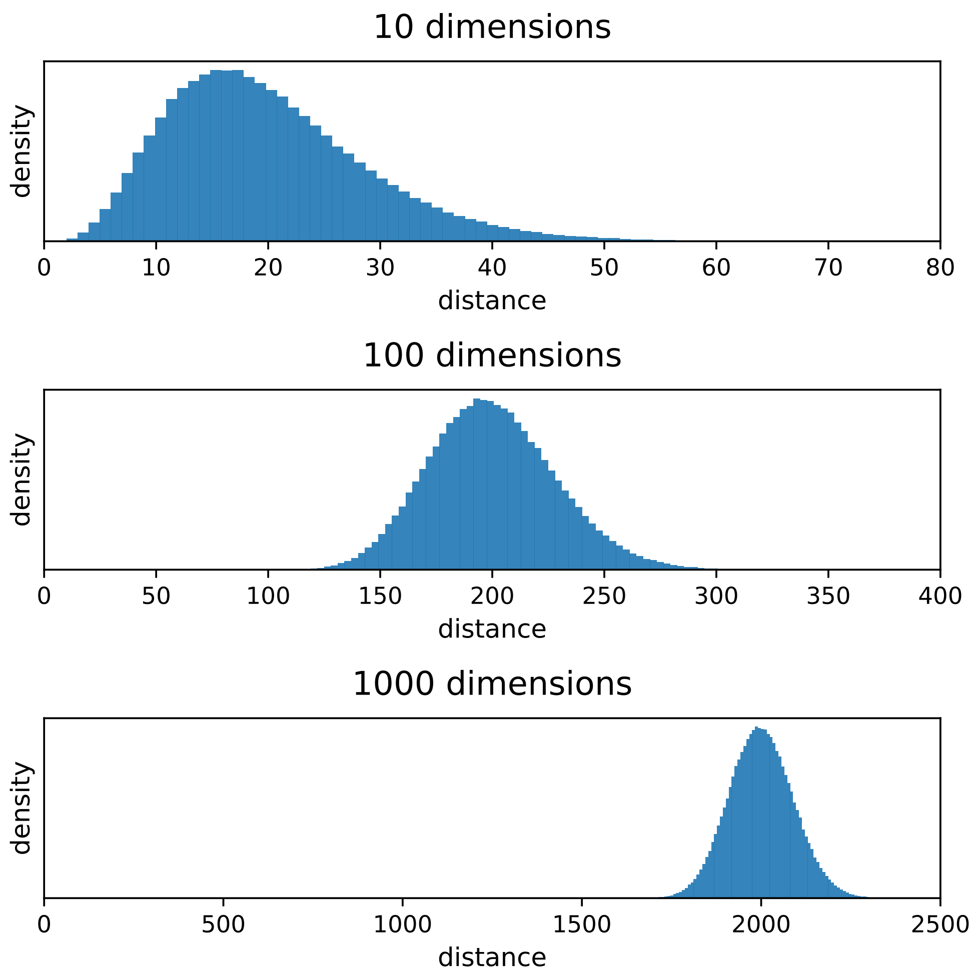


Figure 30: In higher dimensions, pairwise distances between random points show a sharper distribution around the mean distance (a more even distribution in space)

A.10.8 Reducing the variance of the Monte Carlo Estimator

A.10.8.1 Antithetic Estimators*

Two random variables with the same distribution on the same probability space are called *antithetic*, if their covariance is negative. We can use such antithetic variables to reduce the variance of a Monte Carlo estimator.

This motivates the following ansatz

$$\begin{aligned} I &:= \int_0^1 g(x)dx, \quad \hat{I}_{anti} = \frac{\hat{I}_{N/2} + \tilde{I}_{N/2}}{2} \\ \hat{I}_{N/2} &= \frac{1}{N/2}\sum_{i=1}^{N/2} g(u_i), \quad \tilde{I}_{N/2} = \frac{1}{N/2}\sum_{i=1}^{N/2} g(1-u_i), \quad u_i \sim \mathcal{U}(0,1) \end{aligned} \tag{294}$$

with

$$\mathrm{Var}[\hat{I}_{anti}] = \frac{1}{4}\Big(\mathrm{Var}[\hat{I}_{N/2}] + \mathrm{Var}[\tilde{I}_{N/2}] + 2\,\mathrm{Cov}[\hat{I}_{N/2}, \tilde{I}_{N/2}]\Big) \underbrace{=}_{\text{see eq. 275}} \mathrm{Var}[\hat{I}_N] + \frac{1}{2}\,\mathrm{Cov}[\hat{I}_{N/2}, \tilde{I}_{N/2}] \tag{295}$$

where for g continuous and monotonic, $\mathrm{Cov}[\hat{I}_{N/2}, \tilde{I}_{N/2}]$ is negative, as U and $1-U$ with $U \sim \mathcal{U}(0,1)$ are negatively correlated. So we can reduce the variance compared to an estimator $\hat{I}_N$ using the same number of function evaluation and only half the number of random numbers.

Let us give an intuition why using antithetic evaluation points makes sense for estimating an integral of a monotonic function from 0 to 1. Using uniform antithetic variables we create pairs of points $(x_i, 1-x_i)$, which are symmetric around $x = 0.5$ and as of the monotony of the function g we want to integrate, we obtain a better balance between sampling points where g is large and where g is small, reducing the variance of the estimate.

Example of using Antithetic Estimators

Consider the integral

$$I = \int_0^2 \frac{1}{1+x^2}dx \underbrace{=}_{\text{exact solution}} \arctan(2) \approx 1.107149 \tag{296}$$

Estimates based on the standard Monte Carlo estimator and the antithetic estimator are given in table 3 and illustrated in Fig. 31. A relative reduction in standard deviation of roughly 80% is achieved using the antithetic estimator.

Estimator	Estimate	Standard Deviation	95% Confidence Interval
Standard Monte Carlo	1.096	0.017	[1.0627 1.128]
Antithetic Monte Carlo	1.106	0.0033	[1.099 1.112]

Table 3: Rounded estimates for the integral $\int_0^2 \frac{1}{1+x^2}dx$ using the standard Monte Carlo estimator and the antithetic Monte Carlo estimator for $N = 100$ samples with seed 1234

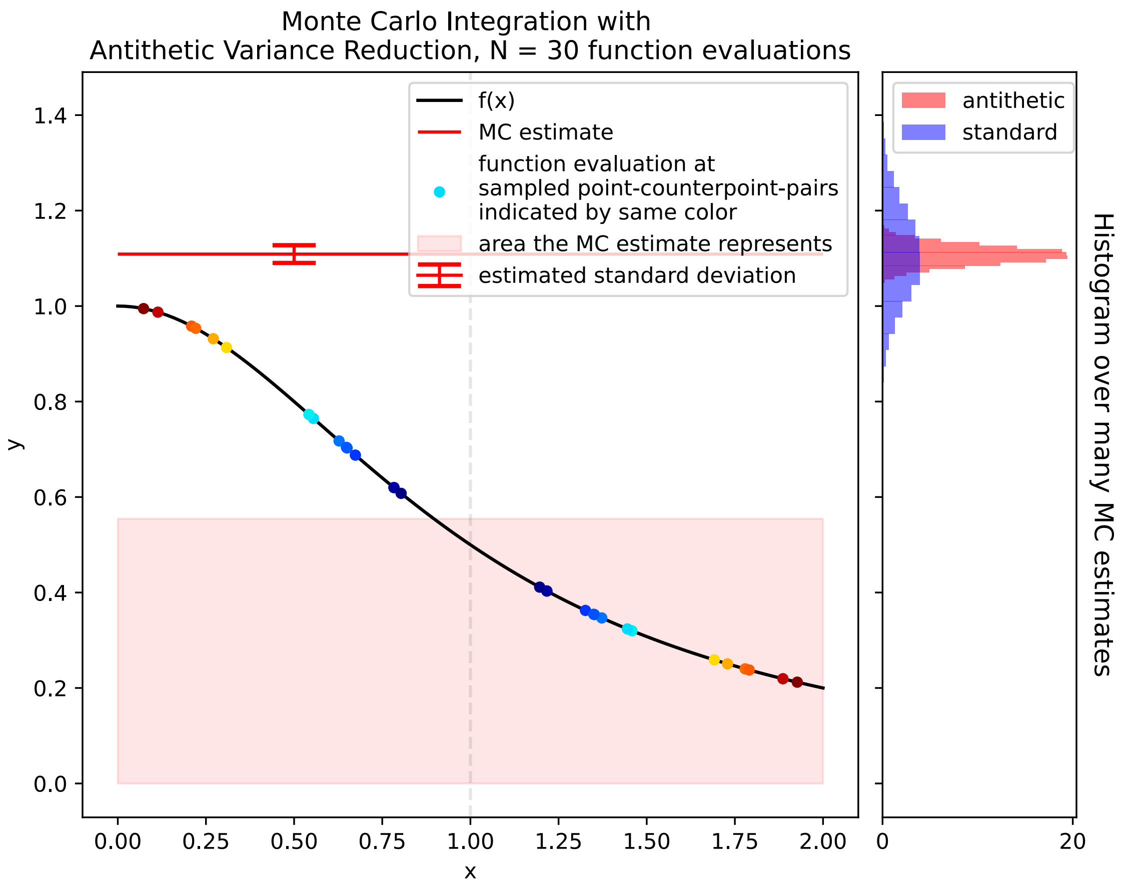


Figure 31: Illustration of Monte Carlo Integration with Antithetic Variables. Notice the even distribution of points around $\frac{a+b}{2} = 1$ by construction yielding an advantage over the standard Monte Carlo estimator.

A.10.8.2 Importance Sampling

Problem: Consider we have a sharply peaked function $g(x)$ over the domain of integration. In this case using evaluation points sampled from the uniform will be inefficient.

Importance sampling idea: Preferably choose points where the integrand is large. This is done by sampling from a distribution $f(x)$ similarly shaped as $g(x)$ but from which we can easily sample (e.g. by direct inversion[a] or the rejection method).

[a]Note that for the inverse transform we need the inverse CDF, so we need to integrate the PDF. So choosing $f(x)$ itself as $g(x)$ is non-sense as we would need to integrate $f(x)$ to obtain the CDF.

The higher the variance of the output of the function $g(x)$ over the domain of integration, the higher the variance in our Monte Carlo Estimate for our integral.

$$I := \int_V g(x)dx = \int_V \frac{g(x)}{f(x)} f(x)dx, \quad I_N = \frac{1}{N}\sum_{i=1}^{N} \frac{g(x_i)}{f(x_i)} \tag{297}$$
$$\text{sample } \{x_i\}_{i=1,\ldots,N} \text{ drawn from } f(x) \text{ limited to } V$$

Let us as an example tackle the integral

$$I = \int_0^1 \frac{4}{1+x^2}dx \underbrace{=}_{\text{exact solution}} \pi \tag{298}$$

using the importance sampling function $f(x) = \frac{4-2x}{3}$ on $0 \leq x \leq 1$. We can sample from $f(x)$ using the inverse transform method using the CDF $F(x) = \frac{4x-x^2}{3}$ (with $F(1) = 1$ as we want) so $F^{-1}(u) = 2 - \sqrt{4-3u}$. We can then simply use equation 297 to obtain an estimate.

The importance sampling approach compared to the standard Monte Carlo approach is illustrated in Fig. 32 and 33. There it is easy to see how effectively integrating a flatter function is to our advantage.

Advantage of Importance Sampling: As we can see in Fig. 32 the flatter $h(x) := \frac{g(x)}{f(x)}$ (best $h \propto g$ as achievable in lattice Monte Carlo) (mind the different naming in the figure) the sharper the probability distribution $p(h)$ (a limited range of h values with higher probabilities), the smaller the Monte Carlo error $\sigma_{\hat{I}_N} = \sqrt{\frac{\sigma_h^2}{N}}$.

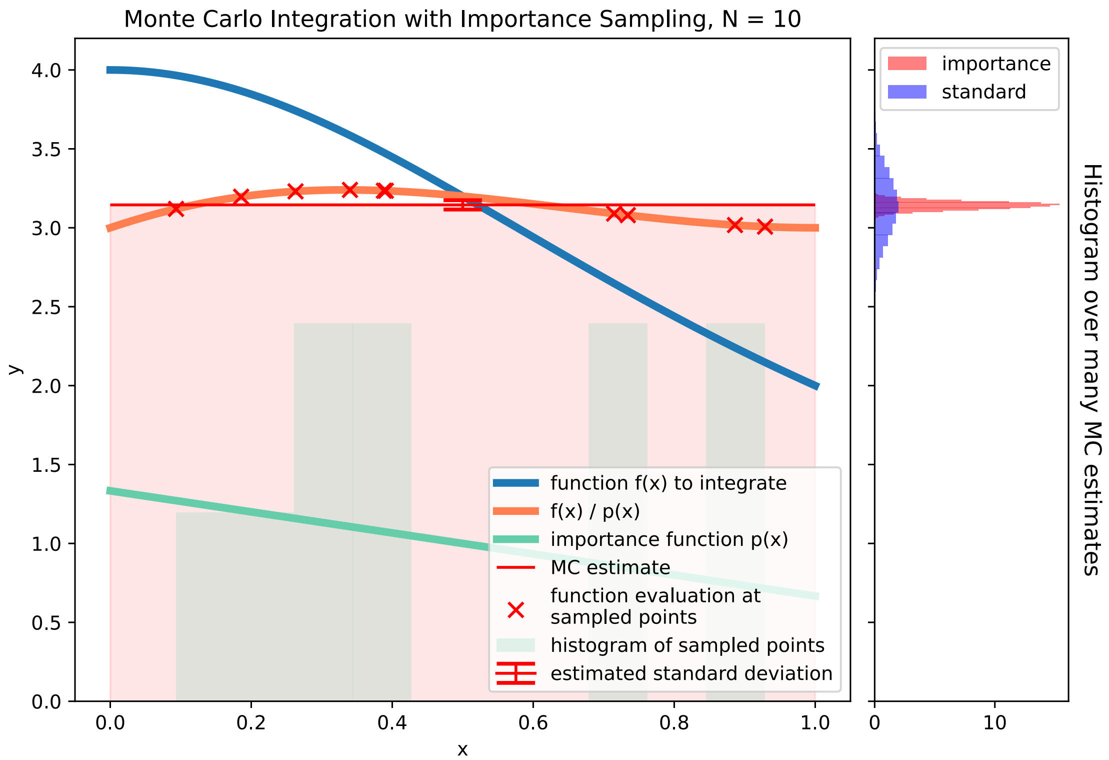


Figure 32: Illustration of Monte Carlo Integration with Importance Sampling for $N = 10$ samples.

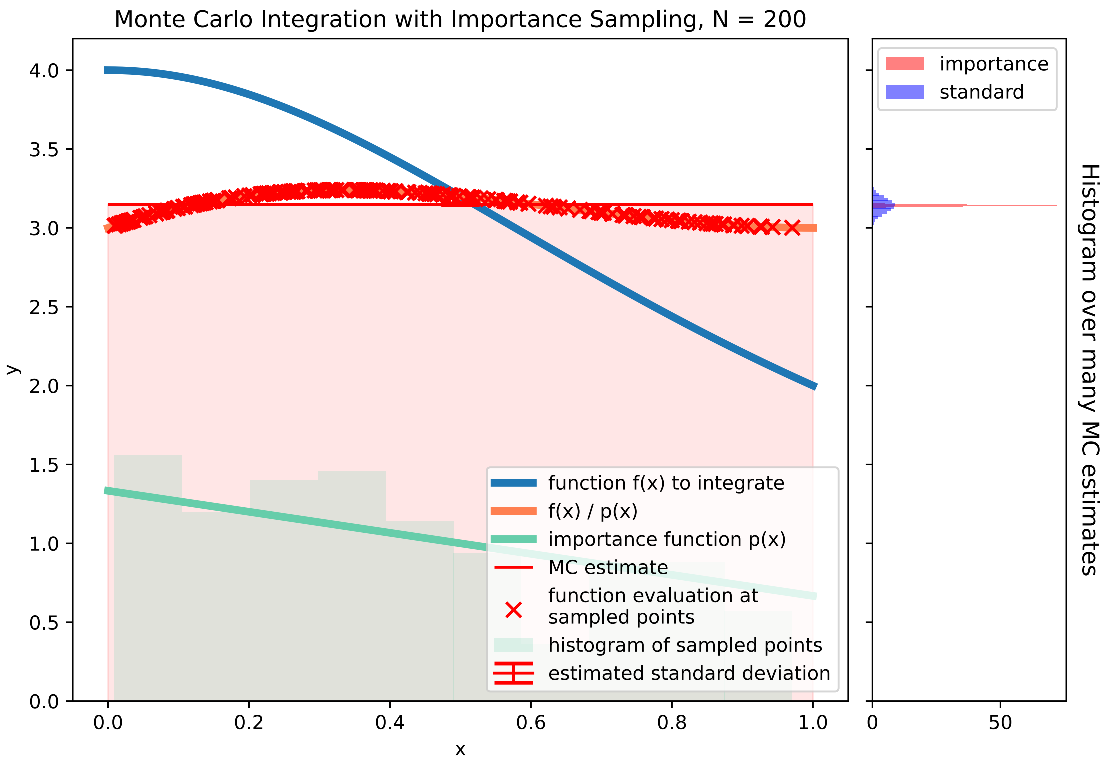


Figure 33: Illustration of Monte Carlo Integration with Importance Sampling for $N = 200$ samples.